

Ingegneria del Software

Corso di Laurea Magistrale in Ingegneria Informatica

UniCenter Documentazione di Progetto

Scilio Alessandra, Platania Simone, Calabrese Alberto

Indice

1	Prima Iterazione	4
1.1	Requisiti	4
1.1.1	Introduzione	4
1.1.2	Casi d'uso	4
1.2	Analisi	9
1.2.1	Introduzione	9
1.2.2	Modello di dominio	9
1.2.3	Caso d'uso UC8 – Immatricolazione Studente	11
1.2.4	Caso d'uso UC1 – Gestione Appelli	13
1.2.5	Caso d'uso UC2 – Iscrizione ad un Appello	15
1.3	Progettazione	17
1.3.1	Introduzione	17
1.3.2	Caso d'uso UC8 – Immatricolazione, diagrammi di interazione . . .	17
1.3.3	Caso d'uso UC1 – Gestione Appelli, diagrammi di interazione . . .	19
1.3.4	Caso d'uso UC2 – Iscrizione ad Appello, diagrammi di interazione .	21
1.3.5	Diagramma delle classi di progetto	23
1.4	Testing	25
1.4.1	Strategia di Testing	25
1.4.2	Dettaglio della Test Suite per Caso d'Uso	26
2	Seconda Iterazione	30
2.1	Requisiti	30
2.1.1	Introduzione	30
2.1.2	Casi d'uso	30
2.2	Analisi	35
2.2.1	Modello di Dominio	35
2.2.2	Caso d'uso UC3 – Accettare/Rifiutare Voto	37
2.2.3	Caso d'uso UC4 – Creazione Corso di Laurea	40
2.2.4	Caso d'uso UC5 – Creazione materie e Finalizzazione Corso di Laurea	42

2.2.5	Caso d'uso UC7 – Invio Comunicazioni	45
2.3	Progettazione	47
2.3.1	Introduzione e Refactoring	47
2.3.2	Caso d'uso UC3 – Accettare/Rifiutare Voto, diagrammi di interazione	49
2.3.3	Caso d'uso UC4 – Creazione Corso di Laurea, diagrammi di interazione	51
2.3.4	Caso d'uso UC5 – Creazione materie e Finalizzazione Corso di Laurea, diagrammi di interazione	53
2.3.5	Caso d'uso UC7 – Invio Comunicazioni, diagrammi di interazione .	56
2.3.6	Diagramma delle Classi di Progetto e Architettura Globale	58
2.4	Testing	60
2.4.1	Strategia di Testing	60
2.4.2	Dettaglio della Test Suite per Caso d'Uso	61
3	Terza Iterazione	66
3.1	Requisiti	66
3.1.1	Introduzione	66
3.1.2	Evoluzione delle Regole di Dominio	66
3.1.3	Casi d'uso	67
3.2	Analisi	71
3.2.1	Introduzione	71
3.2.2	Modello di Dominio	71
3.2.3	Caso d'uso UC6 – Caricamento e Gestione Materiale Didattico . . .	73
3.2.4	Caso d'uso UC9 – Compilazione e Approvazione Piano di Studi . .	76
3.2.5	Caso d'uso UC10 – Consultazione e Download Materiale Didattico .	78
3.3	Progettazione	81
3.3.1	Introduzione e Refactorings rispetto all'Iterazione 2	81
3.3.2	Caso d'uso UC6 – Gestione Materiale Didattico, diagrammi di interazione	83
3.3.3	Caso d'uso UC9 – Compilazione e Approvazione Piano di Studi, diagrammi di interazione	85
3.3.4	Caso d'uso UC10 – Consultazione e Download Materiale, diagrammi di interazione	87
3.3.5	Diagramma delle Classi di Progetto e Architettura Globale	89
3.4	Testing	91

3.4.1	Strategia di Testing	91
3.4.2	Dettaglio della Test Suite per Caso d'Uso	92

1. Prima Iterazione

1.1 Requisiti

1.1.1 Introduzione

UniCenter è un'applicazione software progettata per la gestione delle principali attività accademiche di un'università. All'interno dell'ateneo operano diversi attori, ciascuno con specifiche esigenze e responsabilità:

- **Studenti:** interagiscono con il sistema per monitorare e gestire la propria carriera accademica, consultare l'elenco degli appelli d'esame disponibili e procedere alla loro prenotazione o cancellazione.
- **Amministratore:** gestisce la creazione e finalizzazione di corsi di laurea, materie e l'abilitazione ad esse con i professori.
- **Professori:** interagiscono con UniCenter per la pubblicazione, modifica e gestione degli appelli d'esame relativi agli insegnamenti di propria competenza.

1.1.2 Casi d'uso

Nella prima iterazione del progetto sono stati individuati i seguenti casi d'uso:

- **UC8: Immatricolazione Studente**
- **UC1: Gestione Appelli**
- **UC2: Iscrizione ad un Appello**

UC8: Immatricolazione Studente

Attore Primario: Utente (Futuro Studente)

Scenario Principale:

1. L'utente avvia la procedura di immatricolazione.
2. L'utente inserisce i dati anagrafici e accademici: nome, cognome, email, password, corso di laurea prescelto e codice fiscale.
3. L'utente conferma i dati inseriti.
4. Il sistema verifica l'unicità dei dati identificativi (email e codice fiscale) rispetto al registro degli utenti esistenti.

5. Il sistema valida e registra il nuovo studente, generando automaticamente una matricola univoca sequenziale.
6. Il sistema calcola l'importo totale delle tasse universitarie da pagare.
7. Il sistema conferma l'avvenuta immatricolazione mostrando il riepilogo con i dati del nuovo studente registrato.

Scenari Alternativi:

- **Interruzione improvvisa del sistema:**

1. L'utente aggiorna la sessione o riapre il menu principale.
2. L'utente riavvia la procedura dal passo 1, i dati non vengono salvati.

- **Email o Codice Fiscale già presenti nel sistema:**

1. Il sistema rileva che l'indirizzo email oppure il codice fiscale appartengono già a un utente registrato.
2. Il sistema blocca l'operazione segnalando l'errore: *"Immatricolazione fallita. Dati già inseriti"*.
3. Il caso d'uso termina o l'utente può reinserire i dati ritornando al passo 2.

Regole di Dominio:

- Il sistema deve garantire che non esistano due utenti con lo stesso indirizzo email o codice fiscale.
- La matricola deve essere generata in maniera completamente automatizzata e progressiva dal sistema; non è consentito alcun inserimento manuale da parte dell'operatore o dello studente.
- La procedura di immatricolazione è valida esclusivamente all'interno della finestra temporale accademica consentita (dal 1° agosto al 30 settembre).

UC1: Gestione Appelli

Attore Primario: Docente / Professore

Scenario Principale:

1. Il docente seleziona l'opzione per creare un nuovo appello d'esame.
2. Il sistema recupera e mostra l'elenco delle materie associate al docente autenticato.
3. Il docente inserisce il codice identificativo della materia per la quale intende creare l'appello.
4. Il sistema verifica che il docente sia effettivamente abilitato all'insegnamento di tale materia.

5. Il docente inserisce i parametri logistici e temporali dell'appello: data e ora dell'esame, aula assegnata, numero massimo di posti disponibili, eventuale vincolo alfabetico sulla lettera iniziale del cognome (es. "A-L") e data di termine per la chiusura delle iscrizioni.
6. Il sistema valida formalmente e logicamente i dati inseriti (assicurandosi che la data dell'esame sia futura, che il termine di iscrizione sia coerente e antecedente alla data d'esame e che i posti siano strettamente maggiori di zero).
7. Il sistema registra il nuovo appello nella collezione degli appelli d'ateneo.
8. Il sistema conferma l'avvenuta creazione mostrando al docente il codice identificativo univoco generato per l'appello.

Scenari Alternativi:

- **Interruzione improvvisa del sistema o annullamento:**

1. L'utente esce dalla procedura o torna al menu precedente; l'operazione viene annullata senza modifiche allo stato del sistema.

- **Docente non abilitato all'insegnamento della materia (Passo 4):**

1. Il sistema rileva che il codice materia inserito non appartiene alla lista degli insegnamenti assegnati al docente.
2. Il sistema mostra il messaggio d'errore: *"Il codice inserito non è valido. Riprova."*.
3. Il caso d'uso termina oppure l'utente può effettuare un nuovo inserimento.

- **Formato data/ora non valido (Passo 6):**

1. Il sistema rileva un errore di parsing nei formati temporali attesi.
2. Il sistema segnala l'errore e interrompe la creazione dell'appello.

- **Parametri logici dell'appello non validi (Passo 6):**

1. Il sistema rileva che la data dell'appello risiede nel passato, che il termine di iscrizione oltrepassa la data dell'appello oppure che il numero di posti è ≤ 0 .
2. Il sistema mostra il messaggio d'errore corrispondente e l'appello non viene salvato.

Regole di Dominio:

- Un appello può essere pubblicato unicamente per una materia regolarmente assegnata al docente in sessione.
- Il codice identificativo dell'appello deve essere generato in modo sequenziale, univoco e immutabile dal sistema.
- La data dell'appello e la data di scadenza delle iscrizioni non possono risiedere nel passato; la data di scadenza iscrizioni deve precedere o coincidere con il giorno dell'appello.

UC2: Iscrizione ad un Appello

Attore Primario: Studente

Scenario Principale:

1. Lo studente seleziona l'opzione per iscriversi a un appello d'esame.
2. Il sistema recupera e mostra la lista degli appelli prenotabili (filtrati in base alle materie incluse nel piano di studi approvato dello studente e per le quali non risulta già iscritto).
3. Lo studente seleziona e inserisce il codice dell'appello desiderato.
4. Il sistema attiva la catena di validazione per verificare l'idoneità dello studente all'iscrizione.
5. Superati con successo tutti i vincoli, il sistema approva l'iscrizione, aggiunge lo studente all'elenco degli iscritti e decrementa di un'unità i posti disponibili nell'appello.
6. Il sistema genera e invia una notifica automatica allo studente con i dettagli.
7. Il sistema mostra a schermo la conferma di avvenuta iscrizione.

Scenari Alternativi:

- **Nessun appello disponibile o piano non approvato (Passo 2):**
 1. Il sistema rileva che non sono presenti appelli attivi per le materie dello studente oppure che il piano di studi è ancora nello stato di attesa.
 2. Il sistema mostra il messaggio: *"Nessun appello disponibile al momento"*.
 3. Il caso d'uso termina.
- **Materia non presente nel Piano di Studi (Passo 4):**
 1. Il validatore rileva che la materia dell'appello non fa parte del piano di studi approvato dello studente.
 2. Il sistema respinge l'iscrizione notificando il vincolo violato.
- **Posti disponibili esauriti (Passo 4):**
 1. Il validatore rileva che i posti disponibili per l'appello sono ≤ 0 .
 2. Il sistema respinge l'iscrizione con il messaggio: *"Iscrizione rifiutata: posti esauriti"*.
- **Tasse universitarie non saldate (Passo 4):**
 1. Il validatore rileva che lo studente ha tasse universitarie da pagare.
 2. Il sistema respinge la richiesta con il messaggio: *"Iscrizione rifiutata: tasse universitarie non saldate"*.
- **Vincolo alfabetico sul cognome non rispettato (Passo 4):**
 1. Il validatore rileva che l'iniziale del cognome dello studente non rientra nella fascia alfabetica configurata per l'appello (es. "A-L").

2. Il sistema blocca la registrazione con il messaggio di vincolo non soddisfatto.

- **Scadenza termine iscrizioni superata (Passo 4):**

1. Il validatore rileva che la data odierna è successiva alla data limite stabilita per le iscrizioni.

2. Il sistema rifiuta l'iscrizione con messaggio di iscrizioni chiuse.

Regole di Dominio:

- L'iscrizione agli appelli è vincolata all'esecuzione sequenziale di una catena di responsabilità componibile di validatori.
- Affinché la registrazione abbia successo, lo studente deve soddisfare contemporaneamente tutti i criteri: possedere un piano di studi approvato contenente l'insegnamento, essere in regola con il pagamento dei contributi universitari, rispettare i limiti alfabetici sul cognome, agire prima del termine di scadenza e trovare posti liberi.
- Qualsiasi evento di iscrizione, modifica dell'appello o cancellazione deve notificare lo studente interessato.

1.2 Analisi

1.2.1 Introduzione

La fase di analisi svolta nella prima iterazione ha come obiettivo la modellazione concettuale delle entità presenti.

1.2.2 Modello di dominio

Il modello di dominio presenta le entità concettuali del sistema UniCenter e le associazioni che le collegano.

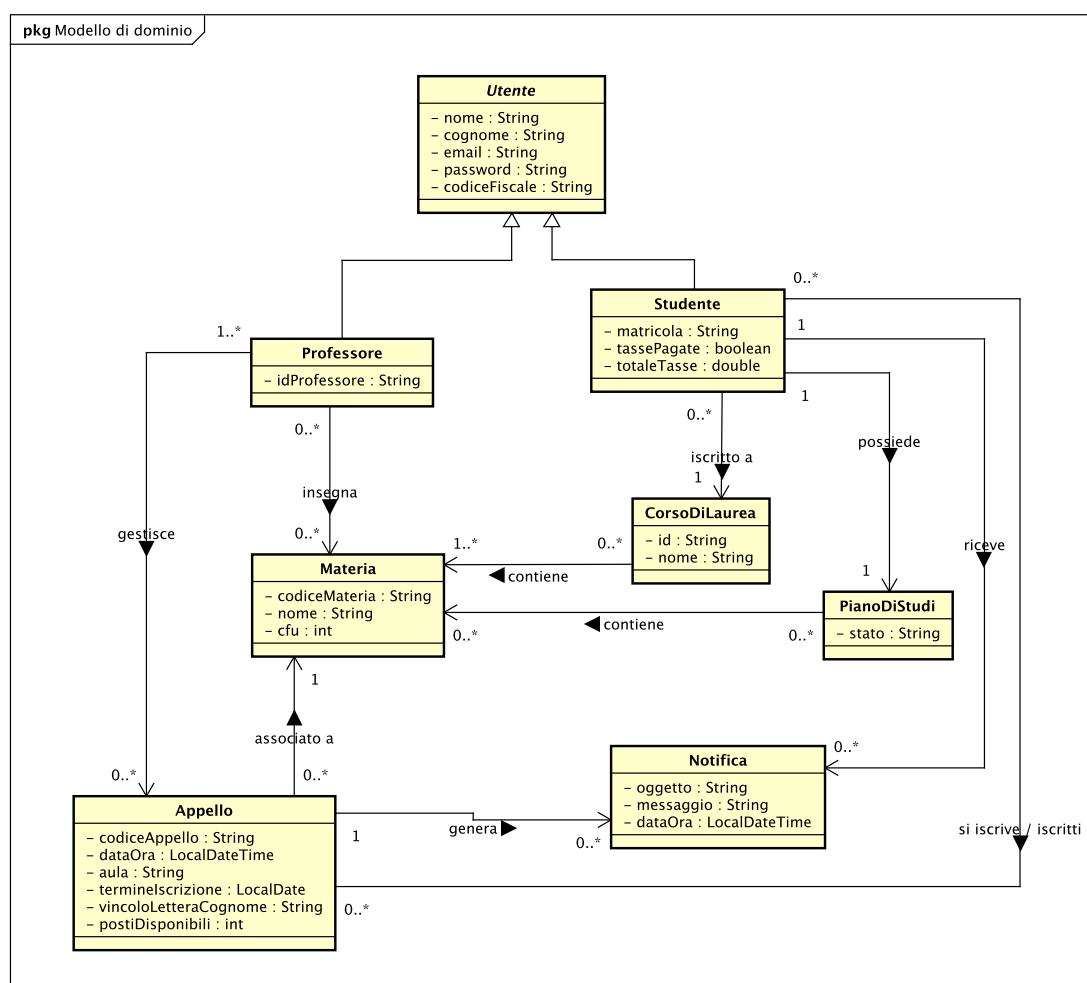


Figura 1.1: Modello di Dominio – Iterazione 1.

Entità Principali e Responsabilità

Le entità del dominio possono essere suddivise in tre aree concettuali:

1. Area Utenti:

- **Utente:** entità astratta di base che racchiude le informazioni anagrafiche.
- **Studente:** specializzazione di **Utente**. È caratterizzato da una **matricola** univoca, dallo stato del debito tasse (**totaleTasse**, **tassePagate**) e dal collegamento al proprio **PianoDiStudi**. Interagisce con il sistema per visualizzare gli appelli prenotabili e ricevere comunicazioni ufficiali tramite una collezione di **Notifica**.
- **Professore:** specializzazione di **Utente** preposta alla gestione degli appelli d'esame per le materie che gli sono formalmente assegnate.

2. Area Didattica:

- **CorsoDiLaurea:** rappresenta l'offerta formativa prescelta dallo studente, aggregando un insieme di **Materia**.
- **Materia:** entità che definisce il singolo insegnamento universitario, identificato da **codiceMateria**, **nome** e crediti formativi universitari (**cfu**).
- **PianoDiStudi:** collezione delle materie approvate nel percorso di ciascun singolo studente.

3. Area Esami e Notifiche:

- **Appello:** esame programmato per una specifica materia. Contiene informazioni logistiche e temporali (**dataOra**, **aula**, **postiDisponibili**, **termineIscrizione**, **vincoloLetteraCognome**) e mantiene la lista degli studenti regolarmente registrati.
- **Notifica:** messaggio strutturato di sistema (**oggetto**, **messaggio**, **dataOra**) recapitato allo studente.

1.2.3 Caso d'uso UC8 – Immatricolazione Studente

Diagramma di sequenza di sistema (SSD)

Il diagramma di sequenza di sistema per l'UC8 illustra l'interazione tra l'attore esterno (l'utente che richiede l'immatricolazione) e il sistema UniCenter .

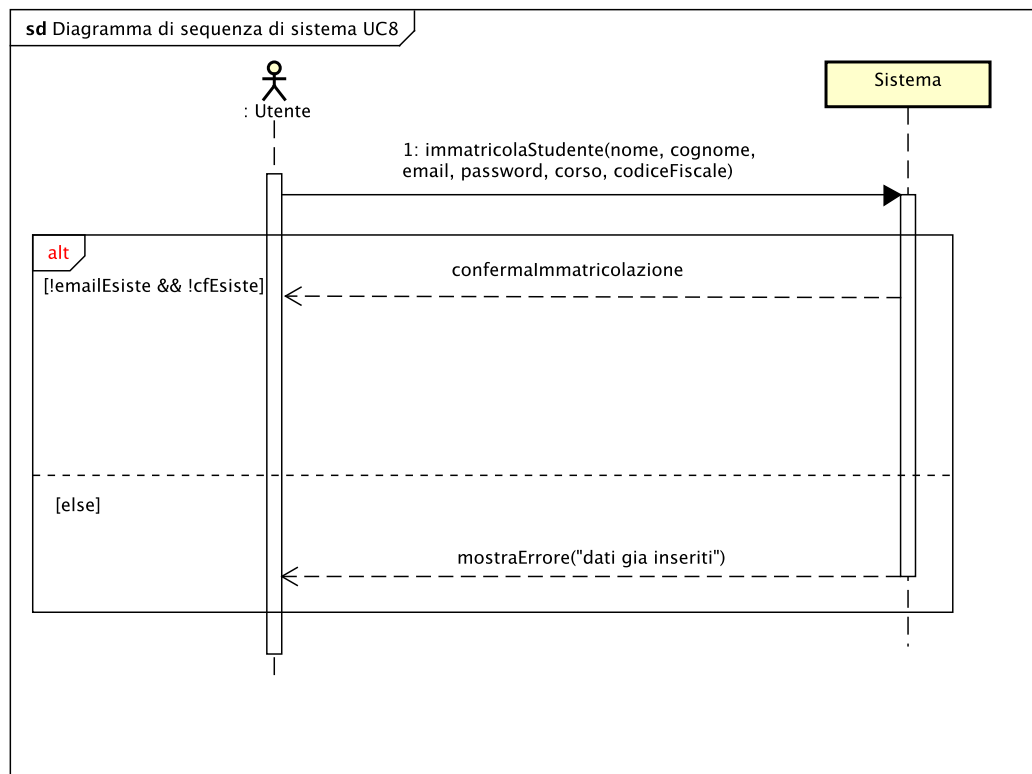


Figura 1.2: Diagramma di sequenza di sistema (SSD) – UC8: Immatricolazione Studente.

L'interazione si articola su un flusso principale con gestione delle alternative:

1. **Richiesta di Immatricolazione (`immatriculaStudente(...)`):** l'utente invia le proprie informazioni anagrafiche, le credenziali, il codice fiscale e il corso di laurea prescelto.
2. **Esito (Blocco alt):**
 - **Ramo di successo:** il sistema registra lo studente, assegna la matricola, calcola le tasse iniziali e restituisce la conferma dell'immatricolazione con i dati generati.
 - **Ramo alternativo:** in presenza di email o codice fiscale duplicati, il sistema respinge l'istanza restituendo l'errore *"Dati già inseriti"*.

Contratti delle operazioni

Operazione	immatricolaStudente(nome: String, cognome: String, email: String, password: String, corso: String, codiceFiscale: String)
Riferimenti	UC8: Immatricolazione Studente
Precondizioni	• L'utente ha avviato la procedura di immatricolazione. • Non esistono nel sistema altre istanze di Utente con la medesima email o lo stesso codiceFiscale. • La data corrente è compresa nella finestra temporale di immatricolazione (1° agosto – 30 settembre).
Postcondizioni	• È stata creata una nuova istanza <i>s</i> di Studente. • Gli attributi di <i>s</i> sono stati inizializzati con i parametri anagrafici e accademici forniti. • È stata generata una stringa alfanumerica univoca assegnata all'attributo matricola di <i>s</i>. • È stata creata un'istanza <i>p</i> di PianoDiStudi associata allo studente <i>s</i>. • È stato calcolato l'importo dovuto ed è stato impostato l'attributo totaleTasse di <i>s</i> secondo la strategia tariffaria standard. • L'istanza <i>s</i> è stata aggiunta alla collezione globale degli utenti registrati nel sistema.

Tabella 1.1: Contratto dell'operazione **immatricolaStudente**.

1.2.4 Caso d'uso UC1 – Gestione Appelli

Diagramma di sequenza di sistema (SSD)

Il diagramma di sequenza di sistema per l'UC1 modella il processo con cui un docente autentificato richiede la pubblicazione di un nuovo appello d'esame.

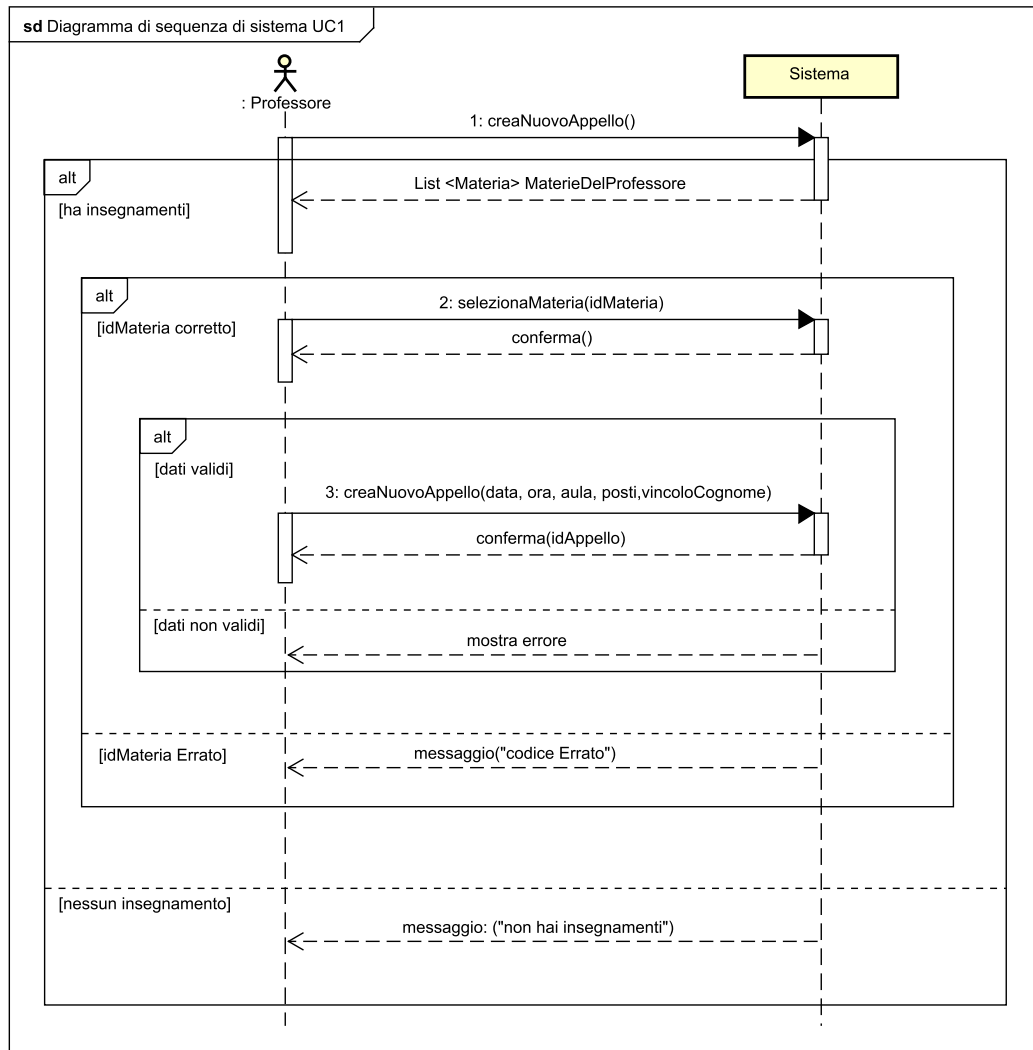


Figura 1.3: Diagramma di sequenza di sistema (SSD) – UC1: Gestione Appelli.

Il flusso informativo si sviluppa nei seguenti passi:

1. **Recupero Insegnamenti:** il docente avvia la procedura; il sistema verifica la presenza di materie assegnate e restituisce l'elenco.
2. **Selezione Materia:** il docente indica la materia prescelta; se valida e assegnata, il sistema conferma la selezione.
3. **Invocazione di Creazione (`creaNuovoAppello(codiceMateria, dataOra, aula, posti, vincoloCognome, termineIscrizione)`):** il docente inserisce i parametri.

Se i dati sono validi, il sistema registra l'appello e restituisce il codice identificativo univoco.

Contratti delle operazioni

Operazione	creaNuovoAppello(codiceMateria: String, dataOra: DateTime, aula: String, posti: int, vincoloCognome: String, termineIscrizione: DateTime)
Riferimenti	UC1: Gestione Appelli
Precondizioni	• Il professore è regolarmente autenticato nel sistema ed è abilitato all'insegnamento identificato da <code>codiceMateria</code>. • Il parametro <code>dataOra</code> è futuro rispetto alla data/ora corrente. • Il parametro <code>termineIscrizione</code> precede o coincide con <code>dataOra</code> ed è futuro rispetto a oggi. • Il valore di <code>posti</code> è strettamente maggiore di zero (> 0).
Postcondizioni	• È stata creata una nuova istanza <i>a</i> di Appello. • Gli attributi di <i>a</i> sono stati inizializzati con i valori logistici e temporali forniti. • È stata generata una stringa alfanumerica sequenziale e univoca assegnata all'attributo <code>codiceAppello</code> dell'istanza <i>a</i>. • L'istanza <i>a</i> è stata associata alla corrispondente istanza di Materia ed inserita nella collezione globale degli appelli.

Tabella 1.2: Contratto dell'operazione `creaNuovoAppello`.

1.2.5 Caso d'uso UC2 – Iscrizione ad un Appello

Diagramma di sequenza di sistema (SSD)

Il diagramma descrive l'interazione tra lo studente e il sistema per effettuare la prenotazione ad un appello d'esame.

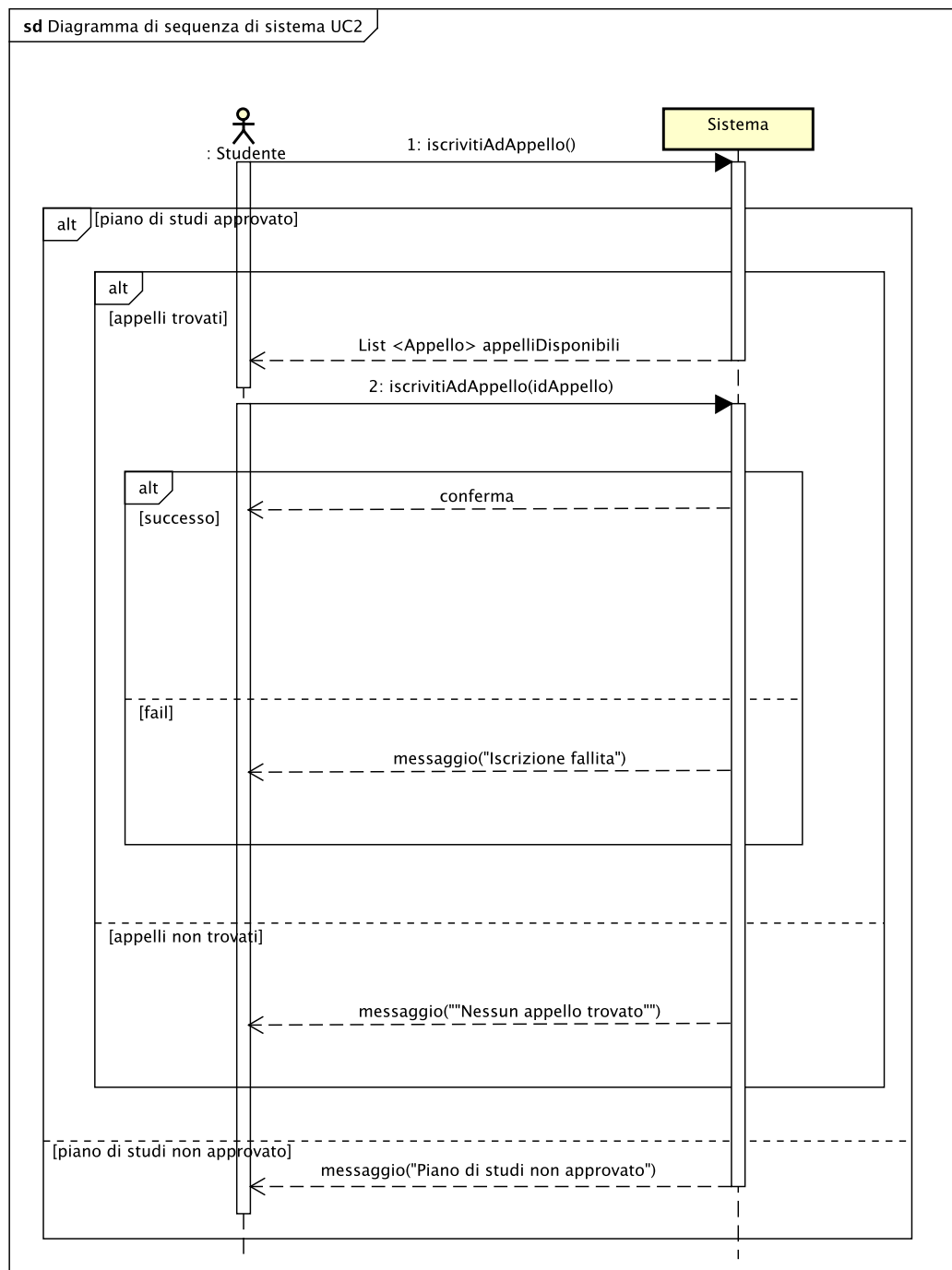


Figura 1.4: Diagramma di sequenza di sistema (SSD) – UC2: Iscrizione ad un Appello.

L'interazione si sviluppa attraverso una sequenza di interrogazione e selezione vincolata:

1. Inizialmente lo studente richiede la lista delle prove d'esame disponibili.
 - **Ramo Piano Non Approvato:** se il piano di studi è ancora in stato di attesa o non approvato, il sistema interrompe immediatamente il flusso restituendo il messaggio *"Piano di studi non approvato"*.
 - **Ramo Piano Approvato:** il sistema verifica la presenza di appelli attivi per le materie inserite a piano. Se presenti, restituisce `appelliDisponibili: List<Appello>`; se non vi sono date utili, notifica l'avviso *"Nessun appello trovato"*.
2. `iscrivitiAdAppello(idAppello)`: Lo studente trasmette il codice identificativo della sessione d'esame prescelta.
 - **Ramo di Successo:** se lo studente soddisfa tutti i requisiti (materia a piano, tasse universitarie saldate, capienza posti > 0 , rispetto della fascia alfabetica del cognome e termini temporali non scaduti), il sistema approva la richiesta, decrementa i posti e restituisce la *conferma* di avvenuta prenotazione.
 - **Ramo di Rifiuto:** qualora anche uno solo dei criteri non risulti adempiuto, il sistema blocca l'operazione senza alterare i posti d'aula e notifica l'errore diagnostico (*"Iscrizione fallita"*, specificando la causa del vincolo violato).

Contratti delle operazioni

Operazione	<code>iscrivitiAdAppello(codiceAppello: String)</code>
Riferimenti	UC2: Iscrizione ad un Appello
Precondizioni	• Lo studente è autenticato nel sistema. • L'appello corrispondente a <code>codiceAppello</code> esiste ed è attivo. • Il piano di studi dello studente si trova nello stato <i>"Approvato"</i> e include la materia dell'appello. • Lo studente è in regola con il pagamento delle tasse universitarie. • Lo studente non risulta già iscritto all'appello specificato. • L'attributo <code>postiDisponibili</code> dell'appello è > 0. • L'iniziale del cognome dello studente soddisfa l'eventuale vincolo di fascia alfabetica dell'appello. • La data/ora corrente non ha superato il <code>termineIscrizione</code> dell'appello.
Postcondizioni	• Lo studente richiedente è stato aggiunto alla collezione degli iscritti dell'istanza <code>Appello</code> selezionata. • Il valore dell'attributo <code>postiDisponibili</code> dell'istanza <code>Appello</code> è stato decrementato di 1. • È stata creata una nuova istanza <i>n</i> di <code>Notifica</code> contenente i dettagli della prenotazione. • L'istanza <i>n</i> è stata associata alla lista delle notifiche personali dello studente.

Tabella 1.3: Contratto dell'operazione `iscrivitiAdAppello`.

1.3 Progettazione

1.3.1 Introduzione

La progettazione software traduce i requisiti e l'analisi concettuale in una struttura a oggetti, guidata dai principi **GRASP** e dai design pattern classici **GoF**.

1.3.2 Caso d'uso UC8 – Immatricolazione, diagrammi di interazione

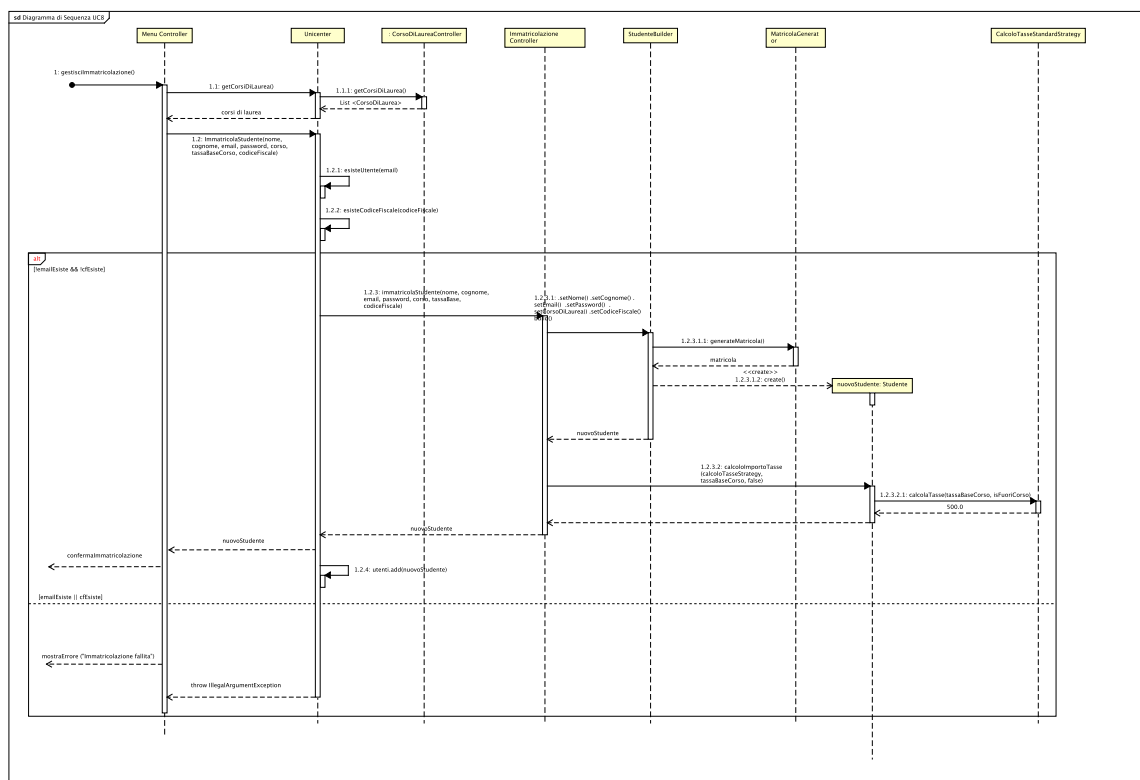


Figura 1.5: Diagramma di Sequenza di Progetto – UC8: Immatricolazione Studente.

Flusso delle Operazioni e Logica Interna

- Verifica dei Duplicati:** Unicenter riceve dal MenuController i dati dello studente e invoca i metodi di controllo unicità `esisteUtente(email)` ed `esisteCodiceFiscale(codiceFiscale)`. Se uno dei due valori è già presente, il flusso si blocca sollevando un'eccezione.
- Delegazione al Controller Specifico:** superati i controlli di unicità, Unicenter delega l'elaborazione applicativa a `ImmatricolazioneController`.
- Validazione e Costruzione dell'Oggetto:** `ImmatricolazioneController` si appoggia alla classe `StudenteBuilder`, la quale valida la correttezza formale di ciascun parametro (assenza di spazi vuoti, formato regex dell'email, robustezza della password).

4. **Generazione Matricola:** durante la fase di costruzione, il builder interroga il generatore `MatricolaGenerator` per ottenere un identificativo della matricola progressivo e univoco.
5. **Calcolo Tasse:** una volta istanziato l'oggetto `Studente`, il controller invoca il calcolo applicando la logica incapsulata nella classe `CalcoloTasseStandardStrategy`.
6. **Persistenza:** lo studente completato viene registrato nella collezione globale degli utenti gestita dal sistema.

Elementi di Design

- **Pattern Controller:** la classe `Unicenter` funge da *Facade Controller* di sistema, mentre `ImmatricolazioneController` agisce come *Use Case Controller* dedicato, garantendo alta coesione e basso accoppiamento.
- **Singleton:** La classe `Unicenter` adotta il pattern Singleton per mantenere un punto di coordinamento univoco della sessione e coordinare i controller operativi del ciclo di vita dell'applicazione. `MatricolaGenerator` e `CodiceAppelloGenerator` sono implementati in modo analogo per incapsulare e garantire in memoria l'univocità e la non-collisione dei progressivi generati.
- **Pattern Builder (`StudenteBuilder`):** separa la validazione e costruzione dei dati anagrafici complessi dall'istanziamento dell'entità, evitando di utilizzare costruttori telescopici. Inoltre è stato adottato per considerare casi futuri (non implementati), in cui gli studenti potrebbero avere diversi campi opzionali, che rappresentano condizioni come: studente lavoratore, studente con disabilità, studente erasmus e così via.
- **Pattern Strategy (`ICalcoloTasseStrategy`):** disaccoppia l'algoritmo di computo delle tasse dalla classe di dominio `Studente`, concretizzando l'**Open/Closed Principle (OCP)** per supportare future estensioni tariffarie (fasce ISEE, studenti lavoratori, agevolazioni di merito).
- **Pure Fabrication (`MatricolaGenerator`):** fabbricazione pura per la generazione deterministica e sequenziale delle matricole universitarie.

1.3.3 Caso d'uso UC1 – Gestione Appelli, diagrammi di interazione

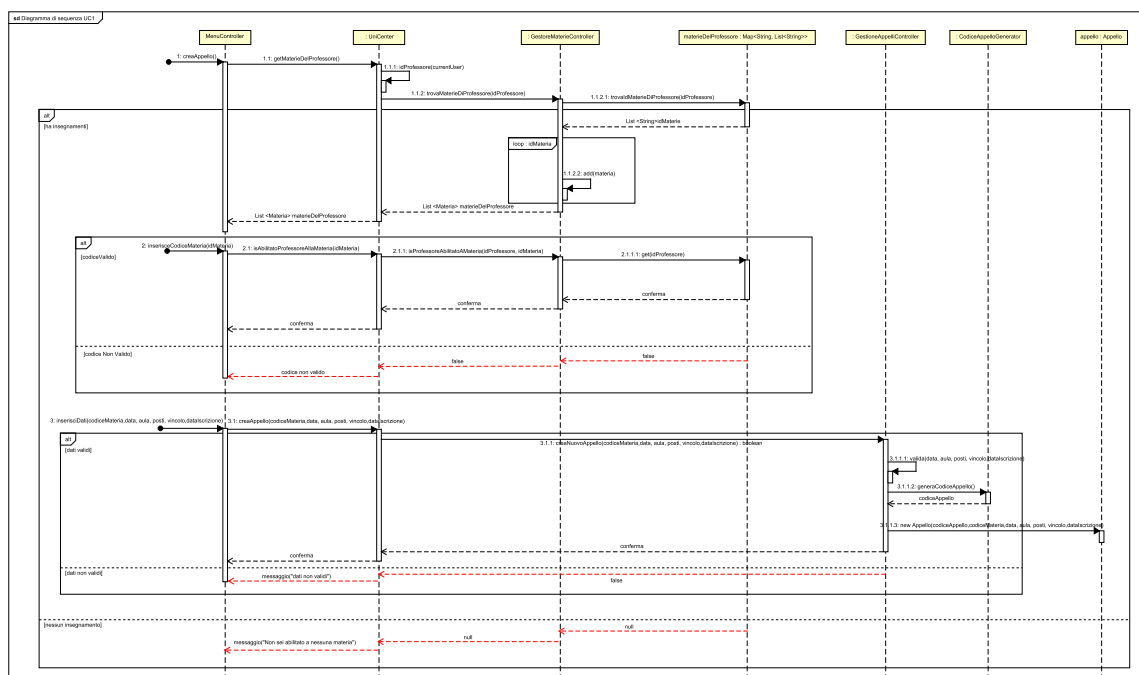


Figura 1.6: Diagramma di Sequenza di Progetto – UC1: Creazione Nuovo Appello.

Flusso delle Operazioni e Logica Interna

1. **Verifica delle Competenze:** il `MenuController` interroga `Unicenter` mediante `getMaterieDelProfessore()` per mostrare gli insegnamenti associati. Alla selezione del codice materia, il metodo `isProfessoreAbilitatoAMateria(...)` delega a `GestoreMaterieController` che restituisce valore booleano.
2. **Delegazione e Validazione:** il `MenuController` trasmette i parametri d'appello (data, aula, posti, vincolo cognome, scadenza iscrizioni) a `Unicenter`, che inoltra la richiesta al `GestioneAppelliController`. Quest'ultimo esegue la validazione interna, controllando scadenze e capienza.
3. **Generazione Codice e Istanziamento:** superata la validazione, il controller ottiene il codice appello da `CodiceAppelloGenerator`, istanzia la nuova entità `Appello` e la memorizza nel registro d'ateneo.

Elementi di Design

- **Pattern Controller:** separazione delle competenze tramite il controller di sistema Unicenter e i controller specializzati `GestioneAppelliController` e `GestoreMaterieController`.
- **Creator & Information Expert:** `GestioneAppelliController` possiede le informazioni per fungere da *Creator* per l'entità `Appello`.

- **Pure Fabrication (CodiceAppelloGenerator):** generazione centralizzata di codici appello univoci.

1.3.4 Caso d'uso UC2 – Iscrizione ad Appello, diagrammi di interazione

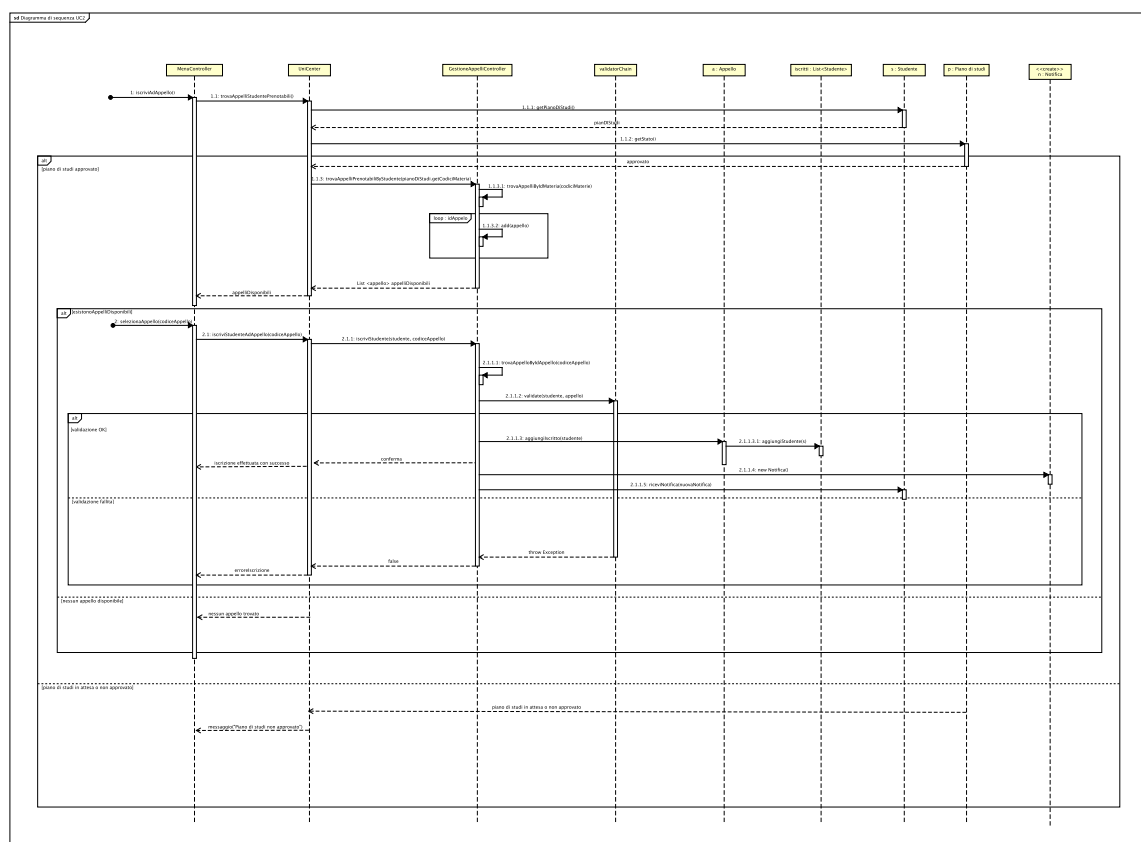


Figura 1.7: Diagramma di Sequenza di Progetto – UC2: Iscrizione ad un Appello.

Flusso delle Operazioni e Logica Interna

1. **Recupero Appelli Prenotabili:** il `MenuController` invoca `trovaAppelliStudentePrenotabili()` su `Unicenter`, che verifica lo stato approvato del piano di studi e delega a `GestioneAppelliController` il recupero degli appelli pertinenti.
2. **Esecuzione della Catena di Validazione:** alla selezione dell'appello, `GestioneAppelliController` sottopone la coppia (`Studente`, `Appello`) alla catena `validatorChain.validate(studente, appello)`. Se un anello rileva una violazione, interrompe il flusso tramite eccezione con messaggio specifico.
3. **Finalizzazione e Notifica:** se tutti i controlli hanno esito favorevole, il controller decrementa i posti, registra lo studente tra gli iscritti e crea una `Notifica` recapitata allo studente tramite l'interfaccia `riceviNotifica(...)`.

Elementi di Design

- **Pattern Chain of Responsibility:** i vincoli di ammissione all'esame non sono implementati come un blocco monolitico di condizioni, bensì distribuiti in classi indipendenti (`PianoStudiValidator`, `PostiDisponibiliValidator`, `TassaPaidValidator`,

`CognomeFasciaValidator`, `DataTermineIscrizioneValidator`) collegate tramite `ValidationChainBuilder`.

- **Pattern Observer:** il recapito della notifica sfrutta il disaccoppiamento tra l'elemento osservato, il generatore dell'evento, e il ricevitore (`Studente`). L'appello sfrutta la lista dei suoi studenti iscritti, che diventano gli observer implementando l'interfaccia `ObserverAppello`.
- **Creator:** `GestioneAppelliController` funge da creator, creando l'istanza della notifica e utilizza il pattern `Observer` per notificare gli utenti, passando l'oggetto.

1.3.5 Diagramma delle classi di progetto

Il diagramma delle classi di progetto sintetizza l'architettura statica del software per la prima iterazione.

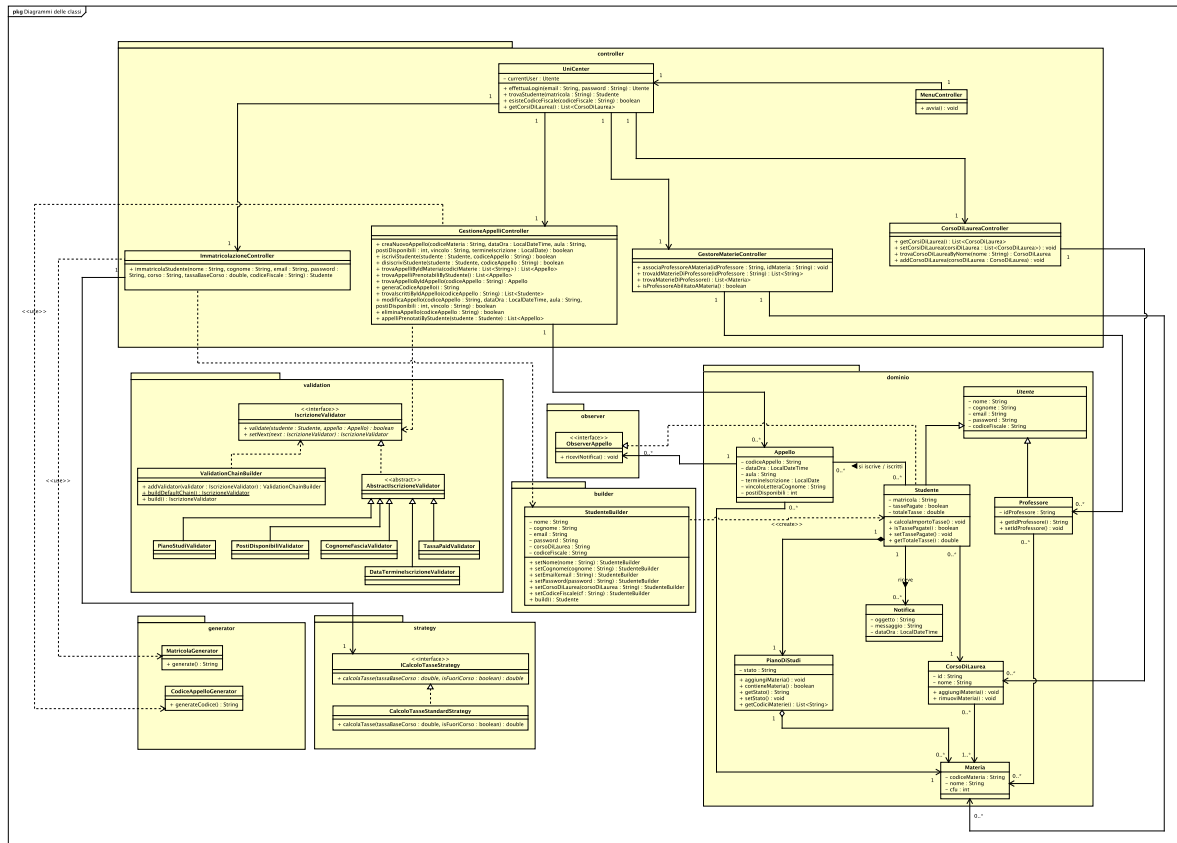


Figura 1.8: Diagramma delle Classi di Progetto – Iterazione 1.

Descrizione del Diagramma delle Classi - Prima Iterazione

- **Package Controller (Pattern Façade & Controller):**
 - **UniCenter** agisce da *Controller principale / Façade*, offrendo un'interfaccia unificata al layer di presentazione (**MenuController**) e coordinando l'accesso ai vari sottosistemi.
 - La logica di business è delegata a controller specializzati: **ImmatricolazioneController**, **GestioneAppelliController**, **GestioneMaterieController** e **CorsoDiLaureaController**.
- **Package Dominio:**
 - **Gerarchia Utenti:** La classe astratta **Utente** modella le informazioni comuni (anagrafica, credenziali), specializzata in **Studente** e **Professore**.
 - **Entità Didattiche:** Modella le relazioni chiave tra **CorsoDiLaurea**, **Materia**, **PianoDiStudi** (in composizione stretta con lo studente) e **Appello** (associato alla materia e a cui gli studenti possono iscriversi).

- **Design Pattern Implementati:**
 - **Chain of Responsibility (validation):** La validazione delle iscrizioni agli appelli è gestita tramite la catena definita da `IscrizioneValidator` e assemblata da `ValidationChainBuilder`. Consente di verificare in sequenza vincoli indipendenti (tasse pagate, posti disponibili, lettera/fascia cognome, piano di studi e data termine iscrizione).
 - **Builder (builder):** `StudenteBuilder` gestisce la creazione graduale e controllata dell'istanza complessa di `Studente` durante la fase di immatricolazione.
 - **Strategy (strategy):** L'interfaccia `ICalcoloTasseStrategy` (con l'implementazione `CalcoloTasseStandardStrategy`) incapsula l'algoritmo di calcolo dell'importo tasse, rendendo la logica facilmente estendibile o sostituibile.
 - **Observer (observer):** `Studente` implementa `ObserverAppello` per ricevere notifiche asincrone (`Notifica`) in caso di aggiornamenti o modifiche relative agli appelli di interesse.
- **Package Generator:** Classi di utilità (`MatricolaGenerator`, `CodiceAppelloGenerator`) per la generazione univoca e isolata degli identificatori di sistema.

1.4 Testing

1.4.1 Strategia di Testing

Durante la prima iterazione, sono stati progettati vari test per verificare la logica di dominio, i contratti operativi e i design pattern fondamentali introdotti nel sistema: la costruzione incrementale e validata delle entità mediante **Builder** (`StudenteBuilder`), la catena di controllo per l'accesso agli esami tramite la **Chain of Responsibility** (`IscrizioneValidator`), il disaccoppiamento del calcolo delle rette universitarie con **Strategy** (`ICalcoloTasseStrategy`) e il meccanismo di notifica agli iscritti basato su **Observer** per le variazioni degli appelli.

La strategia di collaudo del dominio si articola su tre livelli di verifica:

1. **Test Unitari di Dominio e Pattern (JUnit 5 & Mockito):** verifica in isolamento delle regole di validazione dei builder, degli anelli sequenziali della catena di responsabilità, delle strategie di calcolo tasse e dell'integrità dei vincoli di dominio su studenti, corsi di laurea e appelli d'esame.
2. **Test con Mocking Statico e Isolamento Temporale:** collaudo deterministico dei vincoli temporali (finestre di immatricolazione e scadenze di prenotazione) mediante l'intercettazione dei metodi statici di sistema (`mockStatic(LocalDate.class)`), garantendo la riproducibilità dei test indipendentemente dalla data reale di esecuzione.
3. **Test dei Controller di Dominio e Meccanismi Autorizzativi:** validazione dei flussi applicativi coordinati da `ImmatricolazioneController`, `GestioneAppelliController` e `GestoreMaterieController`, certificando i controlli di unicità su email e codici fiscali, le abilitazioni per i docenti e la gestione delle co-docenze.

Sintesi delle Scelte Architetture e di Design nel Testing dell'Iterazione 1

1. **Verifica della Catena di Validazione (Chain of Responsibility Testing):** L'introduzione della catena `IscrizioneValidator` ha richiesto la creazione di una suite di test dedicata (`ValidationChainBuilderTest`) per verificare la costruzione sequenziale degli anelli (`setNext`) e garantire che se un vincolo prioritario fallisce, l'esecuzione della catena si interrompe immediatamente senza interrogare i validatori successivi.
2. **Costruzione Robusta e Normalizzazione Dati (Builder Testing):** La classe `StudenteBuilderTest` isola la logica di assemblaggio dell'entità `Studente`, verificando la validazione puntuale di ciascun parametro (lunghezza minima password, formato email con regex, presenza di caratteri alfabetici nei nominativi) e la normalizzazione automatica dei dati in ingresso (trimming degli spazi e conversione dell'email in minuscolo).
3. **Isolamento Temporale Deterministico tramite Mock Statici:** L'utilizzo di Mockito Inline per il mocking statico di `LocalDate.now()` consente di collaudare sia i casi nominali che i casi limite (giorni di apertura e chiusura delle finestre) sia per le immatricolazioni (1° agosto – 30 settembre) sia per la chiusura dei termini di iscrizione agli appelli d'esame.

4. **Abilitazioni (Gestore Materie Testing):** La suite `GestoreMaterieController-Test` valida la corretta associazione tra corpo docente e insegnamenti, certificando che un professore possa creare appelli esclusivamente per le materie di cui è titolare o co-docente e bloccando tentativi di creazione non autorizzati.
5. **Disaccoppiamento della Logica di Calcolo Tasse (Strategy Testing):** L'adozione dell'interfaccia `ICalcoloTasseStrategy` permette di verificare il calcolo dell'importo dovuto in fase di immatricolazione in totale isolamento, consentendo l'iniezione controllata di implementazioni mockate nei test del controller.

1.4.2 Dettaglio della Test Suite per Caso d'Uso

Di seguito vengono descritte nel dettaglio le singole classi di test relative alla logica di dominio introdotta nella prima iterazione. Al fine di alleggerire la trattazione, per ciascun caso d'uso vengono riportati unicamente **i casi di test più significativi e rappresentativi**, rimandando alla suite di test completa presente nel repository per una visione esaustiva.

Caso d'uso UC8: Immatricolazione Studente e Creazione del Profilo

Questa suite verifica le funzionalità relative all'immatricolazione degli studenti, la validazione anagrafica tramite **Builder**, l'applicazione della finestra temporale e il calcolo della retta con **Strategy** (`ImmatricolazioneControllerTest`, `StudenteBuilderTest`, `UnicenterTest`):

- **`immatricolaStudente_corsoTrovato_creaStudenteConDatiCorretti`:** Verifica il flusso completo di immatricolazione: recupero del corso di laurea, generazione automatica della matricola univoca da parte di `MatricolaGenerator` e impostazione dello stato iniziale con tasse non ancora saldate.
- **`immatricolaStudente_corsoNonTrovato_lanciaIllegalArgumentException` (UC8 validazione corso):** Accerta che il tentativo di immatricolazione a un corso di laurea inesistente sollevi un'eccezione, interrompendo il flusso prima di istanziare il builder dello studente.
- **`immatricolaStudente_emailGiaRegistrata_lanciaIllegalArgumentException` (UC8 unicità account):** Verifica che il sistema impedisca la registrazione di un nuovo studente se l'indirizzo email inserito è già presente nel database.
- **`immatricolaStudente_codiceFiscaleGiaRegistrato_lanciaIllegalArgument-Exception` (UC8 unicità fiscale):** Controlla il vincolo di unicità sul Codice Fiscale, rigettando l'operazione in caso di duplicazione anagrafica.
- **`immatricolaStudente_usaStrategyPerCalcolareTotaleTasse`:** Verifica l'integrazione con il pattern **Strategy**, accertando che il controller deleghi fedelmente il calcolo dell'importo totale a `ICalcoloTasseStrategy`.

- **validaDataImmatricolazione_meseAgosto_ritornaTrue (UC8 finestra temporale valida):** Collauda con `mockStatic(LocalDate.class)` che una data ricadente nella finestra consentita (es. 15 agosto) autorizzi la prosecuzione dell'immatricolazione.
- **validaDataImmatricolazione_meseFuoriFinestra_lanciaDataNonValidaException:** Verifica che richieste di immatricolazione inviate al di fuori del periodo 1° agosto / 30 settembre vengano respinte sollevando `DataNonValidaException`.
- **build_conTuttiICampiValidi_creaStudenteCorretto:** Verifica che `StudenteBuilder` istanzi correttamente l'oggetto `Studente`, normalizzando automaticamente l'indirizzo email in minuscolo e rimuovendo gli spazi superflui.
- **setEmail_formatoNonValido_lanciaIllegalArgumentException:** Valida il controllo sintattico sull'indirizzo email, accertando che stringhe prive di dominio o del carattere @ causino il rigetto immediato.
- **setPassword_menoDiQuattroCaratteri_lanciaIllegalArgumentException:** Verifica il vincolo di sicurezza sulla password, imponendo una lunghezza minima obbligatoria di almeno 4 caratteri.

Caso d'uso UC1: Creazione, Modifica ed Eliminazione degli Appelli d'Esame (Docente)

Questa suite valida le operazioni a disposizione del professore per la gestione degli esami e la notifica di aggiornamento agli iscritti (`GestioneAppelliControllerTest`, `GestoreMaterieControllerTest`, `CorsoDiLaureaControllerTest`):

- **creaNuovoAppello_conParametriValidi_restituisceTrueECreaAppello:** Verifica la corretta creazione di un appello con generazione del codice identificativo univoco, assegnazione dell'aula, data/ora e capienza massima.
- **creaNuovoAppello_professoreNonAbilitatoAllaMateria_lanciaIllegalArgumentException:** Accerta che un docente privo di associazione alla materia d'esame non possa indire appelli, sollevando un'eccezione di sicurezza.
- **creaNuovoAppello_conDataPassata_lanciaDataNonValidaException:** Verifica che il sistema impedisca la pianificazione di appelli d'esame con data antecedente a quella corrente.
- **creaNuovoAppello_conTermineIscrizioneDopoDataAppello_lanciaEccezione:** Valida la coerenza temporale dell'appello, verificando che la data di chiusura iscrizioni debba precedere la data dello svolgimento dell'esame.
- **creaNuovoAppello_conPostiNonPositivi_lanciaPostiNonValidi:** Accerta che la capienza dell'aula specificata sia un intero strettamente positivo, sollevando l'eccezione di dominio `PostiNonValidi` in caso di valore nullo o negativo.
- **creaNuovoAppello_vincoloInvertito_vieneRiordinatoAlfabeticamente:** Collauda l'algoritmo di normalizzazione del vincolo lettera cognome, verificando che una fascia inserita in ordine decrescente (es. "Z-A") venga automaticamente corretta in ordine alfabetico ("A-Z").

- **modificaAppello_conDatiValidi_aggiornaCampiEInviaNotifica (UC1 aggiornamento e Observer):** Verifica che la modifica di data, aula o capienza aggiorni l'appello e recapiti tempestivamente una notifica di variazione a tutti gli studenti già iscritti.
- **modificaAppello_postiInferioriAgliIscritti_lanciaPostiNonValidi:** Controlla che il docente non possa ridurre la capienza dell'aula a un valore inferiore al numero di studenti attualmente prenotati.
- **eliminaAppello_esistente_restituisceTrueELoRimuove (UC1 cancellazione):** Verifica l'eliminazione controllata dell'appello d'esame dal sistema e l'invio della notifica di cancellazione a ciascuno studente prenotato.
- **associaProfessoreAMateria_stessaMateriaAPiuProfessori_funzionaPerEntrambi:** Collauda la gestione delle co-docenze nel `GestoreMaterieController`, consentendo a più professori titolari di gestire appelli per il medesimo insegnamento.

3. Caso d'uso UC2: Prenotazione Appello e Catena di Validazione Iscrizioni (Studente)

Questa suite certifica il processo di iscrizione e disiscrizione agli appelli da parte degli studenti e l'architettura della **Chain of Responsibility** per il filtraggio delle richieste (`GestioneAppelliControllerTest`, `ValidationChainBuilderTest`, classi del package `validation`):

- **iscriviStudente_conValidazioneSuperata_restituisceTrueEInviaNotifica (UC2 flusso nominale):** Verifica che, al superamento di tutti i controlli della catena di validazione, lo studente venga aggiunto all'elenco degli iscritti e riceva la notifica di conferma.
- **iscriviStudente_studenteGiaIscritto_restituisceFalse:** Previene le registrazioni duplicate, impedendo a uno studente di prenotarsi più volte allo stesso appello.
- **iscriviStudente_validazioneFallita_restituisceFalseENonIscrive:** Verifica che, qualora uno dei validatori della catena fallisca, la procedura d'iscrizione venga bloccata senza apportare modifiche allo stato dell'appello.
- **disiscriviStudente_studenteIscritto_restituisceTrueERimuove (UC2 cancellazione prenotazione):** Collauda la rinuncia all'appello da parte dello studente, verificando la corretta rimozione dalla lista iscritti e la notifica di avvenuta disiscrizione.
- **buildDefaultChain_primoAnelloEPianoStudi_falliscePrimaDeiControlliSuccessivi:** Verifica l'ordine della catena di default e la logica *fail-fast*: se la materia non è presente nel piano di studi dello studente, la catena fallisce immediatamente senza consultare posti, tasse, cognome o date.
- **buildDefaultChain_pianoStudiOkMaTasseNonPagate_falliscePrimaDiPostiCognomeEData:** Verifica che l'irregolarità delle tasse (`TassaPaidValidator`) arresti la catena prima di verificare la capienza o le scadenze temporali.

- **validate_cognomeDentroLaFascia_ritornaTrue / validate_cognomeFuoriFascia_lanciaEccezione:** Collauda `CognomeFasciaValidator`, verificando che studenti con iniziale compresa nella fascia (inclusi gli estremi) siano accettati e che quelli esterni vengano respinti con `IllegalStateException`.
- **validate_postiEsauriti_lanciaIllegalStateException:** Valida `PostiDisponibiliValidator`, accertando che il raggiungimento della capienza massima d'aula impedisca nuove prenotazioni.
- **validate_oggiDopoIlTermine_lanciaDataNonValidaException:** Collauda `DataTermineIscrizioneValidator`, verificando che tentativi di iscrizione effettuati dopo la data limite vengano bloccati.
- **trovaAppelliPrenotabiliByStudente_escludeAppelliGiaPrenotati:** Verifica che la ricerca degli appelli prenotabili escluda automaticamente gli esami a cui lo studente risulta già iscritto.

2. Seconda Iterazione

2.1 Requisiti

2.1.1 Introduzione

Nella seconda iterazione, il sistema **UniCenter** compie un'importante evoluzione strutturale. Viene introdotta nel dominio la figura dell'**Amministratore**, responsabile della configurazione dell'offerta accademica attraverso la creazione, strutturazione e finalizzazione dei **Corsi di Laurea** e delle relative **Materie**. Parallelamente, si estendono le funzionalità dedicate ai **Professori**, che possono ora pubblicare gli esiti delle prove d'esame e inviare comunicazioni dirette a tutti gli iscritti ai propri corsi. Infine, gli **Studenti** acquisiscono la possibilità di visionare i voti conseguiti, accettandoli o rifiutandoli entro scadenze temporali vincolanti. L'interazione tra questi processi è scandita da un sistema di **Notifiche**.

2.1.2 Casi d'uso

Nel corso della seconda iterazione sono stati analizzati, progettati e implementati i seguenti quattro casi d'uso cardine:

- **UC3: Accettare/Rifiutare Voto:** Gestione dello stato e della verbalizzazione degli esami sostenuti da parte dello studente.
- **UC4: Creazione Corso di Laurea:** Inizializzazione e validazione di un nuovo percorso accademico da parte dell'amministratore.
- **UC5: Creazione materie e Finalizzazione Corso di Laurea:** Definizione del manifesto degli studi (associazione materie per anno) e consolidamento immutabile del corso di laurea.
- **UC7: Invio Comunicazioni:** Invio di avvisi didattici da parte del docente agli studenti iscritti alla materia.

UC3: Accettare/Rifiutare Voto

Attore Primario: Studente

Descrizione: Lo studente prende visione dei voti pubblicati dai docenti per gli appelli a cui ha partecipato, decidendo se accettare o rifiutare la votazione proposta entro una finestra temporale stabilita.

Scenario Principale:

1. Lo studente visualizza l'elenco dei propri esiti pendenti (in stato *In attesa di conferma*).

2. Lo studente seleziona l'identificativo del verbale relativo all'esame d'interesse.
3. Lo studente invia la conferma di accettazione del voto.
4. Il sistema modifica lo stato dell'esame in *Approvato*.
5. Il sistema registra formalmente l'esame nel **Libretto** universitario dello studente, aggiornandone i crediti formativi (CFU) e la media ponderata.
6. Il sistema genera e invia una notifica asincrona di avvenuta approvazione allo studente e al docente titolare.

Scenari Alternativi:

- **Rifiuto esplicito del voto (Passo 3):** Lo studente sceglie di rifiutare la votazione. Il sistema transiziona lo stato dell'esame in *Rifiutato*, non procede alla registrazione nel libretto e notifica entrambe le parti. L'esito rifiutato non potrà più essere accettato.
- **Termine per l'accettazione scaduto (Silenzio-Rifiuto):** Lo studente tenta di agire su un esito la cui data limite di conferma è già trascorsa. Il sistema applica automaticamente la regola del silenzio-rifiuto, transiziona l'esame nello stato *Rifiutato* e respinge l'azione manuale tardiva.
- **Tentativo di reiterazione:** Qualora lo studente tenti di accettare o rifiutare un esame già consolidato (Approvato, Rifiutato o Bocciato), il sistema blocca l'operazione segnalando che l'esame si trova in uno stato non modificabile.

Regole di Dominio:

- **Esiti Insufficienti:** Qualsiasi prova registrata dal docente con una votazione numerica inferiore a 18/30 assume in modo istantaneo e irreversibile lo stato *Bocciato*. Tale esito non richiede né ammette alcuna azione da parte dello studente e non viene conteggiato nel libretto.
- **Immutabilità dell'Approvazione:** Un esame approvato e inserito nel libretto diventa definitivo e non può in alcun caso essere revocato o rifiutato a posteriori.
- **Finestra di Conferma:** Il tempo a disposizione per l'accettazione o il rifiuto è vincolato a un numero prefissato di giorni solari (tipicamente 7) a partire dalla data di registrazione dell'esito.

UC4: Creazione Corso di Laurea

Attore Primario: Amministratore

Descrizione: L'amministratore inserisce a sistema un nuovo corso di laurea definendone i parametri strutturali di base.

Scenario Principale:

1. L'amministratore avvia la procedura di definizione di un nuovo corso di studi.

2. L'amministratore specifica la denominazione (nome), la tipologia accademica (ad es. *Triennale*, *Magistrale*, *Magistrale a Ciclo Unico*, *Master*) e la durata legale in anni.
3. Il sistema convalida la coerenza formale dei dati immessi rispetto ai vincoli di ateneo.
4. Il sistema genera in modo univoco e deterministico il codice identificativo alfanumerico del corso.
5. Il sistema crea l'istanza del corso di laurea impostando il suo stato iniziale su *Bozza* / *Non Finalizzato*.
6. Il sistema conferma all'amministratore l'avvenuta creazione fornendo il riepilogo dei dettagli.

Scenari Alternativi:

- **Dati incoerenti o non validi (Passo 3):** Se l'amministratore inserisce parametri discordanti (ad es. tipologia *Triennale* con durata di 2 anni, oppure denominazione nulla o vuota), il sistema intercetta l'anomalia segnalando l'errore nei dati inseriti e interrompe l'operazione.
- **Corso omonimo già presente (Passo 3):** Se nel sistema esiste già un corso di laurea attivo con la medesima denominazione, l'operazione viene respinta per prevenire ambiguità nell'offerta didattica.

Regole di Dominio:

- **Generazione Codice Corso:** Ciascun corso riceve un codice univoco.
- **Invisibilità dei Corsi in Bozza:** I corsi di laurea appena creati e non ancora provvisti di manifesto degli studi (*non finalizzati*) sono visibili esclusivamente all'amministratore e rimangono rigorosamente inaccessibili alle immatricolazioni degli studenti.

UC5: Creazione materie e Finalizzazione Corso di Laurea

Attore Primario: Amministratore

Descrizione: L'amministratore definisce il piano di studi del corso di laurea associando le singole materie ai rispettivi anni accademici e procede infine al consolidamento operativo (finalizzazione) del percorso.

Scenario Principale:

1. L'amministratore seleziona dall'elenco un corso di laurea in stato di bozza (non finalizzato).
2. L'amministratore associa una o più materie al corso, specificando per ciascuna l'anno accademico di erogazione (da 1 fino alla durata massima del corso).
3. Il sistema registra le associazioni nella mappa interna del manifesto degli studi del corso.

4. L'amministratore richiede la finalizzazione ufficiale del corso di laurea.
5. Il sistema verifica che il corso contenga almeno un insegnamento valido e commuta il flag di stato *finalizzato* su **true**.
6. Il sistema notifica l'avvenuta finalizzazione; il corso diviene operativo e immediatamente selezionabile dagli studenti in fase di immatricolazione.

Scenari Alternativi:

- **Tentativo di finalizzazione di un corso vuoto (Passo 4):** Se l'amministratore richiede la finalizzazione di un corso a cui non è stata associata alcuna materia, il sistema impedisce l'azione segnalando l'assenza di materie associate al corso.
- **Anno accademico fuori range (Passo 2):** Se l'anno indicato per l'insegnamento è inferiore a 1 o superiore alla durata legale del corso, il sistema respinge l'inserimento con un errore esplicito.
- **Modifica su corso già finalizzato:** Qualsiasi tentativo di associare ulteriori materie a un corso già finalizzato viene bloccato, preservando l'integrità dei piani di studio.

Regole di Dominio:

- **Immutabilità Strutturale:** Una volta finalizzato, il manifesto degli studi di un corso non può più essere alterato. Per dismettere un'offerta formativa non più erogata, l'amministratore può unicamente impostare lo stato del corso su *Obsoleto*, mantenendo inalterata la carriera degli studenti già iscritti.

UC7: Invio Comunicazioni

Attore Primario: Professore

Descrizione: Il docente responsabile di una materia dirama un avviso o una comunicazione didattica a tutti gli allievi iscritti all'insegnamento.

Scenario Principale:

1. Il professore seleziona la materia di proprio interesse tra quelle a lui associate.
2. Il professore inserisce l'oggetto e il testo dettagliato del messaggio.
3. Il sistema verifica che il docente sia effettivamente abilitato all'insegnamento della materia indicata.
4. Il sistema individua l'insieme dei destinatari, cioè gli studenti che hanno la materia inserita nel proprio piano di studi e che non l'hanno ancora superata/verbalizzata.
5. Il sistema crea e recapita una notifica a ciascuno studente individuato.
6. Il sistema invia una notifica di conferma di avvenuto invio al professore mittente con il conteggio dei destinatari raggiunti.

Scenari Alternativi:

- **Docente non abilitato alla materia (Passo 3):** Se il docente tenta di inviare una comunicazione per un insegnamento di cui non ha la responsabilità, il sistema rifiuta l'operazione segnalando che il docente non è abilitato all'insegnamento.
- **Campi testo vuoti o non validi (Passo 2):** Se l'oggetto o il corpo dell'avviso sono nulli o contengono unicamente spazi vuoti, l'invio viene interrotto con messaggio di errore.
- **Materia priva di iscritti attivi:** Se nessun allievo deve sostenere quella materia, il sistema completa l'operazione notificando al docente il recapito a 0 destinatari.

Regole di Dominio:

- **Filtro di Idoneità dei Destinatari:** Gli studenti che hanno già superato l'esame e ne hanno registrato l'esito nel proprio Libretto universitario non devono ricevere ulteriori comunicazioni didattiche relative a quell'insegnamento.

2.2 Analisi

2.2.1 Modello di Dominio

Durante la seconda iterazione, il modello di dominio concettuale di UniCenter è stato arricchito per introdurre le entità necessarie alla gestione dell'offerta formativa, alla valutazione del profitto e alla messaggistica didattica.

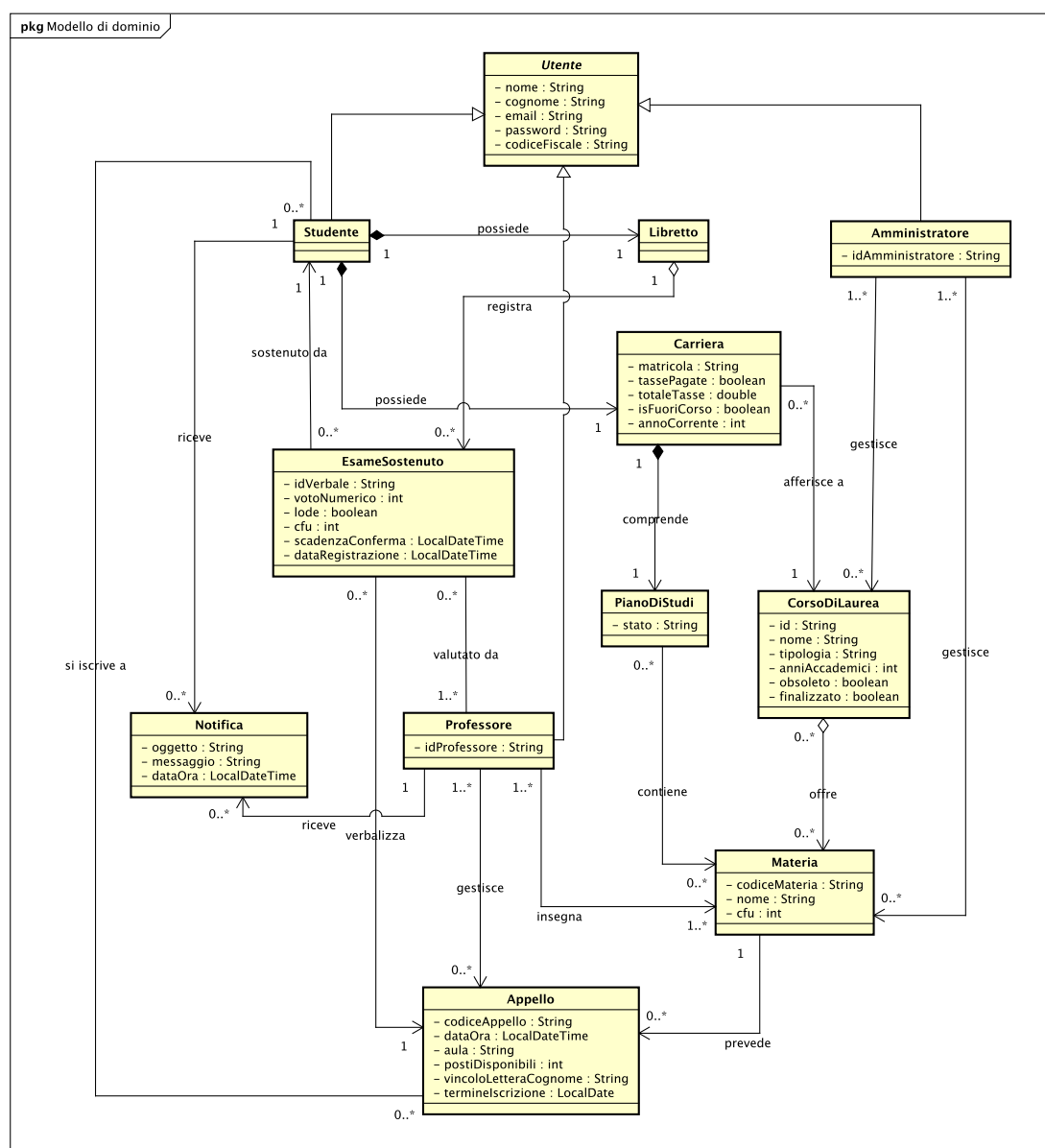


Figura 2.1: Modello di Dominio Concettuale – Seconda Iterazione.

Le entità concettuali del sistema sono strutturate in tre macro-aree logiche:

1. Area Utenti e Amministrazione

- **Utente:** Rappresenta la classe radice della gerarchia degli utenti, incapsulando attributi anagrafici.
- **Studente:** Specializzazione di Utente caratterizzata da una matricola univoca, dallo stato amministrativo delle tasse e dai riferimenti univoci alla propria **Carriera** e al proprio **Libretto**. Riceve e colleziona le **Notifiche** inviate dal sistema.
- **Professore:** Specializzazione di Utente responsabile della gestione delle materie assegnate, della creazione degli appelli d'esame, della valutazione delle prove sostenute (**EsameSostenuto**).
- **Amministratore:** Nuova specializzazione che detiene la responsabilità della configurazione dell'offerta formativa di ateneo, curando il ciclo di vita dei corsi di studio.

2. Area Didattica e Offerta Formativa

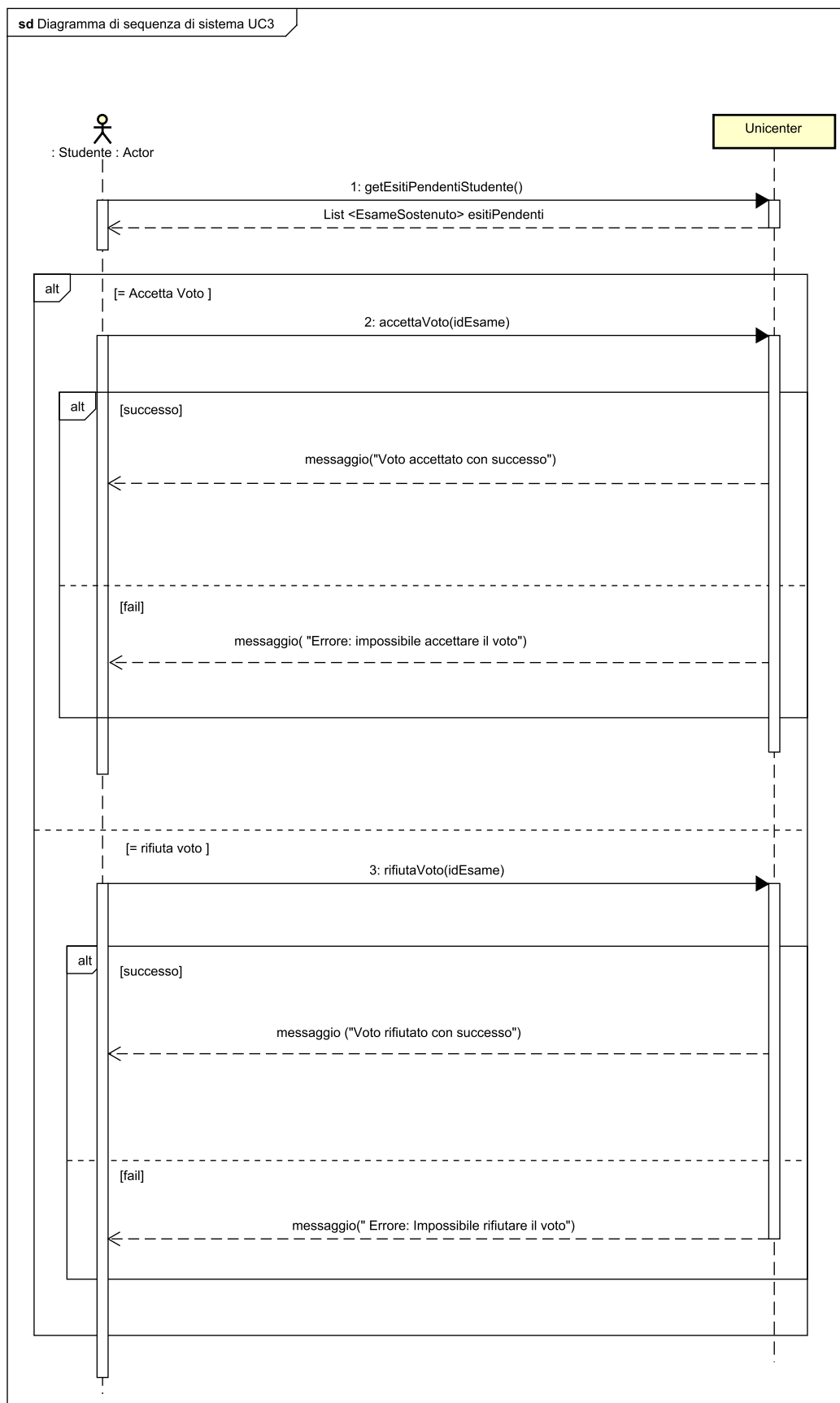
- **CorsoDiLaurea:** Entità centrale dell'offerta didattica. Definisce il nome, la tipologia, la durata in anni accademici e aggrega la mappa delle materie erogate per ciascun anno di corso. Può trovarsi in tre stati (*Bozza*, *Finalizzato*, *Obsoleto*).
- **Materia:** Rappresenta il singolo insegnamento universitario con codice univoco, denominazione e crediti formativi (CFU). Costituisce l'elemento aggregato nel corso di laurea, presente nel piano di studi e oggetto di appelli d'esame e comunicazioni.
- **PianoDiStudi:** Contenitore dei codici delle materie approvate per il percorso curriculare del singolo studente, indispensabile per la convalida dei vincoli di prenotazione e di ricezione avvisi.
- **Carriera:** Traccia lo stato globale dello studente (anno di corso, regolarità tasse, piano di studi associato).

3. Area Esami, Valutazione e Notifiche

- **Appello:** Sessione d'esame definita da data, ora, aula, posti disponibili e vincoli di ammissione (scadenza prenotazioni, scaglioni alfabetici).
- **EsameSostenuto:** Modella l'esito di una valutazione d'esame associata a un appello. Include identificativo del verbale, matricola dello studente, codice materia, docente verbalizzante, voto numerico, eventuale lode, CFU, data di registrazione e termine temporale di scadenza per la conferma. Possiede uno stato esplicito (*In Attesa di Conferma*, *Approvato*, *Rifiutato*, *Bocciato*).
- **Libretto:** Registro accademico ufficiale dello studente che memorizza esclusivamente le istanze di **EsameSostenuto** approvate in via definitiva. È il responsabile del calcolo dei CFU conseguiti e della media ponderata.
- **Notifica:** Messaggio informativo (oggetto, testo, data e ora) istanziato e recapitato in modo asincrono agli attori a seguito di eventi significativi nel sistema.

2.2.2 Caso d'uso UC3 – Accettare/Rifiutare Voto

Diagramma di Sequenza di Sistema (SSD)



Il diagramma descrive l'interazione tra lo **Studente** e **UniCenter**. Il flusso è governato da un costrutto condizionale (*alt*) che dirama l'esecuzione tra l'accettazione e il rifiuto del voto, con sotto-blocchi per gestire l'esito positivo o l'eventuale errore di validazione.

Dettaglio dello scambio di messaggi:

1. **getEsitiPendentiStudente()**: Lo studente richiede la lista dei propri voti pendenti. Il sistema risponde restituendo l'elenco delle istanze in attesa (**elencoVotiPendenti: List**).
2. **Ramo Accettazione (accettaVoto(idEsame))**: Lo studente invia il comando di accettazione. Se la richiesta è lecita e nei termini, il sistema conferma l'esito con *"Voto accettato con successo"*; in caso contrario risponde con un messaggio di fallimento.
3. **Ramo Rifiuto (rifiutaVoto(idEsame))**: Lo studente invia il comando di rifiuto. Se la richiesta è valida, il sistema risponde con *"Voto rifiutato con successo"*; altrimenti segnala l'errore.

Contratti delle Operazioni

Operazione	accettaVoto(idEsame: String)
Riferimenti	UC3: Accettare/Rifiutare Voto
Precondizioni	• Lo studente è autenticato nel sistema. • L'istanza di EsameSostenuto identificata da idEsame esiste nel sistema. • L'esame si trova nello stato <i>"In attesa di conferma"</i>. • La data corrente non ha superato la data di scadenzaConferma (vincolo di silenzio-rifiuto).
Postcondizioni	• Lo stato dell'istanza EsameSostenuto è mutato in <i>"Approvato"</i>. • L'istanza di EsameSostenuto è stata registrata nella collezione degli esami superati del Libretto dello studente. • Sono state create e recapitate nuove istanze di Notifica per lo studente e per il professore verbalizzante. • Eventuali altri esiti pendenti per la medesima materia dello studente vengono neutralizzati.

Tabella 2.1: Contratto di sistema dell'operazione **accettaVoto**.

Operazione	rifiutaVoto(idEsame: String)
Riferimenti	UC3: Accettare/Rifiutare Voto
Precondizioni	• Lo studente è autenticato nel sistema. • L'istanza di EsameSostenuto con identificativo idEsame esiste ed è in stato <i>"In attesa di conferma"</i>.
Postcondizioni	• Lo stato dell'istanza EsameSostenuto è mutato irrevocabilmente in <i>"Rifiutato"</i>. • Nessuna associazione tra l'istanza EsameSostenuto e il Libretto dello studente è stata creata. • Sono state create e recapitate le relative istanze di Notifica allo studente e al professore.

Tabella 2.2: Contratto di sistema dell'operazione **rifiutaVoto**.

2.2.3 Caso d'uso UC4 – Creazione Corso di Laurea

Diagramma di Sequenza di Sistema (SSD)

L'SSD per l'UC4 modella la procedura con cui l'Amministratore inizializza un corso di studi trasmettendo nome, tipologia e durata legale.

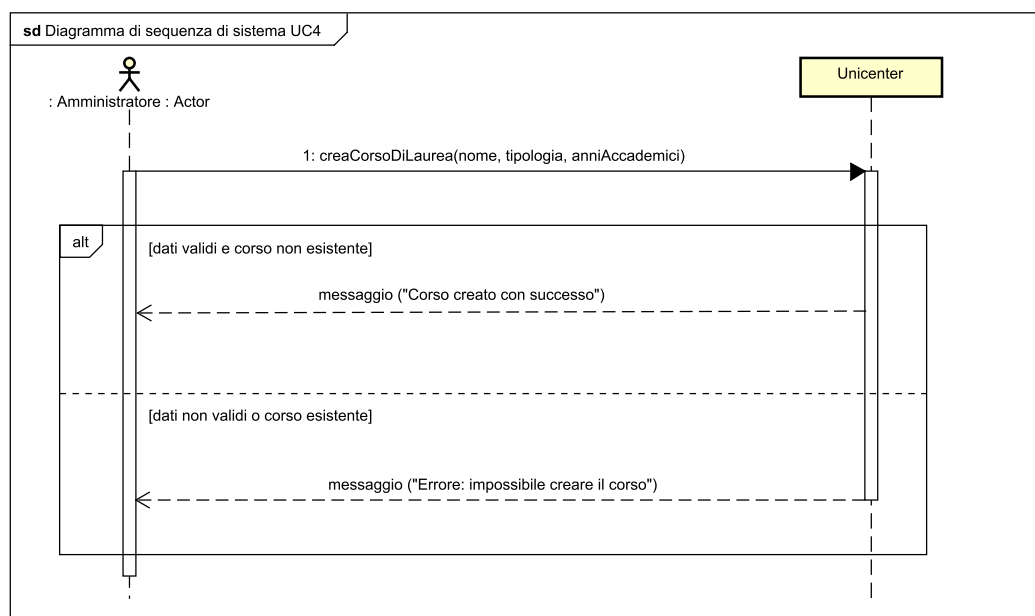


Figura 2.3: SSD per il caso d'uso UC4 – Creazione Corso di Laurea.

Dettaglio dello scambio di messaggi (SSD): Il flusso descrive l'inizializzazione del percorso accademico da parte dell'amministratore:

1. **creaCorsoDiLaurea(nome, tipologia, anniAccademici):** L'attore trasmette i parametri strutturali dell'offerta formativa.
 - **Ramo di Successo [dati validi e corso non esistente]:** se la tipologia accademica rientra tra quelle abilitate dall'ateneo, la durata legale in anni è conforme (es. Triennale a 3 anni, Magistrale a 2 anni) e la denominazione non appartiene a un corso preesistente, il sistema crea l'istanza in stato di *Bozza*, genera il codice identificativo univoco e restituisce il messaggio *"Corso creato con successo"*.
 - **Ramo Alternativo [dati non validi o corso esistente]:** se il nome è nullo, vuoto o già registrato nel catalogo, oppure se la durata è discordante rispetto alla tipologia accademica, il sistema rigetta l'istanza sollevando una segnalazione d'errore (*"Errore: impossibile creare il corso"*) senza salvare record parziali.

Contratti delle Operazioni

Operazione	creaCorsoDiLaurea(nome: String, tipologia: String, anniAccademici: int)
Riferimenti	UC4: Creazione Corso di Laurea
Precondizioni	• L'utente è autenticato con privilegi di <i>Amministratore</i>. • Il parametro nome è valorizzato e non coincide con quello di alcun corso già esistente. • La tipologia appartiene all'insieme delle tipologie valide (<i>Triennale</i>, <i>Magistrale</i>, <i>Magistrale a Ciclo Unico</i>, <i>Master</i>). • Il parametro anniAccademici è congruente con la tipologia specificata.
Postcondizioni	• È stata creata una nuova istanza <i>c</i> di <i>CorsoDiLaurea</i>. • Gli attributi nome, tipologia e anniAccademici di <i>c</i> sono stati inizializzati con i valori forniti. • È stata generata una stringa alfanumerica univoca assegnata all'attributo id di <i>c</i>. • L'attributo finalizzato di <i>c</i> è impostato su false (stato <i>Bozza</i>). • L'attributo obsoleto di <i>c</i> è impostato su false. • L'istanza <i>c</i> è stata registrata nella collezione globale dei corsi di laurea del sistema.

Tabella 2.3: Contratto di sistema dell'operazione **creaCorsoDiLaurea**.

2.2.4 Caso d'uso UC5 – Creazione materie e Finalizzazione Corso di Laurea

Diagramma di Sequenza di Sistema (SSD)

L'SSD per l'UC5 evidenzia un'interazione articolata in due fasi sequenziali: una fase iterativa (blocco *loop*) per associare una o più materie al corso di studi e una fase di consolidamento conclusivo per sancire la finalizzazione del percorso.

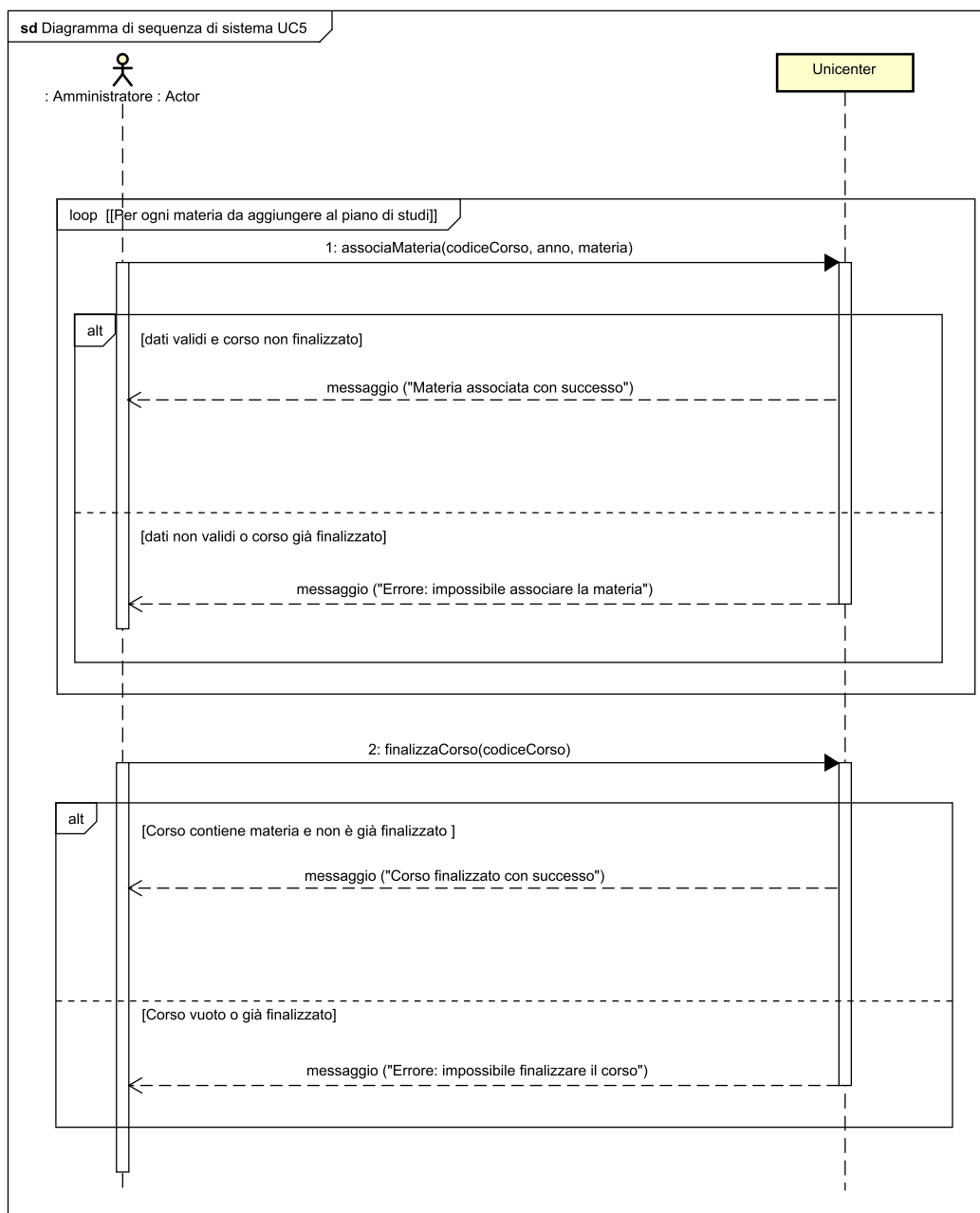


Figura 2.4: SSD per il caso d'uso UC5 – Creazione materie e Finalizzazione Corso di Laurea.

Dettaglio dello scambio di messaggi (SSD): Il protocollo operativo si articola in una fase iterativa di popolamento e una transizione conclusiva di blocco:

1. **Ciclo di Associazione (Blocco loop) – associaMateria(codiceCorso, anno, codiceMateria):** L'amministratore associa sequenzialmente gli insegnamenti al corso.
 - **Ramo Valido:** se il corso si trova ancora in stato di bozza e l'anno indicato è compreso tra 1 e la durata legale prevista, il legame viene memorizzato e il sistema conferma con *"Materia associata con successo"*.
 - **Ramo di Errore:** se l'anno è fuori range o se si tenta di alterare un corso già finalizzato/consolidato, l'azione viene respinta con il messaggio *"Errore: impossibile associare la materia"*.
2. **Consolidamento Ufficiale – finalizzaCorso(codiceCorso):** L'amministratore richiede la chiusura della struttura didattica.
 - **Ramo di Chiusura:** se il corso contiene almeno un insegnamento valido e non è già attivo, il sistema commuta il flag di finalizzazione a `true`, rendendo il manifesto immutabile e restituendo *"Corso finalizzato con successo"*.
 - **Ramo Corso Vuoto o Già Finalizzato:** se il corso è privo di materie o già operativo, il sistema blocca il comando notificando *"Errore: impossibile finalizzare il corso"*.

Contratti delle Operazioni

Operazione	associaMateria(codiceCorso: String, anno: int, codiceMateria: String)
Riferimenti	UC5: Creazione materie e Finalizzazione Corso di Laurea
Precondizioni	• L'utente è autenticato con privilegi di Amministratore. • Esiste un'istanza <i>c</i> di <code>CorsoDiLaurea</code> identificata da <code>codiceCorso</code>. • Il corso <i>c</i> si trova in stato di bozza (<code>finalizzato == false</code> e <code>obsoleto == false</code>). • Il parametro <code>anno</code> soddisfa $1 \leq \text{anno} \leq c.\text{anniAccademici}$. • Esiste un'istanza <i>m</i> di <code>Materia</code> identificata da <code>codiceMateria</code>.
Postcondizioni	• L'istanza <i>m</i> di <code>Materia</code> è stata associata alla chiave <code>anno</code> all'interno della mappa <code>materiePerAnno</code> del corso <i>c</i>.

Tabella 2.4: Contratto di sistema dell'operazione `associaMateria`.

Operazione	finalizzaCorso(codiceCorso: String)
Riferimenti	UC5: Creazione materie e Finalizzazione Corso di Laurea
Precondizioni	• L'utente è autenticato come Amministratore. • Esiste un'istanza <i>c</i> di CorsoDiLaurea con identificativo codiceCorso. • Il corso <i>c</i> ha lo stato finalizzato == false. • La mappa <i>c.materiePerAnno</i> non è vuota (contiene almeno una materia associata).
Postcondizioni	• L'attributo finalizzato di <i>c</i> è mutato in true. • Il corso <i>c</i> diviene immutabile alle modifiche ed entra nella lista dei corsi attivi visibili per le immatricolazioni degli studenti.

Tabella 2.5: Contratto di sistema dell'operazione **finalizzaCorso**.

2.2.5 Caso d'uso UC7 – Invio Comunicazioni

Diagramma di Sequenza di Sistema (SSD)

L'SSD per l'UC7 descrive la trasmissione di comunicazioni didattiche da parte del docente verso la platea degli studenti iscritti.

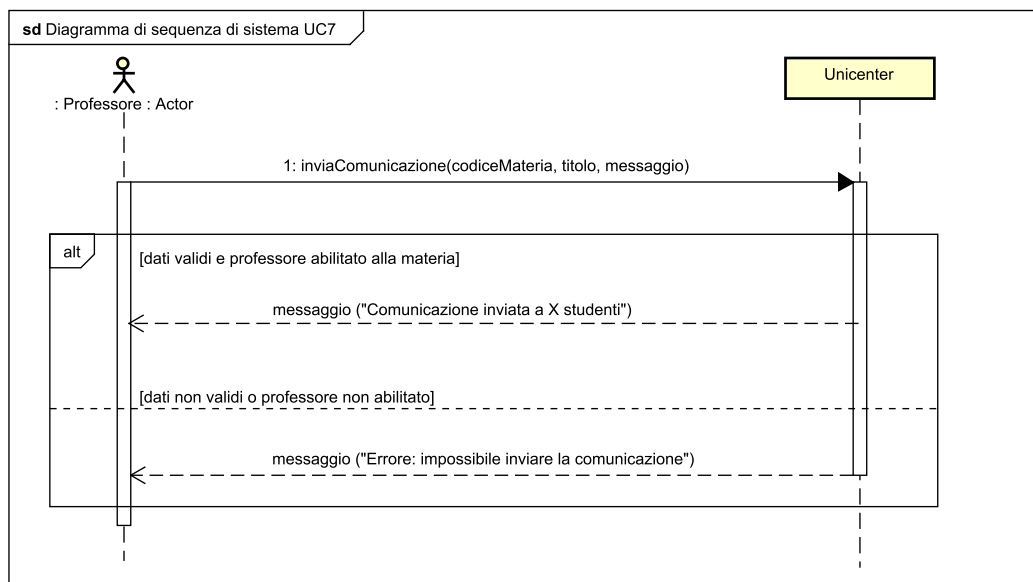


Figura 2.5: SSD per il caso d'uso UC7 – Invio Comunicazioni.

Dettaglio dello scambio di messaggi (SSD): Il flusso modella la trasmissione di circolari didattiche mirate:

1. **inviaComunicazione(codiceMateria, titolo, messaggio):** Il docente titolare invia l'intestazione e il corpo della comunicazione.
 - **Ramo di Successo [dati validi e docente abilitato]:** il sistema accerta che il professore sia effettivamente responsabile della materia indicata e che i testi non siano vuoti. Individua l'insieme degli studenti che possiedono la materia nel piano di studi e che non l'abbiano ancora registrata nel libretto, recapita la notifica asincrona e conferma l'esito al docente mittente (*"Comunicazione inviata a X studenti"*, restituendo $X = 0$ qualora non vi siano allievi con l'esame pendente).
 - **Ramo Non Autorizzato o Dati Nulli:** se il docente tenta di inviare comunicazioni per una materia affidata ad altri colleghi o inserisce parametri composti da soli spazi bianchi, il sistema blocca l'operazione notificando *"Errore: impossibile inviare la comunicazione"*.

Contratti delle Operazioni

Operazione	inviaComunicazione(codiceMateria: String, titolo: String, messaggio: String)
Riferimenti	UC7: Invio Comunicazioni
Precondizioni	• L'utente è autenticato con il ruolo di Professore. • Il professore è ufficialmente assegnato come docente titolare dell'insegnamento codiceMateria. • I parametri titolo e messaggio non sono nulli né composti unicamente da spazi bianchi.
Postcondizioni	• È stata istanziata una Notifica contenente il titolo, il testo formattato e il timestamp corrente. • L'istanza di Notifica è stata aggiunta alla lista delle notifiche di ogni Studente che possiede la materia nel proprio PianoDiStudi e non l'ha ancora superata nel proprio Libretto. • È stata creata un'ulteriore Notifica di conferma di avvenuto invio e recapitata alla lista notifiche del Professore mittente.

Tabella 2.6: Contratto di sistema dell'operazione **inviaComunicazione**.

2.3 Progettazione

2.3.1 Introduzione e Refactoring

La progettazione della seconda iterazione introduce rilevanti **interventi di refactoring** per garantire manutenibilità, basso accoppiamento e alta coesione a fronte dell'incremento di complessità del dominio.

Sintesi dei Refactoring Architetturali e di Design

Rispetto alla prima iterazione, l'architettura è stata significativamente perfezionata attraverso le seguenti scelte strategiche:

1. **Evoluzione della Gerarchia Utenti:** Nella prima iterazione, la gerarchia di **Utente** prevedeva unicamente **Studente** e **Professore**. Per introdurre la gestione dei corsi di studio, è stata introdotta la classe **Amministratore** che eredita da **Utente**.
2. **Refactoring dal Controllo Condizionale allo State Pattern (GoF Comportamentale):** Nella seconda iterazione, per governare il ciclo di vita dell'entità **EsameSostenuto**, è stato introdotto il pattern **State**. La classe delega il comportamento all'interfaccia **IStatoVoto**, declinata negli stati concreti **InAttesaConfermaState**, **ApprovatoState**, **RifiutatoState** e **BocciatoState**. Ciascun oggetto-stato incapsula le sole transizioni ammesse e solleva eccezioni dedicate sui passaggi vietati. Lo stesso principio è stato applicato per il ciclo di vita del **PianoDiStudi** (**IStatoPianoDiStudi**).
3. **Estensione e Potenziamento del Pattern Observer (GoF Comportamentale):** Il meccanismo di notifica asincrona è stato potenziato:
 - Per la gestione degli esami, è stata introdotta l'interfaccia **ObserverEsitoVoto** e il listener concreto **NotificaEsitoObserver**. L'entità **EsameSostenuto** (Subject) non conosce le modalità di consegna della notifica, ma si limita a invocare il listener a ogni cambio di stato.
 - Per l'invio delle comunicazioni (UC7), l'entità **Materia** funge da *Subject/Observable* concettuale per la diramazione mirata dei messaggi. Il controller sincronizza dinamicamente la lista degli studenti destinatari idonei (iscritti con esame ancora pendente), delegando all'entità **Materia** il compito di notificare in modo uniforme e disaccoppiato gli allievi attivi.
4. **Pattern Simple Factory e Pure Fabrication (GRASP/Creazionale):** Per sollevare il controller dalle responsabilità di validazione sintattica e costruzione complessa dei corsi di studio (evitando di violare il principio di *High Cohesion*), è stata introdotta la classe **CorsoDiLaureaFactory**. Essa agisce come *Pure Fabrication* e concretizza una *Simple Factory*: concentra il controllo della coerenza tra tipologia accademica e anni di corso, interagendo con la *Pure Fabrication* **CodiceCorsoGenerator** per produrre istanze di **CorsoDiLaurea** conformi alle regole di business e garantendo basso accoppiamento.

5. **Pattern Strategy (GoF Comportamentale):** L'approccio Strategy, già introdotto per la quantificazione delle tasse universitarie (`ICalcoloTasseStrategy`), è stato generalizzato ed esteso alla valutazione e validazione dei piani di studio tramite l'interfaccia `PoliticaApprovazione` (`ApprovazioneAutomatica` e `ApprovazioneManuale`).
6. **Architettura a Doppio Livello di Controller (GRASP):** Si consolida la netta separazione tra il **Facade Controller** di sistema (`Unicenter`), che gestisce la sessione globale e la sicurezza, e i molteplici **Use Case Controller** dedicati (`GestioneVotoController`, `GestioneCorsiLaureaController`, `InvioComunicazioniController`, `GestoreMaterieController`, `ImmatricolazioneController`), garantendo coesione.

2.3.2 Caso d'uso UC3 – Accettare/Rifiutare Voto, diagrammi di interazione

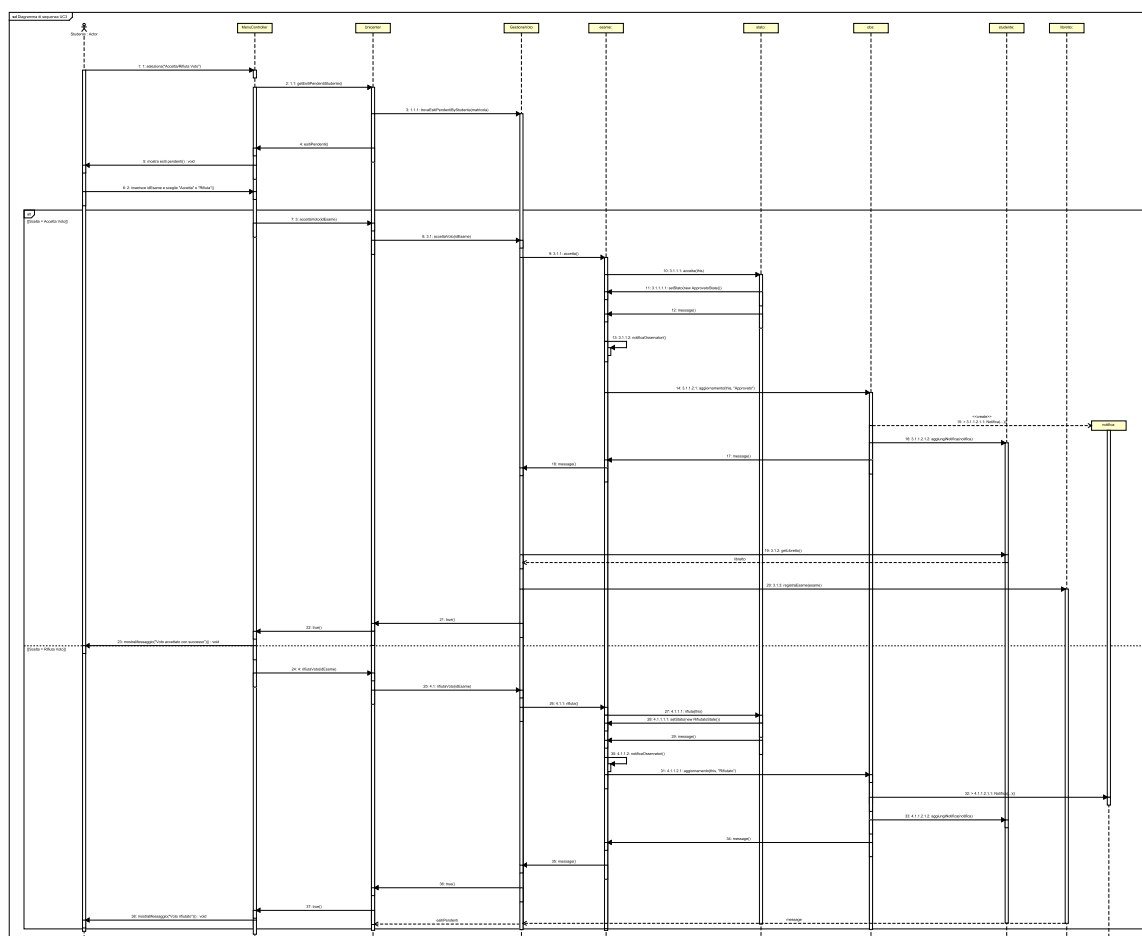


Figura 2.6: Diagramma di Sequenza di Progetto per UC3 – Accettare/Rifiutare Voto.

Flusso delle Operazioni e Logica Interna

Il diagramma di sequenza illustra il flusso di interazione che consente allo studente di visualizzare e gestire (accettare o rifiutare) gli esiti d'esame pendenti.

Inizialmente, l'attore richiede la consultazione dei propri voti pendenti tramite il **MenuController**, il quale interroga la facciata **Unicenter** e, a cascata, il componente **GestioneVotoController** mediante il metodo `trovaEsitiPendentiByStudente`. Gli esiti recuperati vengono quindi mostrati.

Successivamente, lo studente seleziona l'identificativo dell'esame (`idEsame`) e compie la propria scelta:

- **Accettazione del voto:** Il controller inoltra la richiesta di accettazione a **GestioneVoto**, che delega all'istanza di **esame**. Attraverso l'applicazione del pattern *State*, l'esame demanda la transizione all'oggetto `stato` (`accetta()`), aggiornando lo stato interno dell'esame (`setStato()`). Contestualmente, mediante il pattern *Observer*, vengono notificati gli osservatori (`obs`) che generano una nuova **Notifica** associata

allo studente. Infine, **GestioneVoto** recupera il libretto dello studente e provvede alla registrazione dell'esame superato (**registraEsame()**), restituendo l'esito positivo all'interfaccia.

- **Rifiuto del voto:** Il flusso segue una dinamica analoga per quanto concerne la transizione di stato (**rifiuta()**) e l'invio della notifica tramite l'osservatore, omettendo tuttavia la fase di registrazione sul libretto di carriera.

In entrambi i rami alternativi, il controller riceve conferma dell'avvenuta operazione e visualizza un messaggio di riscontro allo studente.

2.3.3 Caso d'uso UC4 – Creazione Corso di Laurea, diagrammi di interazione

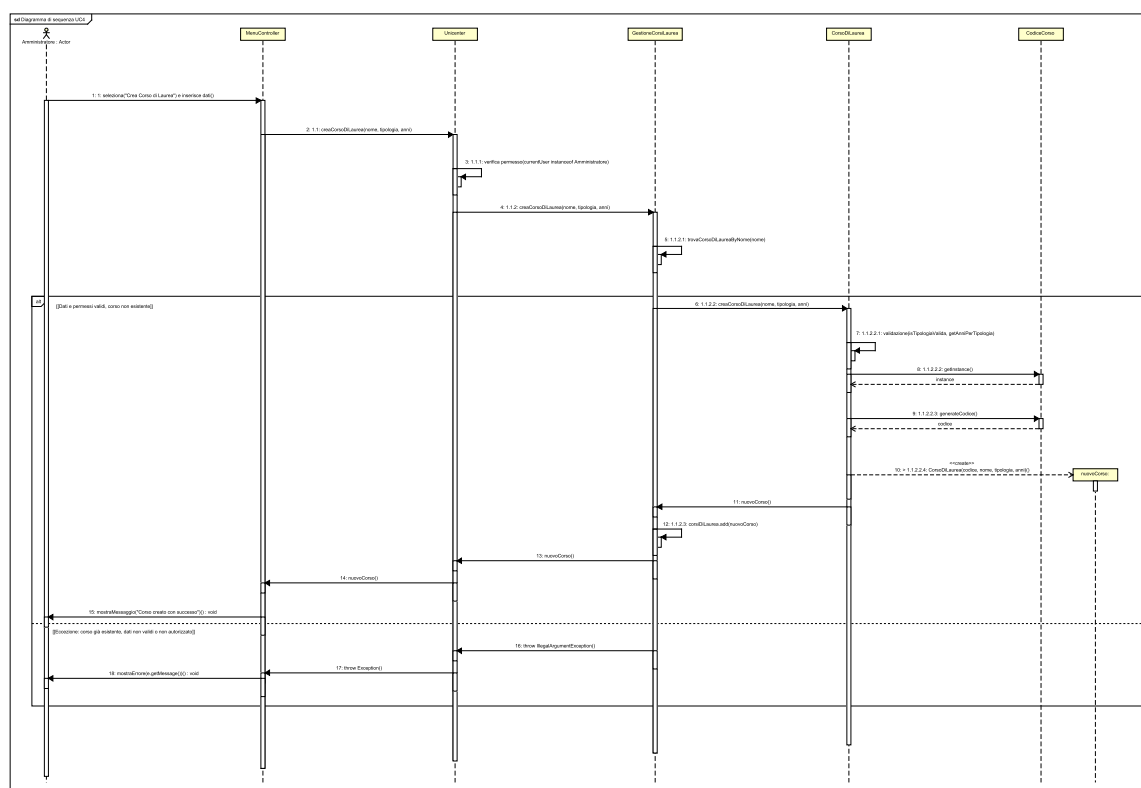


Figura 2.7: Diagramma di Sequenza di Progetto per UC4 – Creazione Corso di Laurea.

Flusso delle Operazioni e Logica Interna

Il diagramma di sequenza descrive il processo di creazione e registrazione di un nuovo corso di laurea all'interno del sistema da parte dell'amministratore.

L'operazione viene avviata dall'attore che seleziona la voce *"Crea Corso di Laurea"* tramite il *MenuController*, fornendo i relativi dati informativi. La richiesta viene inoltrata alla facciata *Unicenter*, la quale esegue innanzitutto una verifica sui permessi di accesso per accertare l'autorizzazione dell'utente, per poi delegare l'operazione al controller di dominio *GestioneCorsiLaureaController*. Quest'ultimo verifica l'univocità del corso mediante una ricerca per nome (*trovaCorsoDiLaureaByNome*).

A questo punto si distinguono due scenari alternativi:

- **Dati e permessi validi (corso non esistente):** Viene invocato il metodo di fabbricazione *creaCorsoDiLaurea* su *CorsoDiLaurea*, che procede alla validazione dei campi. Per assegnare un identificativo univoco al nuovo corso, viene recuperata l'istanza del generatore (*CodiceCorsoGenerator*) e generato il codice identificativo (*generateCodice*). Viene quindi istanziato l'oggetto *nuovoCorso*, il quale viene aggiunto alla collezione dei corsi gestiti da *GestioneCorsiLaureaController* (*corsiDiLaurea.add*). Il riferimento al nuovo corso risale infine la catena dei com-

ponenti fino al `MenuController`, che notifica all'amministratore l'avvenuta creazione con un messaggio di successo.

- **Gestione degli errori (corso già esistente, dati non validi o non autorizzato):** Qualora i vincoli di validazione o di unicità non siano rispettati, il componente `GestioneCorsiLaurea` solleva una `IllegalArgumentException`, propagata a `UniCenter` e intercettata dal `MenuController`, il quale provvede a visualizzare un opportuno messaggio di errore all'amministratore.

2.3.4 Caso d'uso UC5 – Creazione materie e Finalizzazione Corso di Laurea, diagrammi di interazione

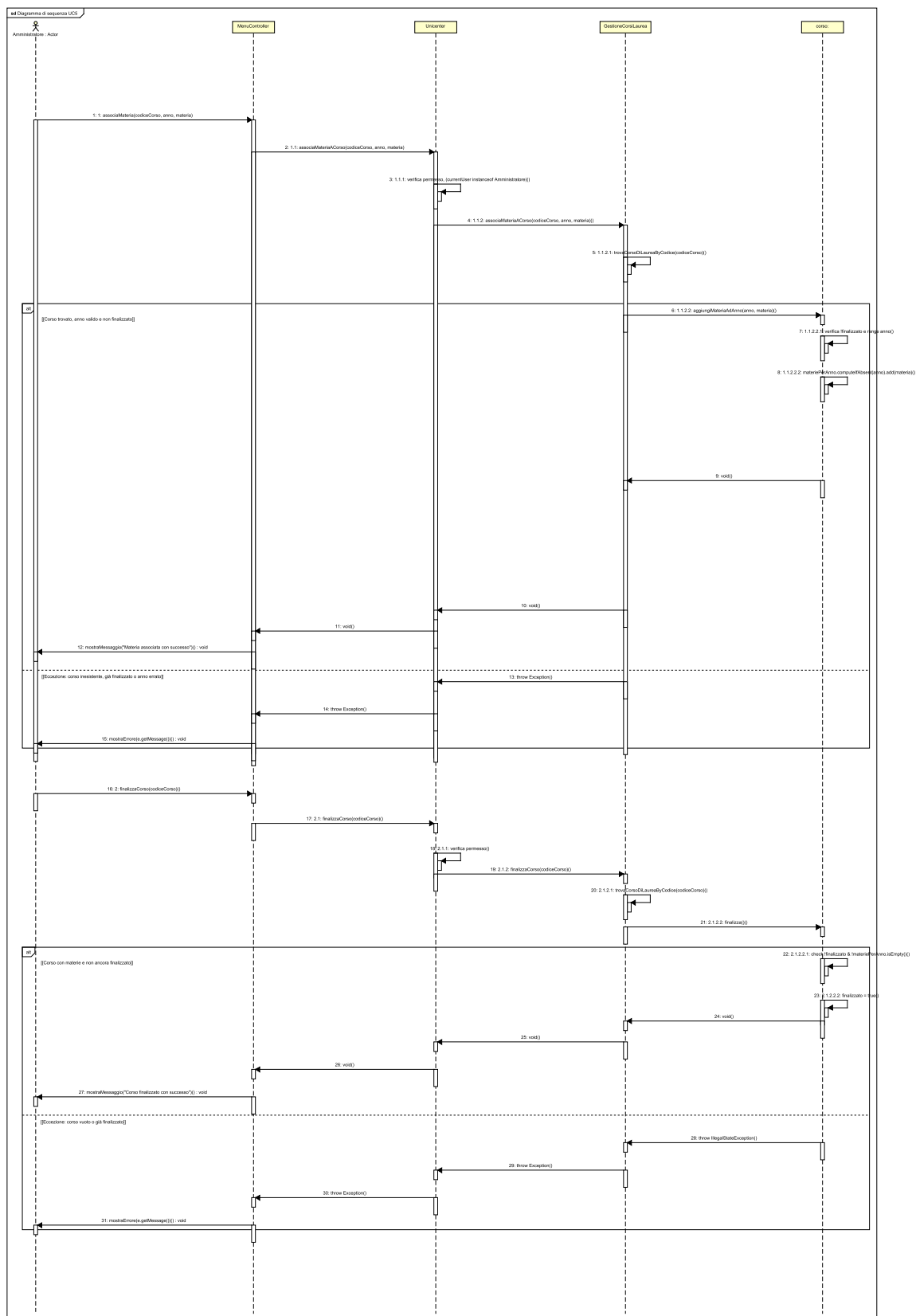


Figura 2.8: Diagramma di Sequenza di Progetto per UC5 – Configurazione e Finalizzazione Corso.

Flusso delle Operazioni e Logica Interna

Il diagramma di sequenza illustra le due fasi consecutive mediante le quali l'amministratore configura il piano di studi di un corso di laurea: l'associazione delle materie ai singoli anni accademici e la successiva finalizzazione del corso.

L'amministratore seleziona l'azione di associazione tramite il `MenuController`, il quale delega la richiesta alla facciata `Unicenter`. Verificati i permessi di amministrazione, la chiamata passa al componente `GestioneCorsiLaureaController`, che recupera l'istanza del corso di interesse (`trovaCorsoDiLaureaByCodice`).

- **Scenario di successo:** Se il corso è presente, non ancora finalizzato e l'anno di corso specificato è valido, l'oggetto `Corso` esegue il metodo `aggiungiMateriaAdAnno`. Verificati i prerequisiti internamente, la materia viene inserita nella struttura dati associativa (`materiePerAnno.computeIfAbsent(anno).add(materia)`). L'esito positivo si propaga fino al `MenuController`, che notifica all'amministratore il messaggio *"Materia associata con successo"*.
- **Gestione delle eccezioni:** Nel caso in cui il corso risulti inesistente, già finalizzato o l'anno sia al di fuori del range consentito, viene sollevata un'eccezione che risale fino al controller dell'interfaccia, mostrando l'errore all'utente.

Una volta completata l'aggiunta degli insegnamenti, l'amministratore richiede la finalizzazione del corso per renderlo definitivo e non più modificabile:

- **Scenario di successo:** Attraverso la medesima catena di invocazione (`Unicenter` e `GestioneCorsiLaureaController`), viene invocato il metodo `finalizza` sull'istanza `Corso`. Quest'ultima verifica che il corso non sia già stato finalizzato e che contenga almeno una materia (`!materiePerAnno.isEmpty()`). Superati i controlli, lo stato del corso viene commutato in finalizzato (`finalizzato = true`) e viene restituito un messaggio di conferma all'amministratore.
- **Gestione delle eccezioni:** Qualora il corso sia privo di materie associate o risulti già finalizzato, viene sollevata una `IllegalStateException`, intercettata e segnalata all'utente tramite apposito messaggio d'errore.

Elementi di Design

- **Information Expert (GRASP):** La classe di dominio `CorsoDiLaurea` detiene la conoscenza completa della propria struttura interna: la durata legale (`anniAccademici`), la mappa degli insegnamenti erogati per anno (`materiePerAnno`) e lo stato evolutivo (`finalizzato`, `obsoleto`). Di conseguenza, è l'unico componente designato a validare la congruenza dell'anno accademico ($1 \leq \text{anno} \leq \text{anniAccademici}$) e a verificare la presenza di almeno una materia prima di autorizzare la finalizzazione.
- **Elevata Coesione:** Il controller applicativo non estrae né manipola direttamente le strutture dati interne dell'entità, ma richiama metodi dedicati che ne esprimono le operazioni (`aggiungiMateriaAdAnno` e `finalizza`). Preserva l'incapsulamento ed evita accoppiamenti.

- **Immutabilità Strutturale:** Una volta commutato lo stato su `finalizzato = true`, l'entità respinge qualsiasi tentativo successivo di aggiungere o rimuovere insegnamenti, sollevando un'eccezione di stato illegale. Questa scelta architetturale tutela l'integrità accademica e la consistenza delle carriere degli studenti già immatricolati, consentendo la dismissione del corso unicamente marcandolo come obsoleto.
- **Separazione delle Competenze:** L'architettura mantiene separata la responsabilità di controllo della sessione e sicurezza (demandata al Facade **Unicenter**) dalla logica di orchestrazione del caso d'uso, incapsulata nello Use Case Controller specializzato **GestioneCorsiLaureaController**, garantendo basso accoppiamento e alta riusabilità.

2.3.5 Caso d'uso UC7 – Invio Comunicazioni, diagrammi di interazione

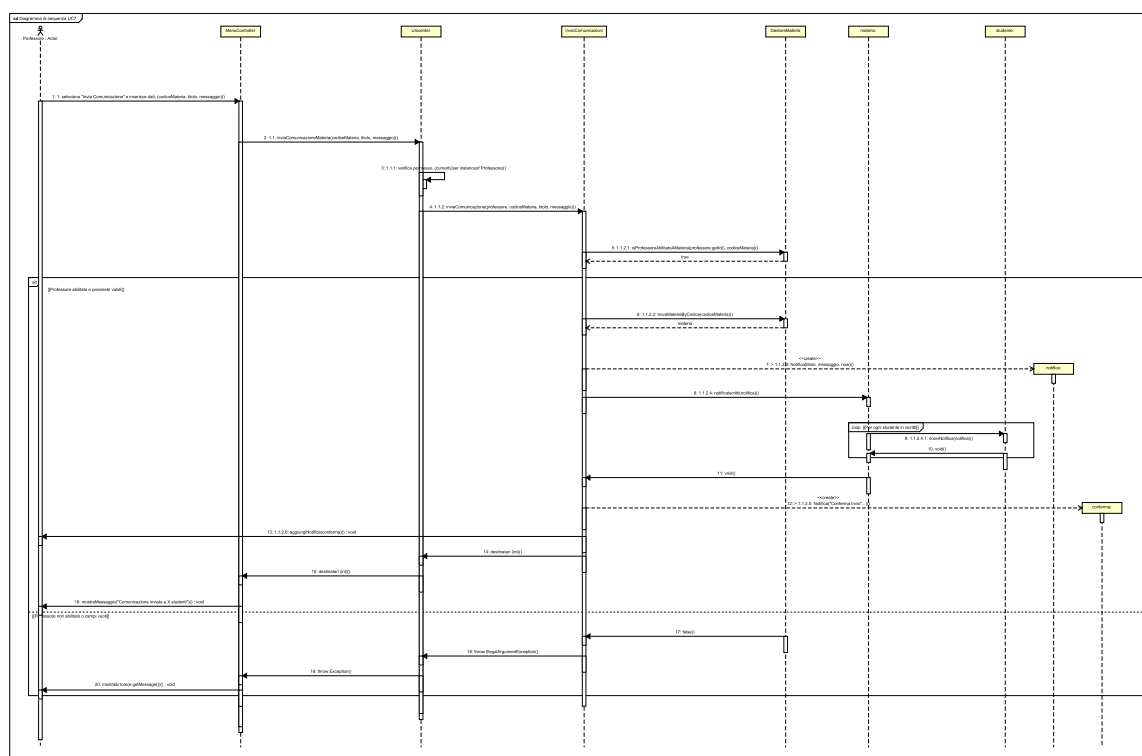


Figura 2.9: Diagramma di Sequenza di Progetto per UC7 – Invio Comunicazioni.

Flusso delle Operazioni e Logica Interna

Invio di Comunicazioni agli Iscritti da parte del Docente

Il diagramma di sequenza rappresenta il flusso di interazione che permette a un docente di inviare una comunicazione ufficiale a tutti gli studenti iscritti a una determinata materia.

Il processo ha inizio quando il docente seleziona l'opzione *"Invia Comunicazione"* e compila i campi richiesti all'interno del **MenuController**. La richiesta viene inoltrata alla facciata di sistema **Unicenter**, la quale esegue una prima verifica dei permessi e delega l'operazione al controller specializzato **InvioComunicazioniController**. Quest'ultimo interroga il **GestoreMaterieController** invocando **isProfessoreAbilitatoAMateria** per accertare che il docente sia effettivamente titolare dell'insegnamento.

Si delineano a questo punto due scenari alternativi:

- **Docente abilitato e parametri validi:** Una volta confermata l'abilitazione, `InvioComunicazioniController` recupera l'entità della materia tramite `trovaMaterieByCodice`. Viene quindi creata una nuova istanza di `notifica` contenente il messaggio, e viene invocato il metodo `notificaIscritti` sull'oggetto `materia`. Attraverso un ciclo iterativo su tutti gli studenti registrati all'insegnamento, la materia provvede a recapitare il messaggio a ciascuno studente (`riceviNotifica`).

Successivamente, il controller genera un'ulteriore notifica di **conferma** assegnata direttamente al docente mittente (`aggiungiNotifica`) e restituisce il numero dei destinatari raggiunti risalendo fino al `MenuController`, il quale mostra un messaggio di avvenuto invio.

- **Docente non abilitato o parametri non validi:** Qualora la verifica di abilitazione fallisca o i campi inseriti non rispettino i vincoli previsti, viene sollevata una `IllegalArgumentException`. L'eccezione viene propagata attraverso la facciata fino al `MenuController`, che si occupa di segnalare l'errore al docente bloccando la procedura.

Elementi di Design

- **Observer Pattern:** L'entità `Materia` assume il ruolo di *Subject* per la distribuzione dell'avviso agli studenti attivi, disaccoppiando il mittente (docente) dalle modalità concrete di ricezione e salvataggio della notifica da parte di ciascun allievo.
- **Creator (GRASP):** `InvioComunicazioniController` agisce da creatore per le istanze di `Notifica`, assemblando testo, intestazione e metadati temporali.
- **Information Expert:** `GestoreMaterieController` accentra le verifiche di responsabilità della cattedra, evitando duplicazioni di logica di autorizzazione.

2.3.6 Diagramma delle Classi di Progetto e Architettura Globale

Il diagramma delle classi di progetto rappresenta la struttura statica del sistema UniCenter al termine della seconda iterazione.

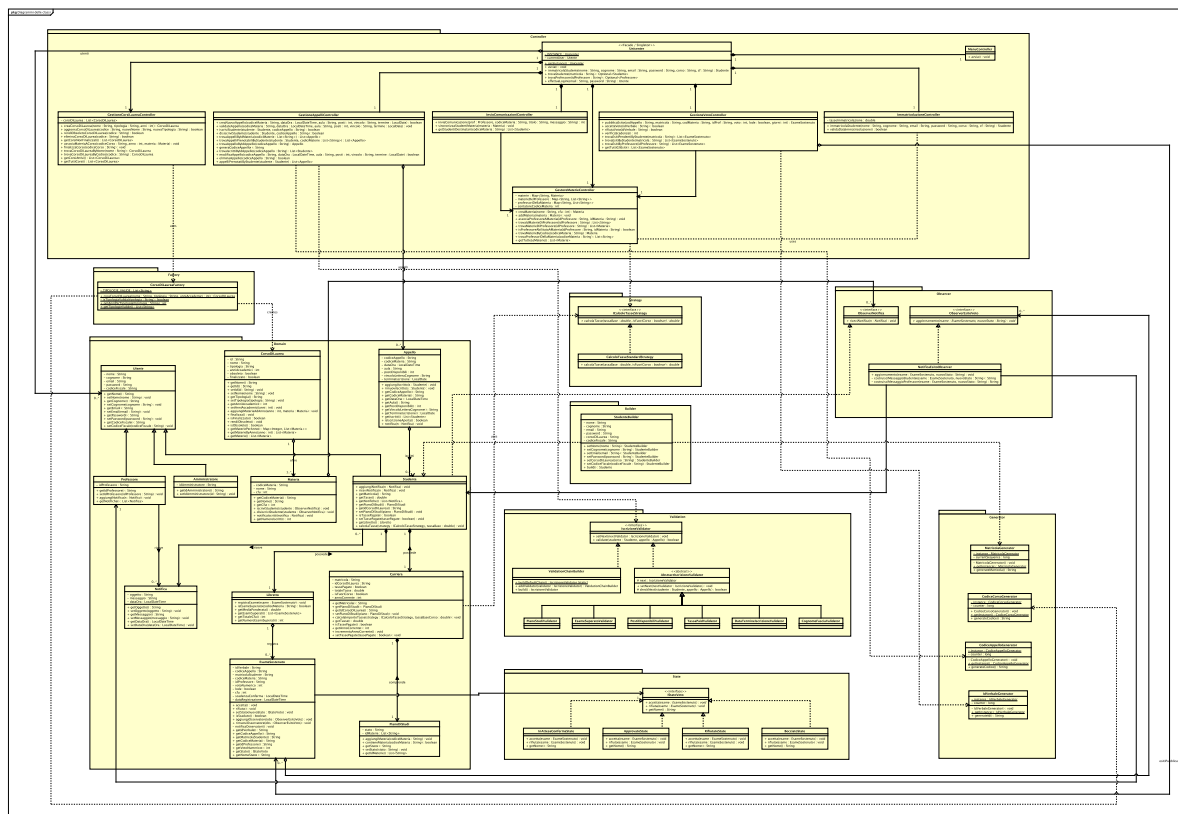


Figura 2.10: Diagramma delle Classi di Progetto – Seconda Iterazione.

Descrizione del Diagramma delle Classi - Seconda Iterazione

Nella seconda iterazione, il diagramma delle classi evolve per supportare la gestione degli esiti, le carriere universitarie complete, la configurazione accademica e la messaggistica, introducendo nuovi pattern e raffinando il modello di dominio precedente.

- **Evoluzione del Dominio e Refactoring di Studente:**

- Viene introdotto l'**Amministratore** nella gerarchia degli **Utente**, mentre la classe **Studente** subisce un'importante modularizzazione: delega la gestione del percorso didattico alla classe **Carriera** (che ingloba **PianoDiStudi**, tasse e anno di corso) e la registrazione degli esiti alla classe **Libretto**.
- Vengono modellate le entità **EsameSostenuto** (con gestione di voto, lode e scadenze di accettazione) e arricchito **CorsoDiLaurea** (gestione del ciclo di vita: bozza, finalizzato o obsoleto).

- **Nuovi Controller :**

- Si aggiungono `GestioneVotoController` (pubblicazione, accettazione/rifiuto e silenzio-assenso), `InvioComunicazioniController` (broadcast docenti-studenti) e `GestioneCorsiLaureaController` (amministrazione offerta formativa).

- **Nuovi Design Pattern e Potenziamanti:**

- **State Pattern (State):** Gestisce il ciclo di vita dell'esito d'esame in `EsameSostenuto` tramite l'interfaccia `IStatoVoto` e gli stati concreti (`InAttesaConferma`, `Approvato`, `Rifiutato`, `Bocciato`), incapsulando le regole di transizione.
- **Factory Pattern (Factory):** `CorsoDiLaureaFactory` centralizza la creazione e validazione dei corsi di studio in base alla tipologia e durata legale.
- **Observer Pattern (Observer):** Esteso significativamente rispetto alla prima iterazione:
 - * `ObserverEsitoVoto` / `NotificaEsitoObserver` aggiorna automaticamente docente e studente a fronte di transizioni di stato del voto.
 - * `Materia` implementa il pattern per inoltrare comunicazioni a tutti gli studenti iscritti (`ObserverNotifica`).
- **Estensione Chain of Responsibility (Validation):** La catena di validazione per l'iscrizione agli appelli viene arricchita con `EsameSuperatoValidator`, impedendo l'iscrizione a materie già superate nel libretto.

2.4 Testing

2.4.1 Strategia di Testing

Durante la seconda iterazione, la suite di testing è stata estesa e rifattorizzata per supportare l'evoluzione delle funzionalità implementate e i nuovi pattern architetturali: la gestione del ciclo di vita e della transizione degli esiti d'esame mediante il pattern **State** (**IStatoVoto** e le relative classi di stato concreto), la generazione controllata e normalizzata dell'offerta formativa con il **Simple Factory** (**CorsoDiLaureaFactory**), l'evoluzione della catena di controllo **Chain of Responsibility** con l'introduzione di nuovi anelli prioritari (**EsameSuperatoValidator**), il disaccoppiamento della carriera accademica con il modello **Libretto/Carriera**, e l'estensione del meccanismo di notifica **Observer** sia per la pubblicazione/aggiornamento degli esiti d'esame (**ObserverEsitoVoto**) sia per le comunicazioni didattiche mirate del corpo docente (**InvioComunicazioniController**, **ObserverNotifica**).

La strategia di collaudo del dominio nell'Iterazione 2 si articola su quattro livelli di verifica:

1. **Test Unitari di Dominio, Pattern e Transizioni di Stato (JUnit 5 & Mockito):** collaudo in isolamento della macchina a stati del voto (**InAttesaConfermaState**, **ApprovatoState**, **BocciatoState**, **RifiutatoState**), validazione della coerenza tipologia-durata nei corsi di laurea tramite test parametrici con la Factory, e verifica dell'integrità del libretto e della carriera dello studente.
2. **Test Parametrici e di Normalizzazione Dati (JUnit 5 Parameterized Tests):** verifica i casi limite e la tolleranza al formato (case-insensitivity, rimozione spazi superflui) nell'istanziamento dei corsi di studio e nei vincoli alfabetici.
3. **Test di Integrazione dei Pattern Comportamentali (State + Observer + Chain):** verifica combinata delle transizioni di stato del voto con recapito delle notifiche allo studente e aggiornamento automatico del libretto universitario, test della nuova catena di validazione per prevenire doppie iscrizioni.
4. **Test dei Controller di Dominio e Comunicazione Didattica:** validazione dei flussi coordinati da **GestioneVotoController**, **GestioneCorsiLaureaController**, **InvioComunicazioniController** e **MenuController**, certificando i filtri di recapito comunicazioni basati sul piano di studi attivo.

Sintesi delle Scelte Architetture e di Design nel Testing dell'Iterazione 2

1. **Ciclo di Vita del Voto e Macchina a Stati (State Pattern Testing):** L'introduzione del pattern **State** per la verbalizzazione degli esami ha portato alla creazione di una suite dedicata (**StatoVotoIntegrationTest** e i test dei singoli stati). La suite valida che un esame sufficiente (≥ 18) inizi nello stato **InAttesaConfermaState** consentendo solo le azioni di accettazione e rifiuto, mentre un esame insufficiente (< 18) transiti direttamente nello stato **BocciatoState**, impedendo qualsiasi successiva modifica di stato e sollevando **IllegalStateException**.

2. **Istanziamento Tipizzato e Validato (Factory Pattern Testing):** La classe `CorsoDiLaureaFactoryTest` collauda la creazione centralizzata dell'offerta formativa per tutte le tipologie ammesse (*Triennale, Magistrale, Magistrale a Ciclo Unico, Master*), verificando mediante test parametrici la corrispondenza rigorosa con il numero di anni accademici previsti e il rifiuto di combinazioni incoerenti o tipologie non riconosciute.
3. **Evoluzione ed Estensione della Catena di Validazione (Chain of Responsibility Evolution):** *Modifica rispetto all'Iterazione 1:* la catena di default (`ValidationChainBuilder`) è stata estesa introducendo `EsameSuperatoValidator` come **primo anello assoluto**. La suite verifica che se lo studente possiede già un voto approvato nel proprio libretto per la materia dell'appello, la validazione fallisca tempestivamente (*fail-fast*) prima ancora di interrogare il piano di studi, le tasse, i posti o i vincoli di cognome.
4. **Estensione del Pattern Observer per Esiti e Comunicazioni Didattiche:** L'Observer è stato esteso oltre le variazioni degli appelli: la suite `InvioComunicazioniControllerTest` e i test dello State certificano che gli studenti ricevano notifiche sia al cambio di stato del voto (`ObserverEsitoVoto`), sia alla pubblicazione di avvisi del docente, garantendo che gli studenti che hanno già verbalizzato la materia siano automaticamente esclusi dai destinatari.
5. **Refactoring della Carriera e Separazione delle Responsabilità:** *Modifica rispetto all'Iterazione 1:* i vincoli di carriera (stato delle tasse, esami superati, piano di studi) sono stati disaccoppiati dall'entità monolitica dello studente e segregati nelle entità *Carriera* e *Libretto*. I test dei validatori (`TassaPaidValidatorTest`, `PianoStudiValidatorTest`) e dei controller sono stati conseguentemente aggiornati con stub mirati.

2.4.2 Dettaglio della Test Suite per Caso d'Uso

Di seguito vengono descritte nel dettaglio le singole classi di test relative alla logica di dominio, ai controller e ai pattern introdotti o evoluti nella seconda iterazione. Per ciascun caso d'uso vengono illustrati i **casi di test più significativi e rappresentativi**, corredati dalla descrizione dei vincoli collaudati e dalle eventuali evoluzioni rispetto all'iterazione precedente.

Caso d'uso UC3: Pubblicazione Esiti, Accettazione e Rifiuto Voto (State & Observer Pattern)

Questa suite valida il flusso completo di verbalizzazione degli esami: pubblicazione dell'esito da parte del docente, transizioni della macchina a stati del voto (**State**), notifica allo studente (**Observer**) e registrazione definitiva nel libretto universitario (`GestioneVotoControllerTest`, `StatoVotoIntegrationTest`, classi del package `state`):

- **testPubblicaEsito_Sufficiente (UC3 flusso nominale pubblicazione):** Verifica che la pubblicazione di un voto valido (≥ 18 , es. 28) istanzi l'esame nello stato iniziale `InAttesaConfermaState`, associando correttamente voto numerico, lode e scadenza per la decisione dello studente.

- **testPubblicaEsito_Insufficiente (UC3 gestione bocciatura):** Accerta che l'inserimento di un voto insufficiente (< 18 , es. 15) imposti automaticamente lo stato in `BocciatoState`, impedendo successive richieste di conferma.
- **testPubblicaEsito_VotoNonValido:** Verifica che l'immissione di valori fuori scala (es. voti superiori a 30 o negativi) sollevi un'`IllegalArgumentException`.
- **testPubblicaEsito_LodeSenza30:** Valida la regola accademica di integrità del voto, sollevando un'eccezione qualora si tenti di assegnare la lode a un punteggio numerico inferiore a 30.
- **testAccettaVoto_Successo (UC3 accettazione voto e transizione State):** Verifica che l'invocazione di `accetta()` da parte dello studente transiti l'esame dallo stato `InAttesaConfermaState` ad `ApprovatoState`, aggiornando il libretto universitario.
- **testTransizioneAccettaNotificaObserver (Integrazione State + Observer):** Collauda la notifica agli osservatori registrati (`ObserverEsitoVoto`) all'atto dell'accettazione del voto e accerta che ulteriori tentativi di transizione sollevino `IllegalStateException`.
- **testRifiutaVoto_Successo (UC3 rifiuto esito):** Valida la transizione di stato verso `RifiutatoState` in caso di rifiuto esplicito da parte dello studente, escludendo l'esame dal conteggio dei crediti superati.
- **testBocciatoNonConsenteTransizioni:** Verifica la robustezza degli stati terminali, accertando che tentativi di accettare o rifiutare un esame con esito bocciato vengano tassativamente respinti.
- **testAccettaTransitaAdApprovato / testRifiutaTransitaARifiutato (Unitari State):** Collaudano isolatamente il comportamento dei singoli stati concreti tramite Mockito, verificando che deleghino al contesto l'impostazione della nuova istanza di stato.
- **testAccettaLanciaEccezione (Stati terminali protetti):** Verifica che classi come `ApprovatoState`, `BocciatoState` e `RifiutatoState` lancino `IllegalStateException` se sollecitate con ulteriori operazioni di accettazione o rifiuto.
- **testVerificaScadenze_silenzioRifiutoAutomatico:** Collauda il job di controllo temporale per l'applicazione del silenzio-rifiuto, accertando che gli esiti non confermati entro la finestra di 7 giorni solari transitino automaticamente nello stato `RifiutatoState`.

Caso d'uso UC4: Creazione e Configurazione dei Corsi di Laurea (Simple Factory)

Questa suite certifica la creazione conforme dell'offerta formativa mediante il pattern **Simple Factory**, la validazione dei vincoli di durata legale e la gestione del ciclo di vita dei corsi tra bozze, corsi finalizzati e obsolescenza (`CorsoDiLaureaFactoryTest`, `GestioneCorsiLaureaControllerTest`):

- **testCreaCorsoDiLaureaSuccesso (UC4 flusso nominale Factory):** Verifica con test parametrico (@CsvSource) la corretta creazione di corsi di studio per tutte le tipologie (*Triennale*, *Magistrale*, *Magistrale a Ciclo Unico*, *Master*), accertando il corretto calcolo degli anni accademici, la generazione dell'ID univoco e la tolleranza al maiuscolo/minuscolo.
- **testAnniNonCoerentiLanciaEccezione (Validazione coerenza Factory):** Collauda con un'ampia matrice di test parametrizzati il rigetto immediato di combinazioni incoerenti tra tipologia e anni (es. *Triennale* con 2 o 4 anni, *Magistrale* con 3 anni, *Ciclo Unico* con 4 anni, valori negativi o nulli).
- **testTipologiaNonValidaLanciaEccezione:** Verifica che stringhe non corrispondenti a tipologie universitarie riconosciute (es. *"Dottorato"*, *"Specializzazione"*, stringhe numeriche) sollevino *IllegalArgumentException*.
- **testNomeNonValidoLanciaEccezione:** Valida il controllo sintattico sul nome del corso, bloccando valori nulli, vuoti o composti esclusivamente da spazi.
- **testGetTipologieValide (Protezione difensiva):** Accerta che l'elenco delle tipologie valide restituito dalla Factory sia protetto da incapsulamento difensivo, impedendo a modifiche dell'array esterno di alterare lo stato interno.
- **testEliminaCorsoDiLaurea_BozzaNonFinalizzata (UC4 cancellazione bozza):** Verifica che un corso di laurea appena creato e non ancora finalizzato (stato bozza) possa essere eliminato senza restrizioni dal database.
- **testEliminaCorsoDiLaurea_Obsoleto (UC4 cancellazione corso dismesso):** Verifica che un corso di laurea precedentemente finalizzato ma successivamente marcato come obsoleto possa essere eliminato con successo.
- **testEliminaCorsoDiLaurea_AttivoFinalizzato_LanciaEccezione (Protezione corsi attivi):** Controlla che il sistema impedisca categoricamente la cancellazione di corsi di laurea attivi e finalizzati, sollevando *IllegalStateException* a tutela della carriera degli studenti.

Caso d'uso UC5: Gestione Materie e Assegnazione Cattedre

Questa suite verifica la configurazione del catalogo delle materie, l'associazione delle cattedre ai docenti e la gestione delle co-docenze (*GestoreMaterieControllerTest*):

- **associaProfessoreAMateria_associazioneSemplice_vieneRegistrata (UC5 flusso nominale):** Verifica la corretta registrazione del legame di cattedra tra un docente e l'insegnamento di competenza.
- **associaProfessoreAMateria_piuMaterieAlloStessoProfessore_vengonoAccumulate:** Accerta che a un singolo docente possano essere assegnati molteplici corsi di insegnamento senza sovrascritture.
- **associaProfessoreAMateria_stessaCoppiaDueVolte_eIdempotente:** previene duplicazioni interne in caso di ripetute invocazioni dello stesso abbinamento docente-materia.

- **associaProfessoreAMateria_stessaMateriaAPiuProfessori_funzionaPerEntrambi (Co-docenze):** Certifica il supporto alle co-docenze, garantendo che più docenti possano risultare titolari e abilitati al medesimo codice materia.
- **isProfessoreAbilitatoAMateria_associazioneAssente_ritornaFalse:** Valida il controllo di sicurezza che impedisce a docenti non accreditati di operare su insegnamenti non assegnati.
- **trovaMaterieDiProfessore_conMaterieRegistrateEAssociate_ritornaGliOggettiCorretti:** Verifica il recupero consistente di tutte le entità *Materia* appartenenti al carico didattico del professore.
- **addMateria_conStessoCodiceDueVolte_sovrascriveLaPrecedente:** Verifica che il catalogo delle materie, indicizzato per codice univoco, aggiorni coerentemente la scheda dell'insegnamento in caso di inserimento aggiornato.

Caso d'uso UC7: Invio Comunicazioni e Avvisi Didattici agli Studenti (Observer Pattern)

Questa suite certifica il modulo di trasmissione avvisi e comunicazioni didattiche da parte dei docenti, verificando il corretto instradamento dei messaggi in base al piano di studi e allo stato del libretto (*InvioComunicazioniControllerTest*):

- **testInvioComunicazioneSuccesso (UC7 flusso nominale invio avviso):** Verifica l'invio broadcast di un avviso da parte del professore titolare, accertando che tutti gli studenti che hanno la materia nel proprio piano di studi ricevano la notifica con titolo e corpo del messaggio corretti.
- **testStudenteConEsameNelLibrettoNonRiceveComunicazione (Filtro intelligente libretto):** Valida la regola di business che esclude automaticamente dall'invio gli studenti che hanno già sostenuto, verbalizzato e approvato l'esame nel proprio *Libretto*, evitando notifiche superflue a chi ha già completato la materia.
- **testProfessoreNonAbilitatoLanciaEccezione (Controllo autorizzativo):** Accerta che un docente privo di abilitazione sulla specifica materia non possa inviare comunicazioni agli iscritti, sollevando *IllegalArgumentException*.
- **testDocenteRiceveNotificaDiConferma (Ricevuta di invio):** Verifica che, al termine della procedura di spedizione, il docente mittente riceva una notifica di conferma attestante l'avvenuto inoltro della comunicazione.
- **testParametriNonValidiLanciaEccezione:** Collauda il rigetto di richieste con campi nulli, vuoti, privi di oggetto/testo o riferite a materie inesistenti.

Caso d'uso UC2 ed Evoluzione della Catena di Validazione Iscrizioni

Questa suite certifica l'integrazione del nuovo anello di validazione e le modifiche apportate alla catena di controllo delle iscrizioni rispetto all'iterazione precedente (*EsameSuperatoValidatorTest*, *ValidationChainBuilderTest*, *GestioneAppelliControllerTest*, *AbstractIscrizioneValidatorTest*):

- **validate_esameGiaSuperatoNelLibretto_lanciaIllegalStateException (Nuovo controllo Iterazione 2):** Collauda il nuovo anello `EsameSuperatoValidator`, verificando che il tentativo di iscrizione a un appello di una materia già superata e verbalizzata nel Libretto venga bloccato con `IllegalStateException`.
- **validate_esameGiaSuperato_nonDelegaAlProssimoValidator (Fail-Fast sul Libretto):** Dimostra che la presenza dell'esame superato interrompe immediatamente l'esecuzione della catena senza consultare i validatori successivi.
- **validate_esameNonSuperatoConNext_delegaAlProssimoValidator:** Accerta che, qualora l'esame non sia ancora presente nel libretto, la validazione prosegue regolarmente verso il successivo anello della catena.
- **buildDefaultChain_ordineSequenzialeAggiornato (Modifica rispetto a Iterazione 1):** Verifica che l'ordine cablato in `ValidationChainBuilder.buildDefaultChain()` posizioni `EsameSuperatoValidator` prima di `PianoStudiValidator`, garantendo la gerarchia: *Esame Superato* → *Piano di Studi* → *Posti Disponibili* → *Tasse Saldate* → *Fascia Cognome* → *Scadenza Termini*.
- **iscriviStudente_esameGiaSuperatoNelLibretto_lanciaIllegalStateException (Integrazione Controller):** Verifica a livello di `GestioneAppelliController` che la presenza del voto nel libretto impedisca l'iscrizione all'appello, non aggiunga lo studente agli iscritti e non invii notifiche errate.
- **testCheckNextConNextDelega / testCheckNextPropagaEccezione (Test Classe Base Astratta):** Certifica il corretto funzionamento di `AbstractIscrizioneValidator` nella propagazione sequenziale del flusso e delle relative eccezioni di business.
- **validate_materiaObbligatoria_annoFuturo_lanciaAnnoCorsoNonValidoException (AnnoCorsoMateriaValidatorTest):** Verifica che uno studente non possa iscriversi ad appelli di insegnamenti obbligatori previsti per anni di corso successivi a quello attualmente frequentato, autorizzando invece le materie arretrate o a scelta.

3. Terza Iterazione

3.1 Requisiti

3.1.1 Introduzione

La terza iterazione del sistema **UniCenter** completa il quadro funzionale dell'ateneo universitario, focalizzandosi su due aree cruciali: la gestione del **Materiale Didattico** (organizzato secondo una struttura gerarchica e fruibile in modalità multiformato) e la flessibilità curriculare attraverso la compilazione, presentazione e approvazione del **Piano di Studi** con materie a scelta.

L'applicazione abilita i **Docenti** al caricamento, strutturazione in cartelle e manutenzione delle risorse didattiche (slide, documenti PDF, dispense, file di testo, link esterni e video-lezioni) per ciascun insegnamento di cui sono responsabili. Gli **Studenti** possono consultare liberamente le directory, visualizzare anteprime integrate, scaricare i file sul proprio dispositivo e contrassegnare le risorse d'interesse come preferite. Inoltre, gli studenti possono personalizzare il proprio percorso di studi inserendo insegnamenti opzionali, soggetti a regole di approvazione automatica o sottomissione manuale alla commissione didattica. Inoltre si è aggiunta un'interfaccia grafica mediante browser web.

3.1.2 Evoluzione delle Regole di Dominio

Nel corso della terza iterazione, a completamento del ciclo di vita della carriera dello studente (originariamente introdotto con l'immatricolazione in UC8), è stata formalizzata e implementata la regola di dominio che governa la **progressione annuale degli studi** e la determinazione automatica dello status di **Fuori Corso**:

- **Rinnovo Annuale:** Agli studenti iscritti ad anni successivi al primo è consentito rinnovare annualmente l'iscrizione all'interno di una finestra temporale dedicata. Condizione necessaria ed inderogabile per perfezionare il rinnovo è la regolarità contributiva: il sistema inibisce categoricamente la progressione di carriera in presenza di pendenze o debiti relativi agli anni accademici precedenti.
- **Transizione Automatica a Fuori Corso:** All'atto del rinnovo, il sistema confronta il nuovo anno di corso raggiunto dall'allievo con la durata legale del percorso accademico di appartenenza (es. oltre il 3° anno per una laurea Triennale, oltre il 2° per una Magistrale). Al superamento di tale soglia:
 - Lo studente transita automaticamente dallo stato *In Corso* allo stato **Fuori Corso** (`isFuoriCorso = true`).
 - Viene applicata automaticamente una maggiorazione da sommare alla tassa base di rinnovo, impostando il debito contributivo per l'anno corrente.

- **Isolamento e Vincoli Temporal**: L'operazione è vincolata al principio di unicità annuale (un solo rinnovo per anno solare) e non può coincidere con l'anno di prima immatricolazione. Tali controlli fanno perno sul componente **ClockProvider**, garantendo il collaudo deterministico dei cambi di stato anno per anno sia a runtime che nella suite di test.

3.1.3 Casi d'uso

Nella terza iterazione sono stati analizzati, progettati e testati i seguenti casi d'uso:

- **UC6: Caricamento e Gestione Materiale Didattico**: Creazione di cartelle, upload, modifica ed eliminazione di risorse didattiche da parte del docente.
- **UC9: Compilazione e Approvazione Piano di Studi**: Definizione delle materie a scelta dello studente, verifica dei crediti minimi e applicazione della politica di approvazione (automatica o manuale).
- **UC10: Consultazione e Download Materiale Didattico**: Navigazione dell'albero dei materiali, visualizzazione anteprime polimorfiche, download dei file e gestione dei preferiti da parte dello studente.

UC6: Caricamento e Gestione Materiale Didattico

Attore Primario: Professore

Descrizione: Il docente responsabile di una materia organizza lo spazio didattico creando sottocartelle e caricando file di varia tipologia per gli studenti.

Scenario Principale:

1. Il professore seleziona una materia di cui è responsabile.
2. Il professore naviga la cartella radice associata alla propria cattedra e sceglie se creare una nuova sottocartella o caricare direttamente una risorsa.
3. Nel caso di creazione cartella: il professore specifica denominazione e descrizione; il sistema crea il nodo logico e la directory fisica su storage.
4. Nel caso di caricamento file: il professore specifica il nome della risorsa, la tipologia (*Slide*, *Documento PDF*, *Dispensa*, *File di Testo*, *Link*, *Video*), la descrizione e fornisce il contenuto in byte (o l'URL per risorse link/video).
5. Il sistema convalida i dati, istanzia l'elemento polimorfico corrispondente, lo associa alla cartella genitore e persiste fisicamente il file sul repository.
6. Il sistema restituisce la conferma di avvenuto inserimento con il riepilogo delle risorse caricate.

Scenari Alternativi:

- **Docente non abilitato all'insegnamento (Passo 1):** Se un docente tenta di caricare o modificare materiali per una materia a cui non è assegnato, il sistema blocca l'operazione segnalando la mancata autorizzazione.
- **Tentativo di modifica su cartella di altro docente (Passo 2):** Qualora una materia sia co-docente e un professore tenti di aggiungere file o eliminare cartelle all'interno dello spazio riservato a un collega, l'accesso in scrittura viene respinto segnalando la violazione dei permessi di cattedra.
- **Nome non valido o tipo nullo (Passo 4):** Se il nome del file è vuoto o il tipo non è specificato, il sistema segnala l'errore ed interrompe la procedura.
- **Eliminazione di risorsa o cartella:** Il docente seleziona un elemento di propria proprietà e ne richiede l'eliminazione. Il sistema rimuove ricorsivamente l'elemento dall'albero logico e cancella i file fisici dal repository. Se si tenta di eliminare la radice della materia, l'azione viene respinta segnalando che la directory radice non è eliminabile.

Regole di Dominio:

- **Segregazione delle Cattedre:** Ciascun docente assegnato a una materia possiede una cartella radice personale dedicata (**Nome_Cognome**), garantendo che ogni insegnante mantenga il pieno controllo delle proprie risorse senza interferire con quelle dei colleghi.
- **Tipizzazione Rigorosa delle Risorse:** Ogni risorsa didattica appartiene a una categoria ben definita (**TipoMateriale**) dotata di specifici attributi aggiuntivi (ad es. numero di lezione per le *Slide*, autore e anno per le *Dispense*).

UC9: Compilazione e Approvazione Piano di Studi

Attore Primario: Studente (e Amministratore / Commissione Didattica)

Descrizione: Lo studente definisce il proprio piano di studi curriculare selezionando le materie a scelta necessarie a soddisfare il requisito minimo di CFU opzionali.

Scenario Principale:

1. Lo studente accede alla sezione di compilazione del piano di studi, visualizzando le materie obbligatorie già precaricate dal proprio corso di laurea.
2. Lo studente seleziona dal catalogo d'ateneo gli insegnamenti a scelta da inserire nel piano.
3. Il sistema verifica che la somma dei CFU delle materie a scelta sia maggiore o uguale alla soglia minima stabilita (12 CFU).
4. Il sistema controlla che nessuna delle materie a scelta selezionate sia già presente tra le materie obbligatorie del corso.
5. Il sistema verifica se tutte le materie a scelta selezionate appartengono all'elenco delle materie pre-approvate dal manifesto del corso di laurea.

6. **Approvazione Automatica:** Se tutte le materie sono pre-approvate, il sistema applica la strategia automatica, commuta lo stato del piano in *Approvato* e consolida il percorso.
7. **Sottomissione Manuale:** Se tra le scelte sono presenti materie non pre-approvate, il sistema applica la strategia manuale e commuta lo stato del piano in *In Attesa*, inserendolo nella coda di revisione della commissione didattica.

Scenari Alternativi:

- **CFU a scelta insufficienti (Passo 3):** Se il totale dei crediti opzionali è inferiore a 12 CFU, il sistema respinge la compilazione segnalando che il totale dei crediti opzionali non raggiunge la soglia minima richiesta.
- **Materia a scelta coincidente con obbligatoria (Passo 4):** Se lo studente seleziona un insegnamento già inserito d'ufficio nel piano, l'operazione viene respinta per prevenire duplicazioni.
- **Blocco per impegni accademici pendenti:** Se lo studente tenta di ri-compilare il piano ma ha già effettuato una prenotazione a un appello d'esame o ha un esito in attesa di conferma per una materia a scelta già inserita, il sistema nega la modifica segnalando la presenza di prenotazioni o valutazioni pendenti per le materie inserite.
- **Rifiuto Manuale da parte della Commissione:** L'amministratore/commissione esamina un piano in stato *In Attesa* e decide di respingerlo. Il sistema commuta lo stato in *Rifiutato* e depenna le materie a scelta non approvate, preservando unicamente quelle già verbalizzate nel Libretto universitario.
- **Approvazione Manuale da parte della Commissione:** L'amministratore approva il piano; lo stato transiziona in *Approvato* e le materie divengono definitive.

Regole di Dominio:

- **Soglia Minima Curriculare:** Il piano di studi deve includere almeno 12 CFU di discipline a scelta dello studente.
- **Inviolabilità delle Materie Verbalizzate:** Qualsiasi operazione di rigetto o ri-compilazione del piano non può in nessun caso cancellare o rimuovere materie a scelta i cui esami siano già stati superati e verbalizzati con successo nel **Libretto**.

UC10: Consultazione e Download Materiale Didattico

Attore Primario: Studente

Descrizione: Lo studente naviga il materiale didattico di una materia, visualizza anteprime personalizzate in base al tipo di file, effettua il download dei documenti e salva i collegamenti preferiti.

Scenario Principale:

1. Lo studente seleziona un insegnamento dal proprio piano di studi o dal catalogo generale.

2. Il sistema restituisce la struttura gerarchica ad albero di tutte le cartelle e dei file pubblicati dai docenti.
3. Lo studente seleziona una specifica risorsa per visualizzarne l'anteprima: il sistema genera dinamicamente l'anteprima in base al tipo (testo per i TXT, anteprima visiva/download per PDF e Slide, link interattivo per risorse web).
4. Lo studente richiede il download del file: il sistema estrae il contenuto dal repository e lo eroga accompagnato dal corretto MIME type.
5. Lo studente può aggiungere o rimuovere la risorsa dall'elenco personale dei *Preferiti* tramite azione di toggle.

Scenari Alternativi:

- **Tentativo di download di una cartella come file (Passo 4):** Se lo studente richiede il download diretto di un nodo cartella anziché di un file atomico, il sistema segnala l'errore specificando che le cartelle vanno navigate e non possono essere scaricate direttamente.
- **Risorsa inesistente o rimossa:** Se l'identificativo richiesto non corrisponde ad alcun elemento, il sistema risponde con errore di risorsa non trovata.

Regole di Dominio:

- **Accesso in sola lettura per gli Studenti:** Gli studenti hanno visibilità completa sulle risorse didattiche ma non dispongono di permessi di scrittura, creazione o rimozione sull'albero dei materiali.
- **Trattamento Uniforme delle Risorse:** L'accesso all'albero e le operazioni di conteggio e ricerca trattano cartelle e file in modo trasparente e uniforme.

3.2 Analisi

3.2.1 Introduzione

La fase di analisi formalizza concettualmente il dominio della terza iterazione, definendo le entità preposte alla gestione dei percorsi accademici, formalizzando le interazioni di sistema tramite i Diagrammi di Sequenza di Sistema (SSD) e i Contratti delle Operazioni.

3.2.2 Modello di Dominio

Durante la terza iterazione, il modello di dominio concettuale è stato arricchito con la gerarchia del materiale didattico e il modello a stati del piano di studi.

Le entità concettuali introdotte e ampliate sono organizzate in due macro-aree:

1. Area Materiale Didattico e Risorse

- **ElementoDidattico (Astratta):** Radice concettuale di tutti gli elementi inseribili nell'albero didattico. Definisce identificativo univoco, nome, percorso relativo, data di caricamento, docente proprietario e codice materia.
- **Cartella (Composito):** Nodo contenitore che aggrega una collezione di istanze di **ElementoDidattico** (sia file che ulteriori sottocartelle), definendo una struttura ad albero arbitrariamente profonda.
- **MaterialeDidattico (Foglia Astratta):** Specializzazione di **ElementoDidattico** che rappresenta una risorsa atomica erogabile, dotata di dimensione in byte, tipologia e MIME type.
- **Specializzazioni Foglia:**
 - **DocumentoPdf:** Documenti e dispense in formato PDF standard.
 - **Slide:** Presentazioni delle lezioni con annotazione del numero di lezione curriculare.
 - **Dispensa:** Testi di approfondimento con autore docente e anno accademico di riferimento.
 - **FileTesto:** Appunti, codice sorgente e tracce in formato testuale puro (`text/plain`).
 - **RisorsaVideo** e **RisorsaLink:** Riferimenti multimediali e URL esterni.

2. Area Piano di Studi e Regole Curricolari

- **PianoDiStudi:** Contenitore curriculare dello studente che aggrega le materie obbligatorie (ereditate dal corso di laurea) e le materie a scelta deliberate dallo studente. Detiene uno stato esplicito (*Bozza*, *In Attesa*, *Approvato*, *Rifiutato*, *Registrato*).

- **PoliticaApprovazione:** Concetto che modella la regola di validazione curriculare per stabilire se le scelte dello studente siano automaticamente conformi o richiedano il vaglio della commissione.

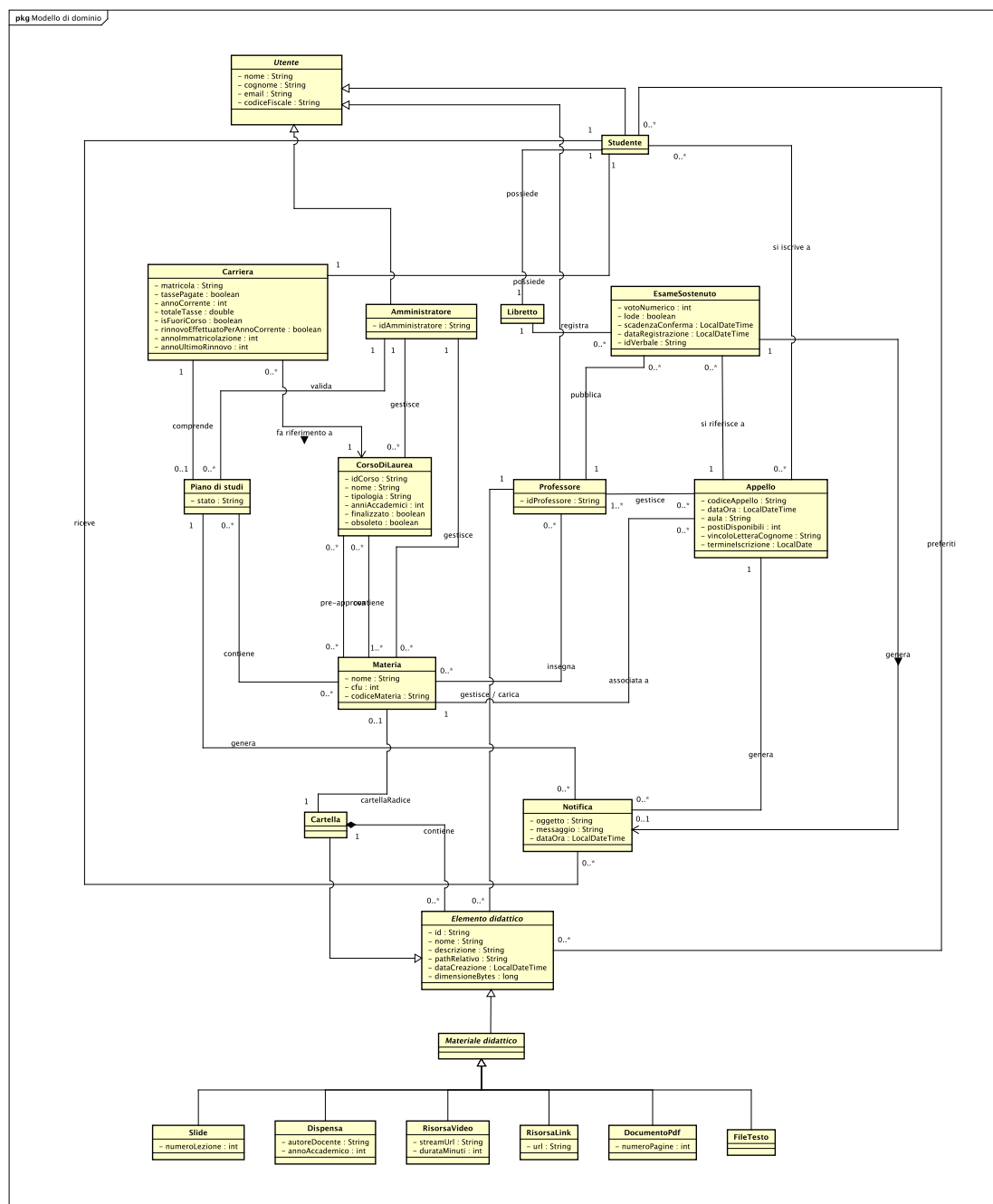


Figura 3.1: Modello di Dominio Concettuale – Terza Iterazione.

3.2.3 Caso d'uso UC6 – Caricamento e Gestione Materiale Didattico

Diagramma di Sequenza di Sistema (SSD)

L'SSD per l'UC6 descrive le interazioni tra il **Professore** e il sistema per la creazione di cartelle, il caricamento di file e l'eventuale rimozione di risorse.

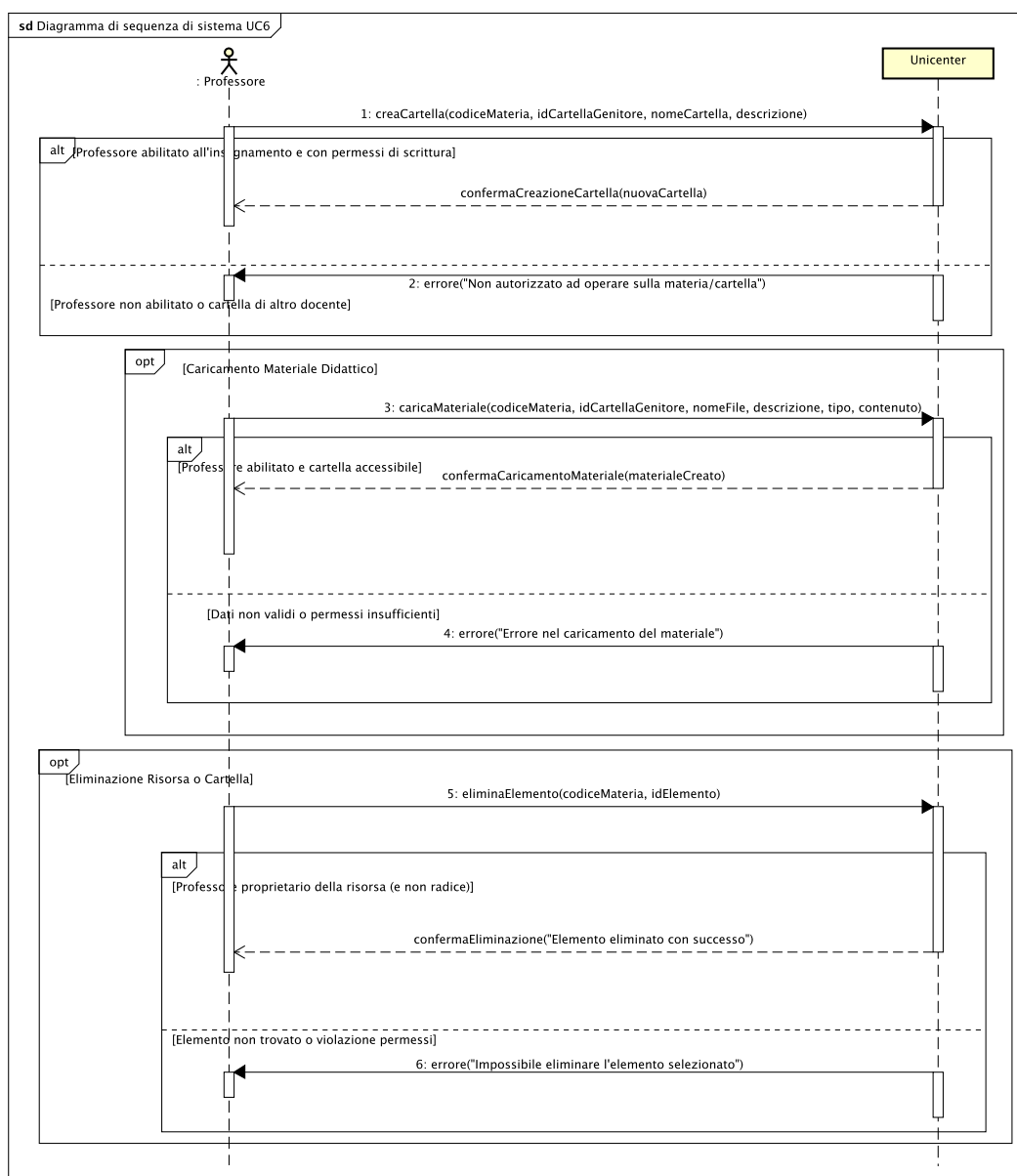


Figura 3.2: SSD per il caso d'uso UC6 – Gestire Materiale Didattico.

Dettaglio dello scambio di messaggi (SSD): Il diagramma modella tre operazioni opzionali di gestione dell'albero documentale:

1. **Creazione Cartella – creaCartella(codiceMateria, idCartellaGenitore, nomeCartella, descrizione):**

- **Successo:** se il docente è titolare della materia e opera all'interno della cartella radice assegnata alla propria cattedra, il nodo viene istanziato e confermato con `confermaCreazioneCartella(nuovaCartella)`.
 - **Violazione Permessi:** se il docente tenta di operare su spazi di altri colleghi o su materie non proprie, l'operazione viene respinta con l'errore *"Non autorizzato ad operare sulla materia/cartella"*.
2. **Caricamento File (Blocco opt) – `caricaMateriale(codiceMateria, idCartellaGenitore, nomeFile, descrizione, tipo, contenuto)`:**
- **Successo:** se la cartella di destinazione appartiene al docente e il file presenta estensione, tipo MIME e payload validi, il documento viene integrato nell'albero logico e persistito su storage, restituendo `confermaCaricamentoMateriale(materialeCreato)`.
 - **Errore Formato o Dimensione:** in caso di file vuoto, parametri mancanti o fallimento di storage, il sistema interrompe l'upload segnalando *"Errore nel caricamento del materiale"*.
3. **Eliminazione Risorsa (Blocco opt) – `eliminaElemento(codiceMateria, idElemento)`:**
- **Successo:** se l'elemento appartiene alla cattedra del docente e non coincide con la radice della materia, viene rimosso ricorsivamente (pulizia logica e cancellazione del binario su storage) con conferma di successo.
 - **Errore / Radice Protetta:** tentativi di eliminare la cartella radice o risorse di altri docenti vengono intercettati con *"Impossibile eliminare l'elemento selezionato"*.

Contratti delle Operazioni

Operazione	<code>creaCartella(codiceMateria: String, idGenitore: String, nome: String, descr: String)</code>
Riferimenti	UC6: Caricamento e Gestione Materiale Didattico
Precondizioni	• Il docente è autenticato ed è abilitato all'insegnamento di <code>codiceMateria</code>. • Il parametro <code>nome</code> non è nullo né vuoto. • La cartella genitore indicata esiste e appartiene allo spazio di scrittura del docente.
Postcondizioni	• È stata creata una nuova istanza di <code>Cartella</code> con identificativo univoco. • La nuova cartella è stata inserita nella collezione dei figli della cartella genitore. • È stata creata la directory fisica corrispondente nel repository di storage.

Tabella 3.1: Contratto di sistema dell'operazione `creaCartella`.

Operazione	caricaMateriale(codiceMateria: String, idGenitore: String, nome: String, tipo: TipoMateriale, contenuto: File/Dati)
Riferimenti	UC6: Caricamento e Gestione Materiale Didattico
Precondizioni	• Il docente è autenticato ed è titolare della materia. • Il tipo di materiale è valorizzato e valido. • La cartella di destinazione appartiene al docente richiedente.
Postcondizioni	• È stata creata un'istanza concreta di MaterialeDidattico corrispondente al tipo fornito. • La risorsa è stata aggiunta alla cartella genitore nell'albero logico della materia. • Il contenuto del file è stato memorizzato sul repository nel percorso relativo calcolato.

Tabella 3.2: Contratto di sistema dell'operazione `caricaMateriale`.

3.2.4 Caso d'uso UC9 – Compilazione e Approvazione Piano di Studi

Diagramma di Sequenza di Sistema (SSD)

L'SSD per l'UC9 illustra la sottomissione del piano da parte dello **Studente** e la successiva fase decisionale del sistema (approvazione automatica o sottomissione alla commissione).

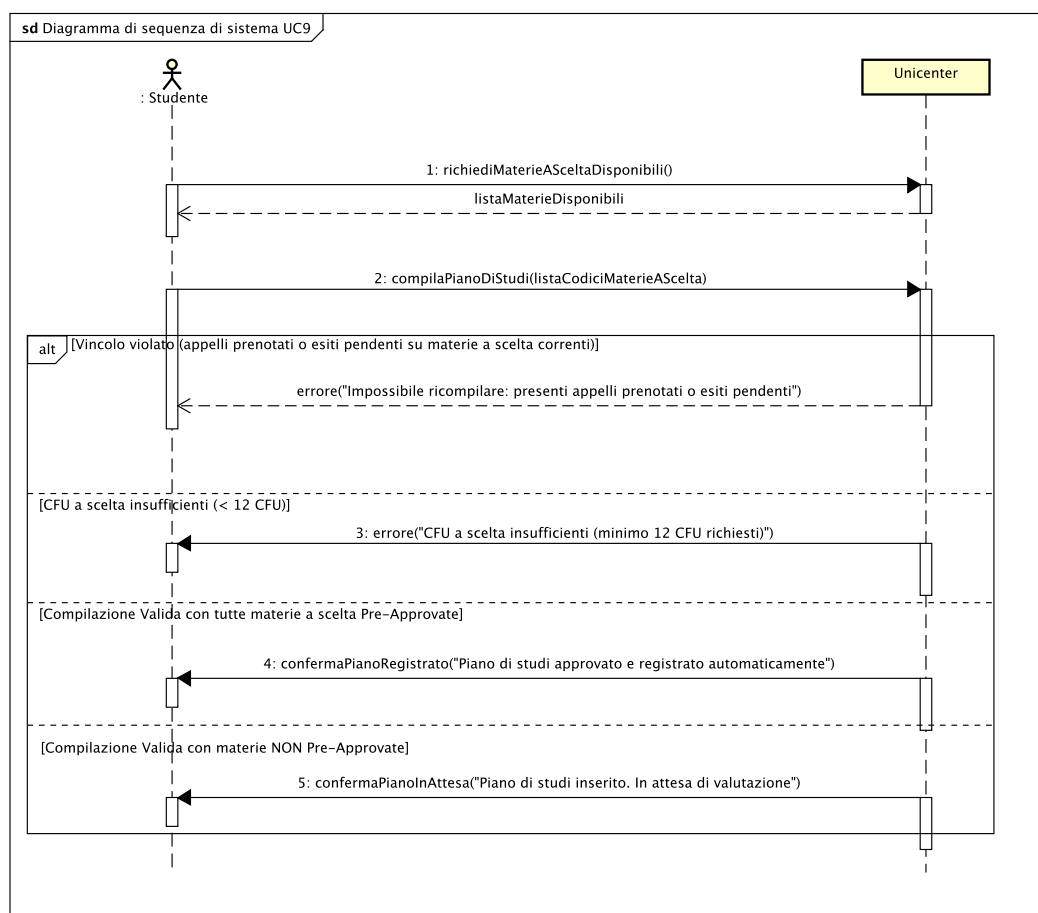


Figura 3.3: SSD per il caso d'uso UC9 – Compilazione Piano di Studi.

Dettaglio dello scambio di messaggi (SSD): Il flusso governa la selezione delle materie opzionali e l'applicazione delle strategie di convalida:

1. `richiediMaterieASceltaDisponibili()`: Lo studente richiede il catalogo degli insegnamenti opzionali erogati per il proprio corso, ricevendo la lista filtrata.
2. `compilaPianoDiStudi(listaCodicimaterieAScelta)`: Lo studente sottomete l'elenco dei corsi a scelta desiderati.
 - **Ramo Impegni Pendenti [alt]**: se lo studente ha prenotazioni d'esame attive o voti in attesa di conferma per materie opzionali precedentemente caricate, il sistema blocca l'operazione notificando *"Impossibile ricompilare: presenti appelli prenotati o esiti pendenti"*.

- **Ramo CFU Insufficienti [alt]:** se il conteggio dei crediti a scelta non raggiunge la soglia minima curriculare, l'operazione viene respinta con *"CFU a scelta insufficienti (minimo 12 CFU richiesti)"*.
- **Ramo Approvazione Automatica [alt]:** se tutte le discipline scelte appartengono al paniere pre-approvato dal manifesto, il sistema commuta il piano in stato *Approvato* restituendo *"Piano di studi approvato e registrato automaticamente"*.
- **Ramo Sottomissione Manuale [alt]:** se tra le scelte figurano insegnamenti esterni o liberi, il piano viene commutato in stato *In Attesa* e indirizzato alla commissione didattica (*"Piano di studi inserito. In attesa di valutazione"*).

Contratti delle Operazioni

Operazione	<code>compilaPianoDiStudi(codiciMaterieAScelta: List)</code>
Riferimenti	UC9: Compilazione Piano di Studi
Precondizioni	• Lo studente è autenticato ed è in regola con le tasse universitarie. • Lo studente non ha prenotazioni attive né esiti pendenti per le materie a scelta già a piano. • La somma dei CFU delle materie selezionate è ≥ 12. • Nessuna materia selezionata fa parte delle materie obbligatorie del corso.
Postcondizioni	• Le materie a scelta selezionate sono state registrate nel <code>PianoDiStudi</code> dello studente. • Eventuali materie a scelta già verbalizzate nel <code>Libretto</code> sono state rigorosamente preservate. • Se tutte le scelte sono pre-approvate, lo stato del piano diviene <i>"Approvato"</i>. • Se vi sono materie non pre-approvate, lo stato del piano diviene <i>"In Attesa"</i>.

Tabella 3.3: Contratto di sistema dell'operazione `compilaPianoDiStudi`.

3.2.5 Caso d'uso UC10 – Consultazione e Download Materiale Didattico

Diagramma di Sequenza di Sistema (SSD)

L'SSD per l'UC10 descrive le azioni di consultazione dell'alberatura, generazione anteprime polimorfiche, download e salvataggio preferiti eseguite dallo **Studente**.

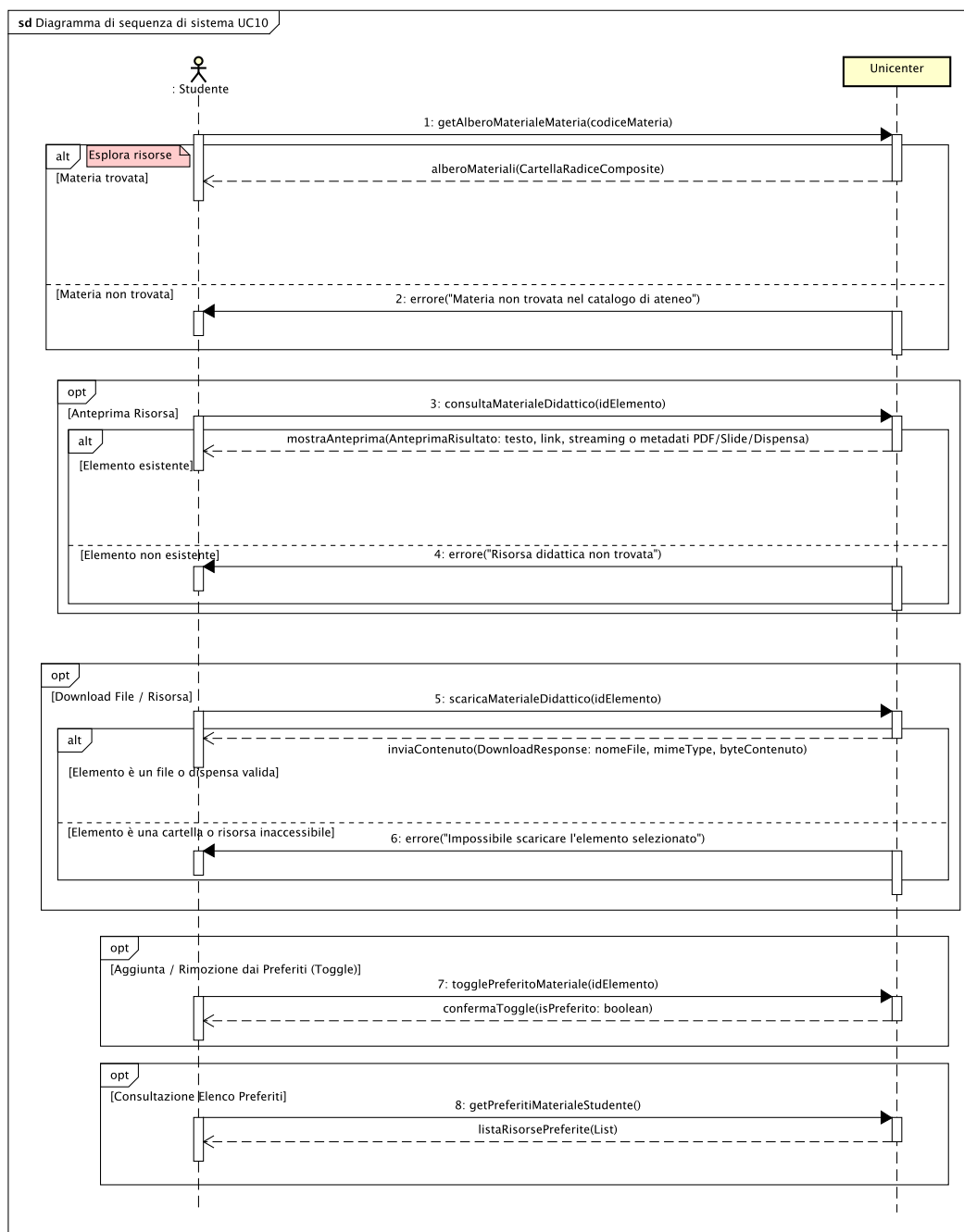


Figura 3.4: SSD per il caso d'uso UC10 – Consultare Materiale Didattico.

Dettaglio dello scambio di messaggi (SSD): L'interazione consente allo studente di navigare liberamente i contenuti multimediali in sola lettura:

1. **getAlberoMaterialeMateria(codiceMateria):** Lo studente richiede l'alberatura dei contenuti.
 - **Materia Esistente:** il sistema restituisce la struttura gerarchica ad albero dei nodi e delle cartelle (**CartellaRadiceComposite**).
 - **Materia Non Trovata:** se il codice non corrisponde ad alcun insegnamento, viene notificato l'errore di materia non presente nel catalogo.
2. **Visualizzazione Anteprima (Blocco opt) – consultaMaterialeDidattico(idElemento):**
 - **Risorsa Trovata:** il sistema estrae i metadati polimorfici restituendo l'anteprima specifica del tipo (testo per TXT, miniature/link per slide e video) tramite l'oggetto **AnteprimaRisultato**.
 - **Risorsa Inesistente:** restituisce l'errore di documento non trovato.
3. **Download Risorsa (Blocco opt) – scaricaMaterialeDidattico(idElemento):**
 - **File Atomico Valido:** il sistema recupera il flusso di byte e invia la risposta **DownloadResponse** con nome file e corretto MIME-type.
 - **Tentativo di Download Cartella:** se lo studente richiede lo scaricamento di un intero nodo cartella anziché di un file foglia, l'azione viene bloccata notificando *"Impossibile scaricare l'elemento selezionato: le cartelle vanno navigate"*.
4. **Gestione Segnalibri (Blocco opt) – togglePreferitoMateriale(idElemento):** Lo studente aggiunge o rimuove con meccanismo toggle la risorsa dal proprio profilo, ricevendo lo stato aggiornato (**boolean**).
5. **Consultazione Preferiti (Blocco opt) – getPreferitiMaterialeStudente():** Il sistema restituisce l'elenco delle risorse contrassegnate come preferite dallo studente autenticato.

Contratti delle Operazioni

Operazione	consultaMaterialeDidattico(idElemento: String)
Riferimenti	UC10: Consultazione Materiale Didattico
Precondizioni	• Lo studente è autenticato nel sistema. • L'elemento con identificativo idElemento esiste nell'albero didattico.
Postcondizioni	• È stato generato e restituito un oggetto AnteprimaRisultato contenente i metadati, il MIME type e il testo/URL di anteprima specifico del formato.

Tabella 3.4: Contratto di sistema dell'operazione **consultaMaterialeDidattico**.

Operazione	scaricaMaterialeDidattico(idElemento: String)
Riferimenti	UC10: Consultazione Materiale Didattico
Precondizioni	• L'elemento identificato da <code>idElemento</code> esiste ed è un'istanza di <code>MaterialeDidattico</code> (non una cartella).
Postcondizioni	• È stato erogato il contenuto del file unitamente al nome originale e al relativo MIME type.

Tabella 3.5: Contratto di sistema dell'operazione `scaricaMaterialeDidattico`.

3.3 Progettazione

3.3.1 Introduzione e Refactorings rispetto all'Iterazione 2

Nella terza iterazione, l'architettura interna di UniCenter si arricchisce per gestire strutture composite ricorsive e politiche decisionali avanzate, applicando i principi GRASP e i pattern GoF.

Sintesi dei Refactoring Architetturali e di Design

1. **Adozione del Pattern Composite (GoF Strutturale):** Per modellare la gerarchia del materiale didattico (UC6, UC10), è stato introdotto il pattern **Composite**. La classe astratta `ElementoDidattico` definisce l'interfaccia comune per nodi compositi (`Cartella`) e foglie atomiche (`MaterialeDidattico`). Le operazioni di calcolo dimensione (`getDimensioneBytes`), ricerca profonda (`trovaElemento`) e cancellazione a cascata (`rimuoviElemento`) sono implementate in modo ricorsivo e trasparente, permettendo ai controller di interagire con cartelle e file senza doversi preoccupare del livello di annidamento.
2. **Polimorfismo per la Gestione Multiformato (GRASP Polymorphism):** Invece di ricorrere a verifiche di tipo tramite istruzioni condizionali sull'estensione dei file, la gestione dei formati è affidata a una gerarchia polimorfica. Ciascuna classe foglia (`DocumentoPdf`, `Slide`, `Dispensa`, `FileTesto`, `RisorsaVideo`, `RisorsaLink`) implementa autonomamente i metodi `anteprima()`, `scarica()` e `getMimeType()`, incapsulando il comportamento peculiare di ogni media.
3. **Pattern Strategy per le Politiche di Approvazione (GoF Comportamentale):** La determinazione della conformità del piano di studi (UC9) è stata incapsulata tramite il pattern **Strategy**. L'interfaccia `PoliticaApprovazione` definisce l'algoritmo di valutazione, declinato nelle strategie concrete:
 - **ApprovazioneAutomatica:** Applicata quando tutti gli insegnamenti a scelta appartengono al paniere di materie pre-approvate dal corso di laurea. Commuta istantaneamente il piano in stato *Approvato*.
 - **ApprovazioneManuale:** Applicata in presenza di discipline opzionali libere o esterne all'ateneo, instradando il piano verso la commissione e ponendolo in stato *In Attesa*.
4. **Pattern State per il Ciclo di Vita del Piano di Studi (GoF Comportamentale):** Il ciclo di vita di `PianoDiStudi` è governato dall'interfaccia `IStatoPianoDiStudi`, articolata negli stati concreti `StatoBozzaPiano`, `StatoInAttesaPiano`, `StatoApprovatoPiano`, `StatoRifiutatoPiano` e `StatoRegistratoPiano`. Ogni stato regola le transizioni lecite, impedendo modifiche indebite e gestendo in modo polimorfico l'aggiunta o la rimozione di materie.
5. **Indirection e Repository Pattern (Pure Fabrication):** Per disaccoppiare il modello di dominio dall'accesso al filesystem fisico del sistema operativo, è stata introdotta

l'interfaccia `MaterialeDidatticoRepository` (con implementazione `FileSystemMaterialeDidatticoRepository`). Tale separazione favorisce la testabilità del software e garantisce la sostituibilità del meccanismo di persistenza.

6. **Focalizzazione sui Controller di Dominio (GRASP):** L'architettura mantiene separata la logica di business dalle interfacce utente mediante due Use Case Controller specializzati: `MaterialeDidatticoController` (coordinatore delle operazioni su alberi e repository) e `PianoStudiController` (coordinatore della compilazione e validazione dei piani).

3.3.2 Caso d'uso UC6 – Gestione Materiale Didattico, diagrammi di interazione

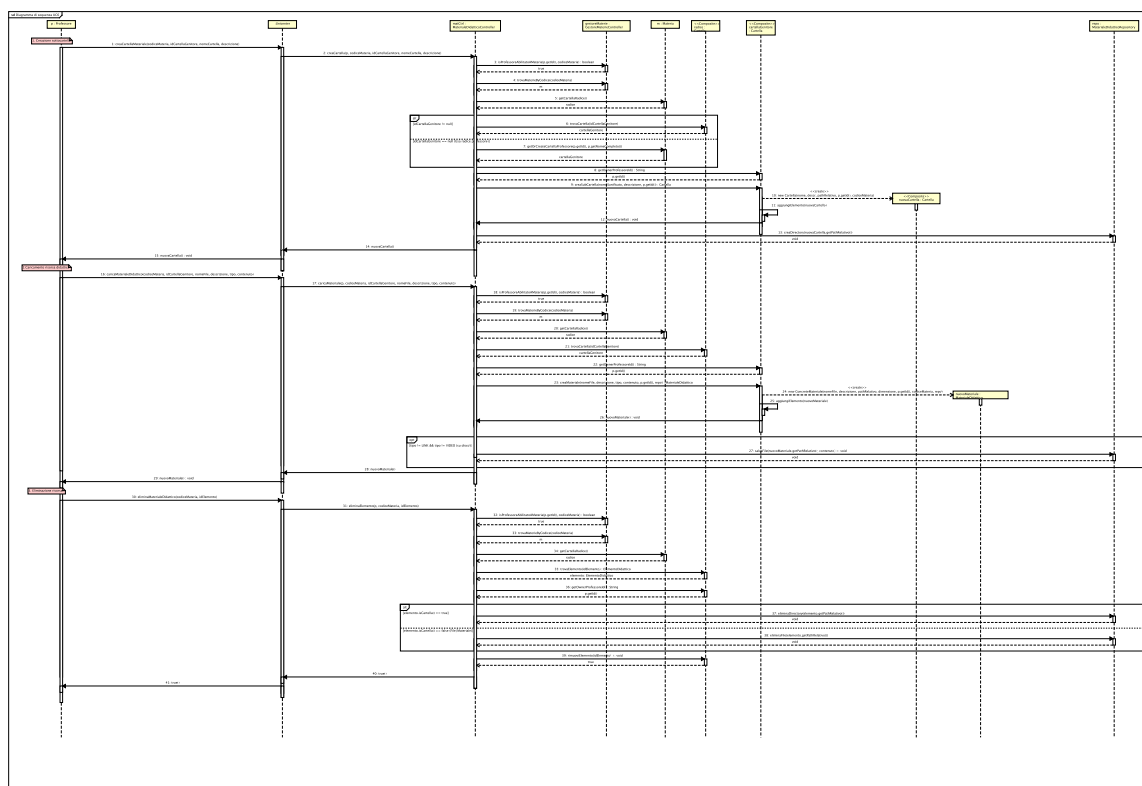


Figura 3.5: Diagramma di Sequenza di Progetto per UC6 – Gestire Materiale Didattico.

Flusso delle Operazioni e Logica Interna

- Inoltro e Controllo Autorizzativo:** Il docente richiede la creazione di una cartella o l'upload di un file. `MaterialeDidatticoController` interroga `GestoreMaterieController.isProfessoreAbilitatoAMateria` per verificare la titolarità di cattedra.
- Verifica di Proprietà sul Nodo Genitore:** Il controller recupera la cartella di destinazione tramite `materia.getCartellaRadice().trovaCartella(idGenitore)` ed esegue il controllo `validaAccessoScritturaProfessore`, assicurandosi che la cartella appartenga all'albero di competenza del docente.
- Creazione Logica nel Composite (Creator):**
 - Per le cartelle: `cartellaGenitore.creaSubCartella(nome, descr, idProf)` istanzia un nuovo nodo `Cartella`.
 - Per i file: `cartellaGenitore.creaMateriale(...)` crea la specifica istanza foglia (es. `Slide`, `DocumentoPdf`, `FileTesto`).
- Persistenza Fisica su Storage (Indirection):** Il controller invoca `repository.salvaFile(path, bytes)` o `repository.creaDirectory(path)`, memorizzando i dati su storage. In caso di errore di I/O, il nodo viene rimosso dal Composite per garantire il rollback e la consistenza.

5. **Eliminazione a Cascata:** In fase di cancellazione, il controller invoca `repository.eliminaDirectory` o `eliminaFile` e rimuove ricorsivamente il nodo tramite `radice.rimuoviElemento(idElemento)`.

Elementi di Design

- **Composite Pattern:** Consente l'annidamento arbitrario di cartelle e risorse senza complessità aggiuntiva nei controller.
- **Creator Pattern:** Le istanze di `Cartella` e `MaterialeDidattico` sono create direttamente dal rispettivo contenitore logico genitore.

3.3.3 Caso d'uso UC9 – Compilazione e Approvazione Piano di Studi, diagrammi di interazione

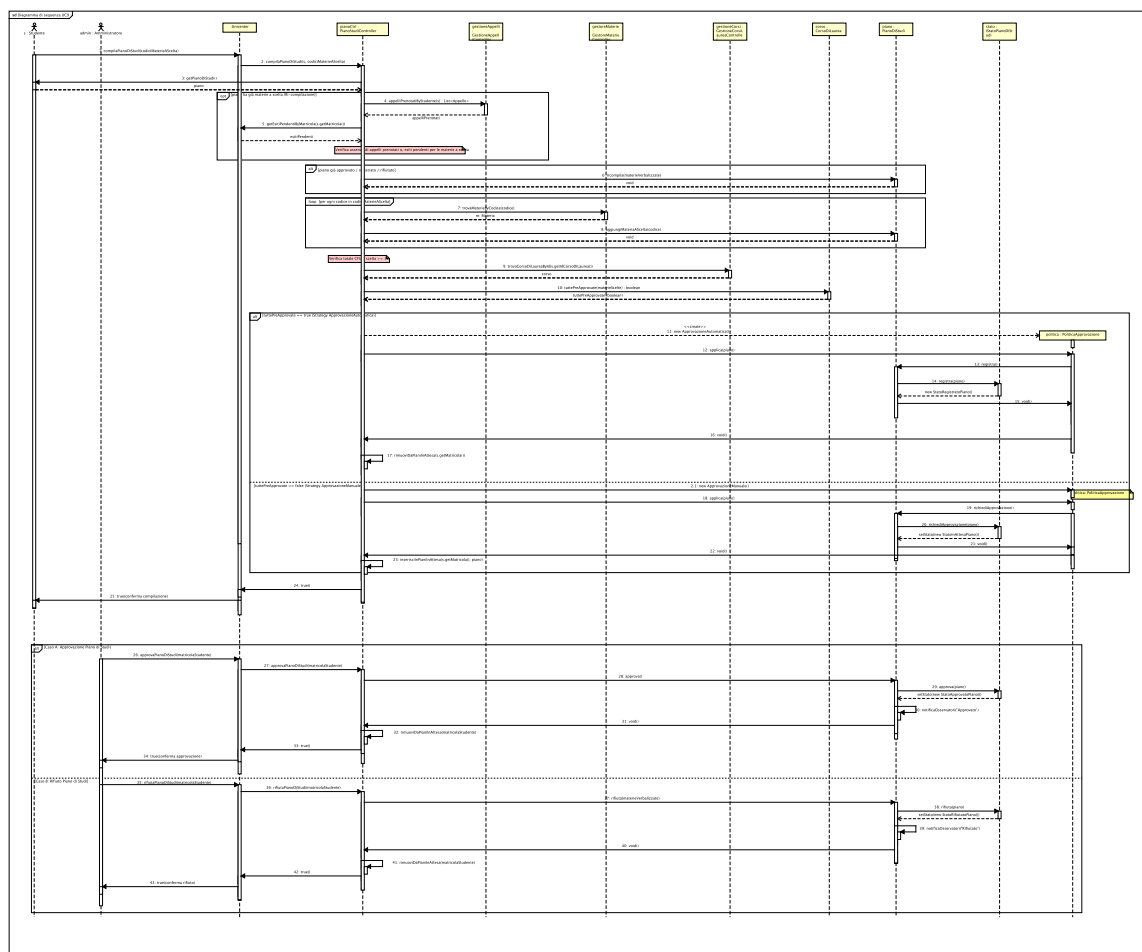


Figura 3.6: Diagramma di Sequenza di Progetto per UC9 – Compilazione Piano di Studi.

Flusso delle Operazioni e Logica Interna

1. **Verifica Vincoli e Impegni Attivi:** `PianoStudiController` accerta che lo studente non abbia prenotazioni d'esame o esiti pendenti relativi alle materie a scelta attualmente a piano.
2. **Tutela del Libretto Universitario:** Il controller recupera la lista delle materie a scelta già verbalizzate e invoca `piano.ricompila(materieVerbalizzate)`, preservando gli esami superati.
3. **Inserimento e Conteggio CFU:** Le nuove discipline selezionate vengono aggiunte al piano. Il controller calcola il totale dei crediti a scelta verificando che sia ≥ 12 CFU.
4. **Selezione e Applicazione della Strategy:** Il controller interroga `CorsoDiLaurea.tuttePreApprovate(materieScelte)`:
 - Se l'esito è positivo, istanzia ed esegue `ApprovazioneAutomatica.appllica(piano)`, impostando lo stato in `StatoApprovatoPiano`.

- Se vi sono materie esterne o non pre-approvate, esegue `ApprovazioneManuale applica(piano)`, impostando lo stato in `StatoInAttesaPiano` e inserendo la richiesta nella coda per l'amministratore.
5. **Risoluzione Manuale:** In caso di approvazione o rigetto da parte dell'amministratore, il controller commuta lo stato del piano rispettivamente in *Approvato* o *Rifiutato*, rimuovendo in quest'ultimo caso le materie non conformi.

Elementi di Design

- **Strategy Pattern:** Disaccoppia la regola di conformità curriculare dal flusso di approvazione, facilitando l'introduzione di future politiche universitarie.
- **State Pattern:** Incapsula i differenti stadi evolutivi del piano di studi.
- **Information Expert:** La classe `CorsoDiLaurea` è l'esperta designata a certificare se una lista di insegnamenti è pre-approvata.

3.3.4 Caso d'uso UC10 – Consultazione e Download Materiale, diagrammi di interazione

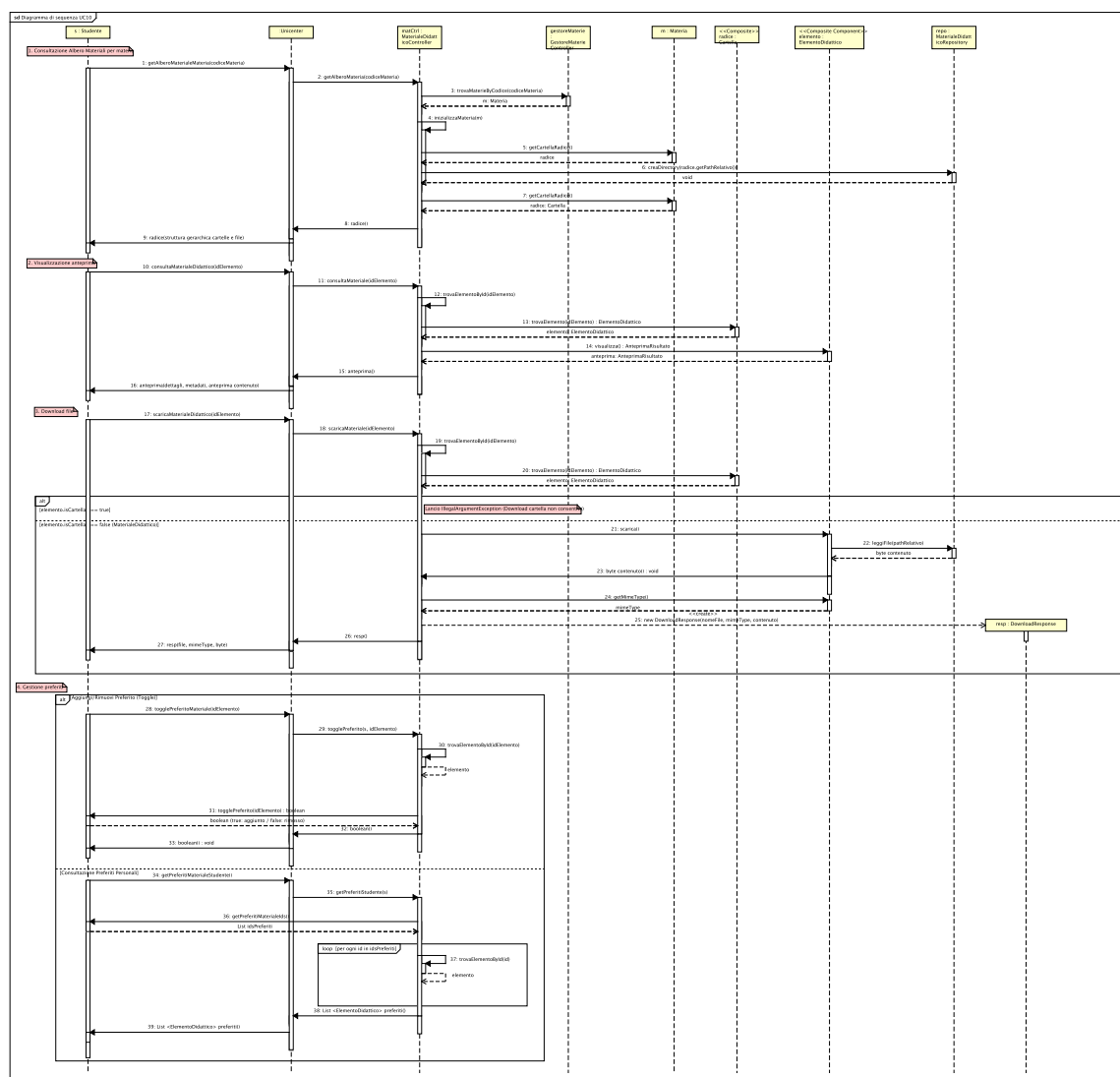


Figura 3.7: Diagramma di Sequenza di Progetto per UC10 – Consultare Materiale Didattico.

Flusso delle Operazioni e Logica Interna

- Recupero del percorso:** Lo studente richiede il percorso di una materia; `MaterialeDidatticoController` inizializza la struttura e restituisce la `Cartella` radice.
- Generazione Polimorfica dell'Anteprima:** Lo studente seleziona un elemento (`consultamateriale(id)`). Il controller recupera l'`ElementoDidattico` e invoca polimorficamente `elemento.visualizza()`. Ciascuna sottoclasse costruisce l'oggetto `AnteprimaRisultato` con il payload testuale o l'URL specifico.
- Streaming del Download:** Lo studente richiede il download del file (`scaricamateriale(id)`). Il controller verifica che l'elemento sia un `MaterialeDidattico` atomico, invoca `materiale.scarica()` estraendo i byte tramite il repository e incapsula il risultato in una `DownloadResponse`.

4. **Gestione dei Preferiti:** Lo studente invoca `togglePreferito(id)`; il controller delega all'entità **Studente** l'inserimento o la rimozione dell'ID dalla propria collezione personale.

Elementi di Design

- **Polymorphism:** Eliminazione totale di istruzioni decisionali sul tipo di file per la visualizzazione dell'anteprima e il download.
- **Composite Navigation:** Ricerca trasparente degli elementi all'interno di strutture ad albero mediante ricorsione interna.

3.3.5 Diagramma delle Classi di Progetto e Architettura Globale

Il diagramma delle classi di progetto illustra la struttura statica del sistema al completamento della terza iterazione.

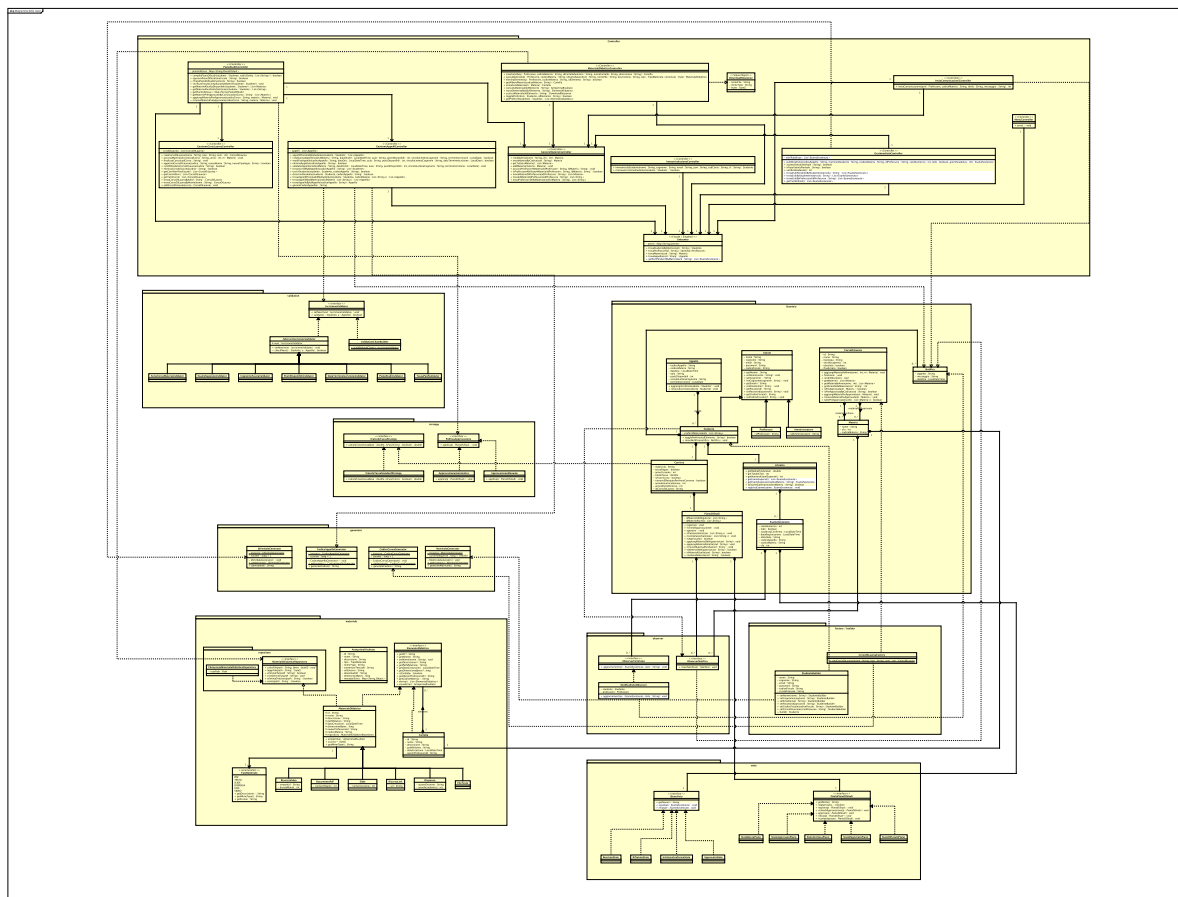


Figura 3.8: Diagramma delle Classi di Progetto – Terza Iterazione.

Il diagramma delle classi relativo alla terza iterazione consolida l'architettura a livelli del sistema, integrando i controllori applicativi, le entità di dominio e una serie di design pattern orientati alla flessibilità e al disaccoppiamento:

- **Strato Controller e Facade:** I controllori specifici (*GestioneAppelliController*, *GestioneVotoController*, *MaterialeDidatticoController*, *PianoStudiController*, ecc.) coordinano la logica applicativa interfacciandosi con il componente *Unicenter*, che implementa il pattern *Facade* (e *Singleton*) per fornire un punto di accesso unificato ai dati e ai servizi di sistema.
- **Modello di Dominio:** Modella la gerarchia degli utenti (*Utente*, specializzato in *Studente*, *Professore* e *Amministratore*) e le relative carriere universitarie, articolate in *Carriera*, *Libretto*, *PianoDiStudi*, *CorsoDiLaurea*, *Materia* e *Appello*.
- **Design Pattern Comportamentali:**

- *Chain of Responsibility* (package `validation`): incapsula le regole per la prenotazione agli appelli (`IscrizioneValidator` e le sue specializzazioni: tasse, propedeuticità, disponibilità posti, ecc.), assemblate dinamicamente tramite `ValidationChainBuilder`.
- *State* (package `state`): modella il ciclo di vita del piano di studi (`IStatoPianoDiStudi`) e l'evoluzione dell'esito d'esame (`IStatoVoto`), garantendo transizioni robuste e coerenti tra gli stati.
- *Strategy* (package `strategy`): astrae gli algoritmi per il calcolo delle tasse universitarie (`ICalcoloTasseStrategy`) e le politiche di approvazione dei piani di studio (`PoliticaApprovazione`).
- *Observer* (package `observer`): disaccoppia la generazione e la ricezione di avvisi ed esiti di voto (`ObserverEsitoVoto`, `ObserverNotifica`).
- **Design Pattern Strutturali e Creazionali:** Il pattern *Composite* (package `materiale`) struttura gerarchicamente le risorse didattiche (`Cartella` e `MaterialeDidattico` come implementazioni di `ElementoDidattico`), supportato dal disaccoppiamento della persistenza tramite `MaterialeDidatticoRepository`. L'istanziamento controllato degli oggetti e la generazione univoca degli identificativi sono infine delegate a pattern creazionali quali *Builder* (`StudenteBuilder`), *Factory* (`CorsoDiLaureaFactory`) e generatori in forma di *Singleton* (`MatricolaGenerator`, `CodiceAppelloGenerator`, ecc.).

3.4 Testing

3.4.1 Strategia di Testing

Durante la terza iterazione, la suite di collaudo automatizzato è stata ulteriormente estesa per certificare le funzionalità avanzate di gestione della didattica digitale, la compilazione assistita dei piani di studio e la progressione della carriera accademica. L'architettura del software si è arricchita con nuovi pattern fondamentali: l'organizzazione gerarchica e ricorsiva delle risorse didattiche mediante il pattern **Composite** (**Cartella**, **ElementoDidattico**), la gestione polimorfica dei formati e delle anteprime con **Polymorphism** (**MaterialeDidattico** e le specializzazioni concrete: **DocumentoPdf**, **Slide**, **Dispensa**, **FileTesto**, **RisorsaVideo**, **RisorsaLink**), il disaccoppiamento delle politiche di approvazione dei piani di studio tramite **Strategy** (**PoliticaApprovazione**, con strategie **ApprovazioneAutomatica** e **ApprovazioneManuale**), la persistenza su disco segregata tramite **Pure Fabrication / Repository** (**FileSystemMaterialeDidatticoRepository**), e il ciclo di vita del rinnovo d'iscrizione e passaggio ad anni successivi/fuori corso in **ImmatricolazioneController**.

La strategia di collaudo del dominio nell'Iterazione 3 si struttura sui seguenti quattro livelli di verifica:

1. **Test Strutturali e Ricorsivi sul Pattern Composite (JUnit 5):** verifica dell'alberatura ad albero per cartelle e file didattici, calcolo ricorsivo e consistente della dimensione totale in byte occupata dai rami e propagazione a cascata delle operazioni di cancellazione e ricerca.
2. **Test Polimorfici di Anteprima e Sicurezza del File System (Repository Testing):** validazione delle implementazioni concrete di **MaterialeDidattico**, certificando il corretto calcolo dei MIME-type, la fruizione dei contenuti (testuali, binari, streaming/link) e la protezione rigorosa contro attacchi di tipo *Path Traversal* sul file system locale.
3. **Test delle Politiche di Approvazione e Integrità del Libretto (Strategy & Business Rules):** verifica del flusso di compilazione del piano di studi (UC9), accertando che il passaggio tra approvazione automatica e manuale avvenga in accordo con l'offerta del corso di laurea e che il rifiuto del piano salvaguardi inderogabilmente le materie a scelta già verbalizzate nel **Libretto**.
4. **Test di Carriera, Transizione ad Anni Successivi e Mocking Temporale:** collaudo deterministico con **ClockProvider** del workflow di rinnovo iscrizione annuale, differenziando lo stato di studente *In Corso* da quello *Fuori Corso* (con applicazione automatica della sovrattassa) e verificando il blocco in presenza di debiti contributivi pregressi.

Sintesi delle Scelte Architettureali e di Design nel Testing dell'Iterazione 3

1. **Navigazione e Trattamento Uniforme delle Risorse (Composite Pattern Testing):** La classe **CompositeCartellaTest** isola la struttura gerarchica definita

dall'interfaccia `ElementoDidattico`. Il collaudo certifica che i client trattino uniformemente singoli file e intere cartelle, verificando che la computazione della dimensione (`getDimensioneBytes`) e l'anteprima si propaghino correttamente attraverso l'intera alberatura della materia.

2. **Specializzazione del Materiale Didattico e MIME Handling (Polymorphism Testing):** La suite `PolymorphismMaterialeTest` verifica le gerarchie specializzate di `MaterialeDidattico`, assicurando che ogni tipologia fornisca l'anteprima corretta (`AnteprimaRisultato`), il MIME-type standard (es. *"application/pdf"*, *"text/plain"*, *"text/html"*) e le relative caratteristiche aggiuntive (es. numero di lezione per `Slide`, autore e anno accademico per `Dispensa`).
3. **Isolamento dell'I/O su File System e Sicurezza (Repository Pattern Testing):** L'adozione dell'interfaccia `MaterialeDidatticoRepository` permette di confinare l'interazione con il disco in cartelle temporanee isolate create ad-hoc nei test (`Files.createTempDirectory`). La classe `FileSystemRepositoryTest` verifica la corretta scrittura/lettura atomica e intercetta tempestivamente tentativi di evasione dalla sandbox (*Path Traversal Protection*) sollevando `SecurityException`.
4. **Flessibilità dell'Offerta con le Politiche di Approvazione (Strategy Testing):** L'introduzione di `PoliticaApprovazione` separa nettamente la compilazione del piano di studi dal meccanismo di validazione. I test validano che se le materie a scelta sono pre-approvate dal Corso di Laurea venga eseguita la strategia `ApprovazioneAutomatica` (transizione immediata a *Approvato*), mentre materie libere attivino `ApprovazioneManuale` ponendo il piano in *In Attesa* dell'amministratore.
5. **Protezione dei Crediti Acquisiti nel Libretto Universitario:** *Evoluzione del Dominio:* la suite `PianoStudiLibrettoTest` certifica un vincolo fondamentale di non-regressione: in caso di bocciatura o rifiuto di un nuovo piano di studi da parte di un amministratore, le materie a scelta per le quali lo studente ha già superato e verbalizzato l'esame nel proprio Libretto vengono preservate e non possono essere eliminate.
6. **Evoluzione dell'Immatricolazione: Rinnovi Annuali e Rilevazione Fuori Corso:** *Modifica rispetto alle Iterazioni 1 e 2:* la classe `ImmatricolazioneControllerTest` è stata arricchita per validare il ciclo di rinnovo iscrizione. Mediante simulazione temporale con `ClockProvider`, si certifica che un rinnovo dopo il terzo anno di una triennale determini automaticamente il passaggio a *Fuori Corso* con l'addebito della maggiorazione di 300 EUR.

3.4.2 Dettaglio della Test Suite per Caso d'Uso

Di seguito vengono descritte nel dettaglio le singole classi di test relative alla logica di dominio introdotta o estesa nella terza iterazione. Per ciascun caso d'uso vengono illustrati **i casi di test più significativi e rappresentativi**, corredati da una sintetica descrizione delle regole di business e dei vincoli collaudati.

Caso d'uso UC6: Caricamento e Gestione del Materiale Didattico (Docente, Composite & Repository)

Questa suite valida le operazioni del corpo docente per la creazione di cartelle, upload di risorse didattiche, isolamento di cattedra e cancellazione controllata (`MaterialeDidatticoControllerTest`, `FileSystemRepositoryTest`):

- **testProfessoreCreaCartellaECaricaMateriale (UC6 flusso nominale):** Verifica la creazione di una nuova cartella da parte del docente titolare e il caricamento di una risorsa PDF (`TipoMateriale.SLIDE`), accertando la corretta integrazione con l'albero Composite della materia e la persistenza fisica su repository.
- **testProfessoreNonAbilitatoLanciaEccezione (UC6 controllo autorizzativo):** Accerta che un docente privo di abilitazione sulla materia riceva una `SecurityException` se tenta di creare cartelle o caricare documenti didattici per tale insegnamento.
- **testProfessoreNonPuoScrivereInCartellaAltroDocente (Segregazione codocenze):** In presenza di più docenti abilitati alla stessa materia, verifica che un docente non possa creare sottocartelle o inserire file all'interno di cartelle di proprietà di un altro collega senza autorizzazione.
- **testEliminaMaterialeDidattico (UC6 cancellazione e pulizia disco):** Valida la rimozione controllata di un file didattico da parte del docente proprietario, verificando sia la dereferenziazione nell'albero Composite sia l'effettiva eliminazione del binario dal file system.
- **testSalvaELeggiFile (Test Repository):** Verifica la corretta scrittura, verifica di esistenza e riletture byte-a-byte dei dati gestiti da `FileSystemMaterialeDidatticoRepository`.
- **testEliminaFileEDirectory:** Accerta la cancellazione ricorsiva di cartelle e relativi file contenuti dal repository di archiviazione.
- **testPathTraversalProtection (Sicurezza del File System):** Collauda la protezione contro attacchi di risalita directory (es. percorsi contenenti `"../../"`), verificando che il repository sollevi immediatamente `SecurityException`.
- **testLetturaFileInesistente:** Verifica che richieste di lettura di file non presenti sul disco sollevino l'eccezione standard `NoSuchFileException`.

Caso d'uso UC10: Consultazione, Download e Gestione Preferiti (Studente & Polymorphism)

Questa suite certifica la fruizione delle risorse didattiche da parte degli studenti, il polimorfismo delle anteprime e la gestione dei preferiti di studio (`MaterialeDidatticoControllerTest`, `PolymorphismMaterialeTest`):

- **testStudenteConsultaEScarica (UC10 flusso nominale anteprima e download):** Verifica la generazione dell'anteprima corretta (`AnteprimaRisultato`) per un file testuale e il successivo download del flusso di byte con MIME-type conforme (`"text/plain"`).

- **testStudentePreferiti (UC10 gestione segnalibri):** Collauda l'aggiunta e la successiva rimozione di una risorsa didattica dall'elenco dei preferiti dello studente (*toggle*), verificando l'aggiornamento della lista persistita sul profilo.
- **testDocumentoPdfPolimorfismo:** Collauda la classe concreta `DocumentoPdf`, accertando che restituisca il MIME-type "*application/pdf*", l'URL per il download e il corretto array di byte.
- **testFileTestoPolimorfismo:** Verifica che l'anteprima di un `FileTesto` consenta la visualizzazione in chiaro del testo senza richiedere il download obbligatorio del file.
- **testSlideEDispensaPolimorfismo:** Valida gli attributi specializzati per `Slide` (numero lezione) e `Dispensa` (autore docente e anno accademico di riferimento).
- **testRisorsaLinkPolimorfismo:** Verifica che le risorse esterne (`RisorsaLink`) esponcano il MIME-type "*text/html*" e incapsulino correttamente l'URL target di reindirizzamento.
- **testVisualizzaCartella (Anteprima Composite):** Accerta che l'invocazione del metodo `visualizza()` su una cartella restituisca un'anteprima di tipo "*application/x-directory*" con il riepilogo del contenuto.

Caso d'uso UC9: Compilazione Piano di Studi e Politiche di Approvazione (Strategy & State)

Questa suite valida il workflow di compilazione del piano di studi, l'applicazione del pattern **Strategy** per l'approvazione e la salvaguardia della carriera accademica (`PianoStudiLibrettoTest`, classi del package `strategy`):

- **testPianoInAttesa (UC9 sottomissione e Strategy manuale):** Verifica che l'inserimento di materie a scelta non pre-approvate attivi la strategia di approvazione manuale, ponendo il piano nello stato "*In Attesa*" dell'approvazione dell'amministratore.
- **testApprovazionePianoMantieneMaterie (UC9 approvazione amministratore):** Accerta che l'approvazione da parte dell'amministratore transiti lo stato del piano in "*Approvato*", confermando l'accreditamento di tutte le materie a scelta selezionate.
- **testRifiutoPianoRimuoveMaterieAScelta (UC9 gestione rifiuto piano):** Verifica che il rifiuto del piano da parte dell'amministratore transiti lo stato in "*Rifiutato*" e ripulisca il piano dalle materie a scelta non autorizzate.
- **testRifiutoPianoMantieneMaterieVerbalizzate (Regola di protezione carriera):** Valida il vincolo per cui, anche a fronte del rifiuto di un nuovo piano di studi, le materie a scelta precedentemente superate e già verbalizzate nel `Libretto` universitario dello studente rimangono tassativamente vincolate al piano.
- **testAggiuntaENavigazioneComposite (Composite Cartella):** Verifica l'aggiunta annidata di sottocartelle e risorse didattiche, certificando il calcolo aggregato e ricorsivo dei byte totali occupati dai nodi foglia.

- **testRimozioneElementoComposite:** Accerta la corretta rimozione ricorsiva di elementi intermedi e il conseguente ricalcolo automatico della dimensione complessiva del ramo Composite.

Test per Gestione Carriera, Rinnovo Annuale e Fuori Corso (ImmatricolazioneController)

Questa suite certifica l'evoluzione del ciclo di vita dello studente per gli anni successivi al primo, con controllo delle finestre temporali e calcolo delle maggiorazioni di mora (ImmatricolazioneControllerTest, LibrettoTest):

- **rinnovalIscrizioneStudente_successo_inCorso (Progressione anno nominale):** Verifica il rinnovo per l'anno successivo (es. passaggio dal 1° al 2° anno di una Triennale), accertando il mantenimento dello status *In Corso*, l'addebito della tassa base di rinnovo e l'impostazione delle tasse in stato non pagato.
- **rinnovalIscrizioneStudente_successo_diventaFuoriCorso (Rilevazione Fuori Corso):** Collauda il rinnovo al 4° anno di una laurea triennale, verificando che il sistema identifichi automaticamente lo studente come *Fuori Corso* e applichi la maggiorazione di 300 EUR sull'importo delle tasse.
- **rinnovalIscrizioneStudente_tasseNonPagate_lanciaIllegalStateException:** Verifica che la presenza di pendenze contributive relative agli anni precedenti impedisca categoricamente il rinnovo dell'iscrizione.
- **rinnovalIscrizioneStudente_giaRinnovato_lanciaIllegalStateException:** Accerta che non sia consentito effettuare più rinnovi all'interno dello stesso anno accademico solare.
- **rinnovalIscrizioneStudente_stessoAnnoImmatricolazione_lanciaDataNonValidaException:** Impedisce il rinnovo agli studenti appena immatricolati nello stesso anno solare.
- **rinnovalIscrizioneStudente_rinnoviMultiAnnoConsecutivi_successo:** Collauda mediante isolamento temporale (ClockProvider) la simulazione completa dell'intera carriera universitaria: immatricolazione (2026), 1° rinnovo (2027), 2° rinnovo (2028) e passaggio a fuori corso (2029).
- **getStatoRinnovoStudente_studenteIdoneo_restituisceDatiCorretti:** Valida la funzione di ispezione dello stato di rinnovo, verificando che la mappa restituita indichi idoneità, apertura finestra e assenza di motivi bloccanti.
- **testRegistraETrovaEsameSuperato (LibrettoTest):** Verifica la corretta memorizzazione nel Libretto degli esami superati con voto e data, e l'intercettazione dell'eccezione `EsameNonTrovatoException` in caso di ricerca di materie non verbalizzate.

Elenco delle figure

1.1	Modello di Dominio – Iterazione 1.	9
1.2	Diagramma di sequenza di sistema (SSD) – UC8: Immatricolazione Studente.	11
1.3	Diagramma di sequenza di sistema (SSD) – UC1: Gestione Appelli.	13
1.4	Diagramma di sequenza di sistema (SSD) – UC2: Iscrizione ad un Appello.	15
1.5	Diagramma di Sequenza di Progetto – UC8: Immatricolazione Studente.	17
1.6	Diagramma di Sequenza di Progetto – UC1: Creazione Nuovo Appello.	19
1.7	Diagramma di Sequenza di Progetto – UC2: Iscrizione ad un Appello.	21
1.8	Diagramma delle Classi di Progetto – Iterazione 1.	23
2.1	Modello di Dominio Concettuale – Seconda Iterazione.	35
2.2	SSD per il caso d’uso UC3 – Accettare/Rifiutare Voto.	37
2.3	SSD per il caso d’uso UC4 – Creazione Corso di Laurea.	40
2.4	SSD per il caso d’uso UC5 – Creazione materie e Finalizzazione Corso di Laurea.	42
2.5	SSD per il caso d’uso UC7 – Invio Comunicazioni.	45
2.6	Diagramma di Sequenza di Progetto per UC3 – Accettare/Rifiutare Voto.	49
2.7	Diagramma di Sequenza di Progetto per UC4 – Creazione Corso di Laurea.	51
2.8	Diagramma di Sequenza di Progetto per UC5 – Configurazione e Finalizza- zione Corso.	53
2.9	Diagramma di Sequenza di Progetto per UC7 – Invio Comunicazioni.	56
2.10	Diagramma delle Classi di Progetto – Seconda Iterazione.	58
3.1	Modello di Dominio Concettuale – Terza Iterazione.	72
3.2	SSD per il caso d’uso UC6 – Gestire Materiale Didattico.	73
3.3	SSD per il caso d’uso UC9 – Compilazione Piano di Studi.	76
3.4	SSD per il caso d’uso UC10 – Consultare Materiale Didattico.	78
3.5	Diagramma di Sequenza di Progetto per UC6 – Gestire Materiale Didattico.	83
3.6	Diagramma di Sequenza di Progetto per UC9 – Compilazione Piano di Studi.	85
3.7	Diagramma di Sequenza di Progetto per UC10 – Consultare Materiale Didattico.	87

3.8	Diagramma delle Classi di Progetto – Terza Iterazione.	89
-----	--	----

Elenco delle tabelle

1.1	Contratto dell'operazione <code>immatricolaStudente</code>	12
1.2	Contratto dell'operazione <code>creaNuovoAppello</code>	14
1.3	Contratto dell'operazione <code>iscrivitiAdAppello</code>	16
2.1	Contratto di sistema dell'operazione <code>accettaVoto</code>	38
2.2	Contratto di sistema dell'operazione <code>rifiutaVoto</code>	39
2.3	Contratto di sistema dell'operazione <code>creaCorsoDiLaurea</code>	41
2.4	Contratto di sistema dell'operazione <code>associaMateria</code>	43
2.5	Contratto di sistema dell'operazione <code>finalizzaCorso</code>	44
2.6	Contratto di sistema dell'operazione <code>inviaComunicazione</code>	46
3.1	Contratto di sistema dell'operazione <code>creaCartella</code>	74
3.2	Contratto di sistema dell'operazione <code>caricaMateriale</code>	75
3.3	Contratto di sistema dell'operazione <code>compilaPianoDiStudi</code>	77
3.4	Contratto di sistema dell'operazione <code>consultaMaterialeDidattico</code>	79
3.5	Contratto di sistema dell'operazione <code>scaricaMaterialeDidattico</code>	80